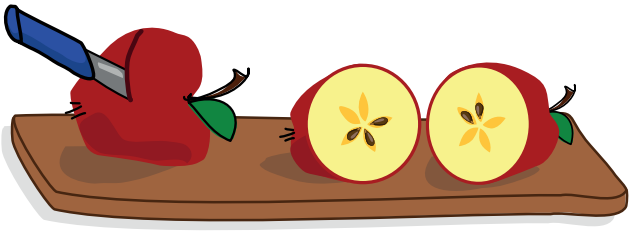
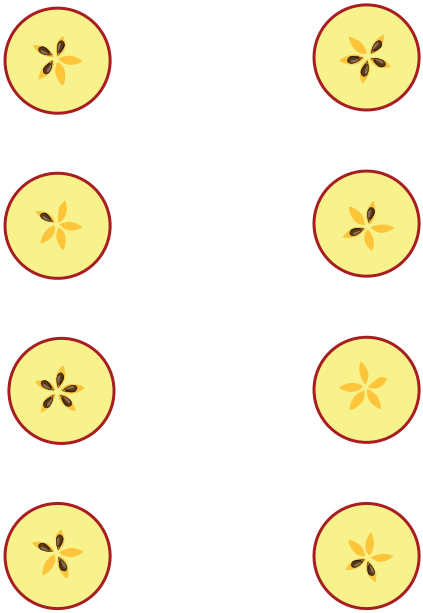
Äpfel halbieren

Äpfel kann man in eine obere und untere Hälfte teilen. Einige Apfelkerne bleiben in der oberen Hälfte, die anderen in der unteren Hälfte. An den Löchern und Kernen sieht man, dass die Hälften zusammen passen:

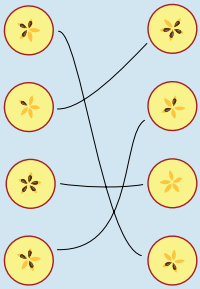


Gala halbiert vier Äpfel. Sie legt die oberen Hälften links und die unteren Hälften rechts untereinander.

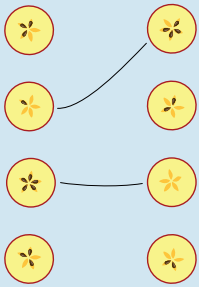
Welche Apfelhälften passen zusammen? Ordne die Apfelhälften einander zu.

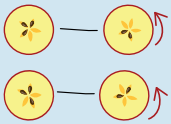


So ist es richtig:



Jeder Apfel hat 5 Apfelkerne. Zwei zueinander passende Apfelhälften müssen also insgesamt 5 Kerne haben. Diese Hälften können damit einfach zugeordnet werden:





Die übrigen vier Hälften sind nicht so einfach zuzuordnen. Die beiden oberen Hälften haben jeweils 3 Kerne, die beiden unteren Hälften haben jeweils 2 Kerne. Deshalb schauen wir uns die Muster der Apfelkerne genauer an. Denn wenn zwei Hälften zusammen passen, passen auch die Muster zusammen. Um das zu sehen, kann es aber nötig sein, die Hälften zu drehen.

Dann lassen sich die Hälften so zuordnen wie im Bild: Oben hat die obere Hälfte drei Kerne direkt hintereinander und danach zwei Löcher (K-K-K-L-L), die untere Hälfte hat drei Löcher direkt hintereinander und danach zwei Kerne (L-L-L-K-K): die Hälften passen zusammen. Auch unten passen die beiden Hälften zusammen: Das Muster der oberen Hälfte lautet (wenn man oben beginnt und im Uhrzeigersinn weitermacht) K-L-K-L-K, das Muster der unteren Hälfte (wenn man unten beginnt) L-K-L-K-L.

Das ist Informatik!
Bei der Erklärung der richtigen Antwort haben wir gesehen: Wenn zwei Hälften zusammen passen, passen nicht nur die Anzahlen der Kerne zusammen, sondern auch die Reihenfolgen der Kerne und Löcher (also der leeren Kernfächer). Für die richtige Zuordnung der Hälften muss man also auch diese Reihenfolgen betrachten. Es genügt nicht zu wissen, wie viele Kerne in jeder Hälfte sind.

Bei Problemen, die mit Hilfe von Computerprogrammen gelöst werden sollen, stellen sich ähnliche Fragen. Informatikerinnen und Informatiker müssen sich Gedanken machen, wie die Informationen, die das Programm berücksichtigen soll, als Daten beschrieben werden. Dabei wird oft versucht, es so einfach wie möglich zu machen. Einfache Programme sind nämlich weniger anfällig für Fehler. Beim Apfelhälften-Problem in dieser Biberaufgabe schien es zunächst ausreichend zu sein, die Hälften allein durch die Anzahl der Kerne zu beschreiben. Doch dann wurde klar, dass das nicht in allen Fällen genügt. Zur Beschreibung der Apfelhälften in einem Computerprogramm muss es also möglich sein, eine Reihenfolge zu beschreiben. Das geht zum Beispiel mit Hilfe der Datenstruktur *Liste,* die in den meisten Programmiersprachen zur Verfügung steht.

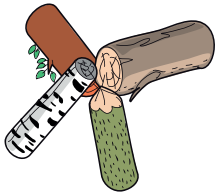
Foto

Der Biber hat gerade ein Foto gemacht.



Welches der vier Fotos ist es?

A



B



C



D



Antwort D ist richtig:



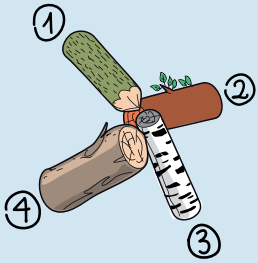
Die Baumstämme, die der Biber fotografiert hat, sind im Kreis angeordnet. Um herauszufinden, welches Foto das richtige ist, betrachten wir die Reihenfolge der Baumstämme in dieser Anordnung. Wir wählen einen Baumstamm aus (z.B. den angespitzten Baumstamm) und geben ihm die Nummer 1. Dann bestimmen wir, welcher Baumstamm links daneben ist und geben ihm die Nummer 2. Das machen wir solange, bis alle Baumstämme eine Nummer haben. In der Situation, die der Biber fotografiert hat, haben die Stämme also diese Reihenfolge: 1 (angespitzter Stamm) – 2 (brauner Stamm mit Blättern) – 3 (Birkenstamm) – 4 (dicker brauner Stamm).



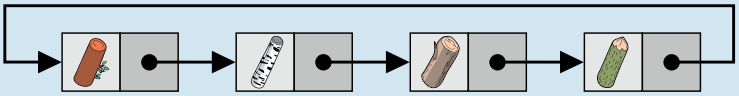
Nun betrachten wir die Reihenfolge der Stämme in den Fotos A bis D. Dabei beginnen wir wie oben mit dem angespitzten Baumstamm 1 und gehen immer nach links:

- Foto A: 1 – 3 – 2 – 4
- Foto B: 1 – 4 – 3 – 2
- Foto C: 1 – 3 – 4 – 2
- Foto D: 1 – 2 – 3 – 4

Nur Foto D zeigt die richtige Reihenfolge.



Das ist Informatik!
In dieser Biberaufgabe wird die Reihenfolge der Baumstämme betrachtet. Was bei wenigen *Elementen* (hier vier Baumstämmen) durch einfaches „Hinsehen" und Vergleichen der Nachbarpaare möglich ist, erfordert bei Problemen mit viel mehr Elementen ein automatisiertes Vorgehen. In einem Computerprogramm, das benachbarte Elemente verarbeiten soll, könnten die Elemente in einer geeigneten Datenstruktur wie einer verketteten Liste gespeichert werden:



In einer *verketteten Liste* wird jedes Datenelement in einem einzelnen Knoten gespeichert. Zusätzlich ist in jedem Knoten ein *Verweis* auf den nächsten Knoten in der Liste gespeichert. Enthält der letzte Knoten einen Verweis auf den ersten Knoten, so handelt es sich um eine ringförmige Datenstruktur. Das ist im Beispiel wichtig, damit man bei jedem beliebigen Baumstamm starten und die Liste durchlaufen kann.

Blumenstrauß

Florian verkauft Blumensträuße. Jeden Blumenstrauß bindet Florian nach dieser Anleitung:

1. Nimm die erste Blume aus Eimer A.

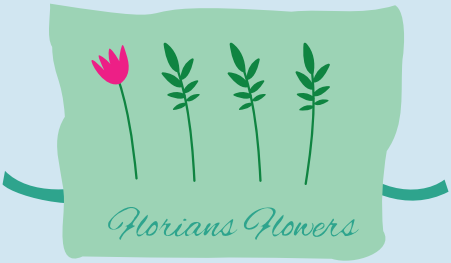
2. Wenn die erste Blume eine Margarite ist, nimm noch eine Margarite .

3. Nun nimm solange einen Zweig aus Eimer B, bis der Blumenstrauß 4 Teile hat. Fertig!

Hilf Florian: Folge der Anleitung und wähle Blumen und Zweige für einen Strauß aus.

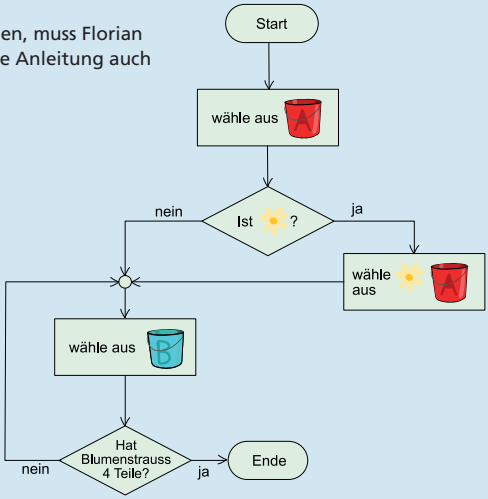


So ist es richtig:
Es gibt zwei richtige Lösungen:





Um die Blumensträuße korrekt zu binden, muss Florian die Anleitung befolgen. Wir können die Anleitung auch mit einem Diagramm beschreiben:



Nachdem Florian die erste Blume aus Eimer A gewählt hat, folgt eine Entscheidung, die abhängig von der ersten Blume ist. Entweder nimmt er noch eine Margarite oder folgt dem „nein"-Pfeil und nimmt einen Zweig .

Dann überprüft er, ob er schon vier Teile hat. Wenn nicht, folgt er dem „nein"-Pfeil und muss einen weiteren Zweig nehmen und dann die Anzahl der Teile wieder überprüfen.

Wenn er also zuerst eine Margarite nimmt, wird er noch eine Margarite nehmen und dann zweimal einen Zweig nehmen. Wenn er aber zuerst eine Tulpe nimmt, wird er danach direkt zu „wähle aus Eimer B" gehen aus Eimer B solange einen Zweig nehmen, bis er 4 Teile hat – also insgesamt 3 Zweige nehmen.

Das ist Informatik!
Die Anleitung fürs Binden des Blumenstraußes ist klar und könnte von einer Maschine ausgeführt werden. In der Informatik nennt man dies einen *Algorithmus.* Die Anleitung benutzt einige Anweisungen, die auch in Computerprogrammen üblich sind:

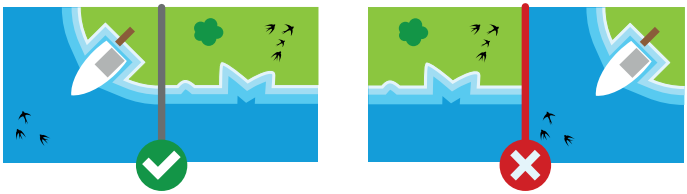
- Die erste Anweisung ist eine zufällige Auswahl aus einer Menge von Objekten.
- Die zweite Anweisung nennt man eine *bedingte Anweisung* (engl. *if-statement*) oder eine *Verzweigung:* Denn du musst aus zwei oder mehr Möglichkeiten auswählen.
- Die dritte Anweisung sieht relativ einfach aus, muss aber in einem Computerprogramm gut strukturiert werden. Der innere Teil der Anweisung (selbst wieder eine Anweisung: „Nimm einen Zweig aus Eimer B") muss mehrmals ausgeführt werden, bis der Blumenstrauß aus 4 Teilen besteht. Die Ausführung der inneren Anweisung wird also solange wiederholt, bis die Bedingungen „Der Blumenstrauß besteht aus 4 Teilen." erfüllt ist. Eine solche *Wiederholungs-Anweisung* nennt man auch *Schleife.*

Ein Algorithmus kann unterschiedlich dargestellt werden. In dieser Biberaufgabe ist Florians „Blumenstrauß-Algorithmus" als Anleitung in natürlicher Sprache formuliert. In der Lösungserklärung ist er als *Programmablaufplan* dargestellt.

Floristik ist eine Handwerkskunst. Es existieren Traditionen und Regeln, wie ein Blumenstrauß oder ein Kranz gebunden wird. Dies ist ein Beispiel dafür, dass Anleitungen oder Algorithmen in vielen Lebensbereichen vorkommen, nicht nur in der Informatik.

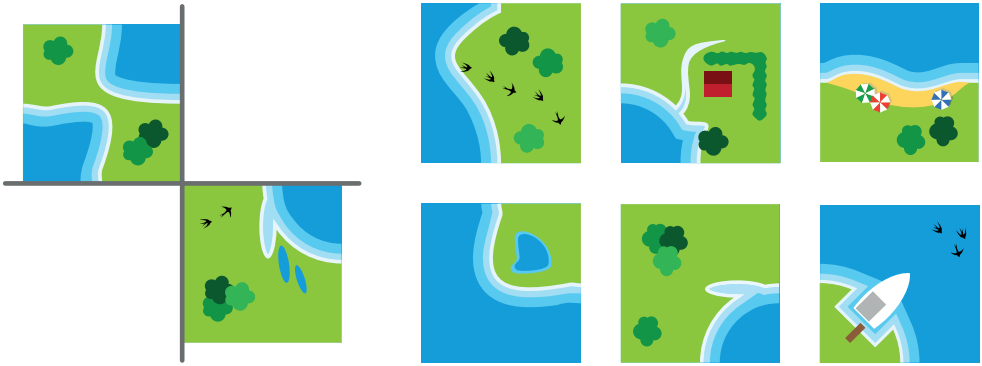
Wasser – Land

Edu hat ein neues Spiel. Das Spiel hat Kärtchen mit Wasser und Land. Edu legt die Kärtchen so aneinander, dass sie passen: Land an Land, Wasser an Wasser. Dann entstehen schöne Landschaften.



Edu legt zwei Kärtchen hin und lässt zwei Lücken.

Welche Kärtchen passen in die Lücken?



So ist es richtig:



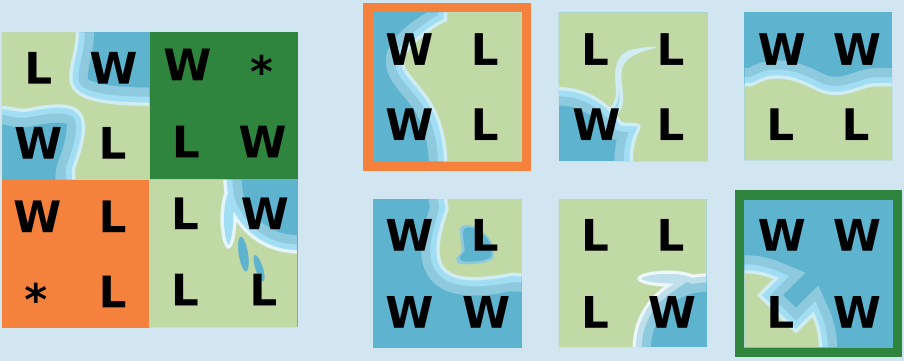
Das Kärtchen mit dem Boot und das mit den fünf Vögeln passen in die Lücken: Überall liegt Wasser an Wasser und Land an Land. Von den sechs möglichen Kärtchen passen nur diese beiden Kärtchen in die Lücken.

Nur wenn man die Kärtchen drehen dürfte, würden noch weitere Kärtchen in die Lücken passen.

Das ist Informatik!
Schauen wir uns Edus Kärtchen genauer an. Alle Kärtchen können in vier Bereiche geteilt werden. Dabei zeigen alle äußeren Ränder jedes Bereichs entweder Land oder Wasser.



Es gibt also nur zwei verschiedene Arten von Bereichen, Wasser (W) oder Land (L). Zwei Kärtchen passen aneinander, wenn benachbarte Bereiche die gleiche Art haben. Daher können wir für je drei Bereiche der beiden Lücken die erforderliche Art eintragen. Der vierte Bereich kann Wasser oder Land sein, deswegen tragen wir „*“ ein.



Auf diese Weise erstellen wir für jede Lücke ein Muster. Die Kärtchen, die die Lücken füllen sollen, müssen zu diesen Mustern passen: Wo ein Muster einen Buchstaben hat, muss das Kärtchen den gleichen Buchstaben haben; wo das Muster einen Stern hat, kann das Kärtchen irgendeinen Buchstaben haben. Zu jedem Muster passt nur ein Kärtchen auf der rechten Seite.

Wir haben eine Eigenschaft der Kärtchen entdeckt. Diese Entdeckung haben wir genutzt, um sie durch eine Anordnung der Zeichen L und W zu beschreiben. Durch diesen Schritt haben wir die in den Bildern enthaltenen Informationen erheblich reduziert. Wir konzentrieren uns auf die Informationen, die zur Lösung dieser Aufgabe erforderlich sind. Informatiker würden sich auf die Anordnung der Zeichen in den Bildern beziehen. Durch die Reduktion der Bilder auf die Bereichsarten L und W entsteht ein Modell für die fehlenden Kärtchen. *Modellierung* bedeutet *Abstraktion* (oder Vereinfachung), und Abstraktion reduziert die Information. Computer müssen mit Modellen von der Realität arbeiten. Bei der Erstellung solcher Modelle muss darauf geachtet werden, dass wichtige Eigenschaften der Realität nicht verloren gehen.

Aylas Regenschirm

Das ist Aylas Regenschirm:



Eines der vier Bilder zeigt Aylas Regenschirm. Welches?



A



B



C



D

Antwort C ist richtig:
Jedes Muster auf Aylas Regenschirm kommt genau einmal vor.



Um das korrekte Bild zu finden, vergleichen wir nacheinander jedes der Bilder mit Aylas Regenschirm:
- Wir wählen das Muster, welches am weitesten links ist, und suchen dessen Position auf Aylas Regenschirm.
- Wir prüfen, ob die angrenzenden Muster dieselben sind wie die auf Aylas Regenschirm.

Antwort	A	B	C	D
Antwortbild				
Aylas Regenschirm				

Jedes Antwortbild zeigt eine Folge von fünf Mustern. Ob eine dieser Folgen mit der vollständigen Reihenfolge aller zehn Muster von Aylas Regenschirm übereinstimmt, können wir nicht wissen. Bild C zeigt aber als einziges eine Folge, die vollständig mit einer Folge von fünf Mustern auf Aylas Regenschirm übereinstimmt. Aus diesem Grund kann nur Bild C Aylas Regenschirm zeigen. Alle anderen Bilder zeigen Musterfolgen, die nicht oder nur teilweise mit Aylas Regenschirm übereinstimmen. Diese Bilder können also nicht Aylas Regenschirm zeigen.

Das ist Informatik!
In den Antwortmöglichkeiten ist jeweils nur ein Teil der Musterfolge abgebildet. Obwohl sie nur eine Teilinformation enthalten, können wir feststellen, welches der vier Bilder Aylas Regenschirm zeigt: Ein Bild zeigt nur dann Aylas Regenschirm, wenn die Musterfolge vollständig in der Musterfolge von Aylas Regenschirm enthalten ist.

Das gleiche Prinzip wie bei der „Regenschirm-Mustersuche" wird bei der Suche in einem Textdokument angewendet. Der Computer sucht mit gegebenen Teilinformationen (Suchwort) nach passenden *Zeichenketten* im Dokument. Eine Zeichenkette ist eine Folge von Zeichen (z.B. Buchstaben, Ziffern, Sonderzeichen). Dabei gilt bei der Suche:

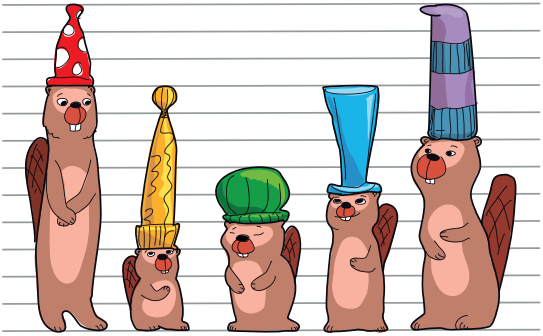
- Je länger das Suchwort, desto weniger mögliche Übereinstimmungen gibt es und desto grösser ist die Chance, die gesuchte Stelle im Dokument zu finden.

- Je kürzer das Suchwort, desto mehr mögliche Übereinstimmungen ergibt die Suche und desto ungenauer ist die Suche.

Um das Durchsuchen zu verbessern, wurden verschiedene Suchverfahren (oder *Suchalgorithmen*) entwickelt. Sie sollen möglichst schnell eine genaue Suche durchführen und ein passendes Resultat liefern. Diese Suchalgorithmen werden ständig weiterentwickelt und können riesige Datenmengen in sehr kurzer Zeit durchsuchen (z.B. Internetsuchmaschinen verwenden solche Suchalgorithmen).

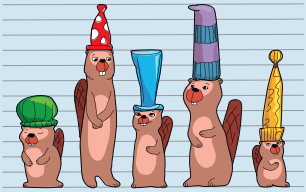
Neue Hüte

Die Biber haben neue Hüte.
Die Hüte sind unterschiedlich hoch.

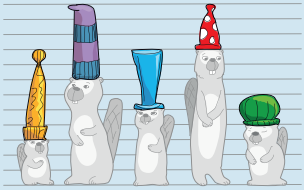


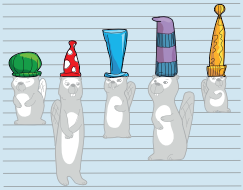
Sortiere die Hüte nach der Höhe. Der Biber mit dem kürzesten Hut soll links stehen.

So ist es richtig:
So sind die Hüte richtig sortiert; sie werden von links nach rechts immer höher:



Beim Umordnen der Biber achten wir nur auf die Hüte. Dann ist es viel einfacher, sie der Höhe nach zu sortieren.





Das ist Informatik!
Aus einer großen Menge von Dingen können wir bestimmte Dinge besser heraussuchen, wenn die Menge sortiert ist: Wenn Werkzeuge nach Größe sortiert sind, lässt sich das passende Werkzeug leichter finden. Weil die Einträge in einem Wörterbuch alphabetisch sortiert sind, kann man die Seite mit dem gesuchten Wort schneller finden. In dieser Biberaufgabe sollst du die Biber sortieren, und zwar nach der Höhe ihrer Hüte. Die Schwierigkeit dabei ist, dass die Eigenschaft „Höhe der Hüte“ nicht leicht erkennbar ist. Deshalb muss man sehr genau hinsehen.

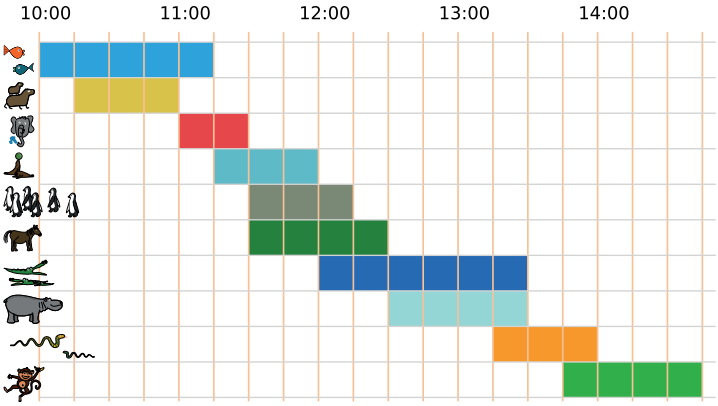
Beim Sortieren ist es also erstens wichtig, die Eigenschaft klar festzulegen, nach der sortiert werden soll. Zweitens müssen die Werte dieser Eigenschaft sortierbar sein. Nach Eigenschaften, die in Zahlen ausgedrückt werden (wie z.B. Höhe, Länge, Gewicht, ...) kann man sortieren: Für zwei verschiedene Zahlen können wir sagen, welche Zahl die kleinere ist. Wörter kann man sortieren, weil die Reihenfolge der Buchstaben im Alphabet festgelegt ist und deshalb für zwei verschiedene Wörter klar ist, welches weiter vorne im Wörterbuch stehen muss. Allgemein kann man sagen: Eine Eigenschaft ist sortierbar, wenn man für ihre einzelnen Werte eine eindeutige „kleiner als“-Beziehung (eine *Ordnung*) angeben kann.

Mit Computern werden große Datenmengen verwaltet. Um darin einzelne Daten schnell finden zu können, müssen die Datenmengen sortiert sein. Die Informatik kennt viele schnelle Sortierverfahren, und es ist gut untersucht, in welchen Fällen welche Verfahren angewendet werden sollen.

Spaß im Zoo

Heute ist Ali im Zoo. Er will möglichst viele verschiedene Vorführungen besuchen.

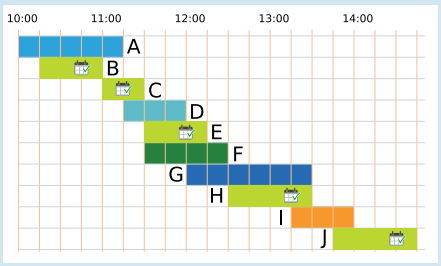
Hier ist ein Plan mit allen Vorführungen. Zum Beispiel siehst du ganz unten:
Die Vorführung der Affen beginnt um 13:45 Uhr und endet um 14:45 Uhr.

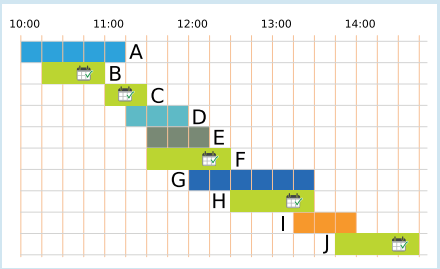


Ali besucht eine Vorführung immer ganz, von Anfang bis Ende. Kannst du Ali helfen?

Wähle so viele Vorführungen wie möglich aus, die Ali nacheinander besuchen kann.

So ist es richtig:
Ali kann höchstens 5 Vorführungen nacheinander besuchen.
Das sind die beiden richtigen Antworten:





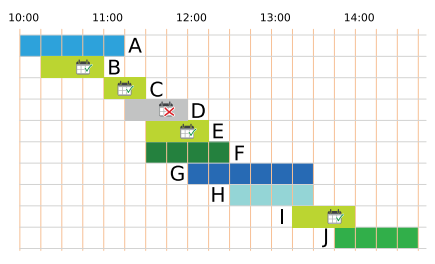
Es gibt unterschiedliche Wege, die richtigen Antworten zu finden.

Ein Besuchsplan für Ali ist eine Auswahl von Vorführungen, die er nacheinander besuchen kann. Ein Weg zu den richtigen Antworten ist es, alle Besuchspläne aufzulisten. In dieser Liste sind die Pläne mit den meisten Vorführungen die richtigen Antworten. Alle Besuchspläne zu finden, ist leider sehr zeitaufwändig.

Zeiteinheiten	Vorführungen
2	C
3	B, D, E, I
4	F, H, J
5	A
6	G

Aber könnte es nicht auch einen Besuchsplan mit 6 Vorführungen geben? Wir versuchen einmal, einen zu erstellen. Vorher schauen wir uns die Dauer der Vorführungen genauer an: Der gesamte Tag ist auf dem Plan in 19 Zeiteinheiten zu je einer Viertelstunde unterteilt. Die Vorführungen dauern 2, 3, 4, 5 oder 6 Zeiteinheiten.

Um möglichst viele Vorführungen in einen Besuchsplan zu packen, wählen wir so *kurze* Vorführungen wie möglich. Die 6 kürzesten Vorführungen dauern zusammen 18 Zeiteinheiten (2 + 3 + 3 + 3 + 3 + 4). Zu diesen kurzen Vorführungen gehören auch die Vorführungen C, D und E. Da die Vorführungen C und E aber genau hintereinander liegen, kann Ali Vorführung D dazwischen nicht besuchen.



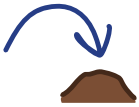
Wir müssen also die Vorführung D durch eine andere möglichst kurze ersetzen. Es sind nur noch Vorführungen mit mindestens 4 Zeiteinheiten übrig. Ohne die Vorführung D benötigen wir deshalb insgesamt mindestens 19 Zeiteinheiten für 6 Vorführungen: 2 + 3 + 3 + 3 + 4 + 4. Aber: Welche zwei Vorführungen mit 4 Zeiteinheiten wir auch wählen, eine davon überschneidet sich immer mit einer Vorführung mit 3 Zeiteinheiten. Wir müssten auch diese durch eine Vorführung mit mindestens 4 Zeiteinheiten ersetzen und würden dann mindestens 20 Zeiteinheiten für 6 Vorführungen benötigen. Es stehen aber nur 19 Zeiteinheiten zur Verfügung! Wir schlussfolgern, dass es keinen Besuchsplan geben kann, der mehr als 5 Vorführungen enthält.

Das ist Informatik!
Diese Biberaufgabe enthält einen Zeitplan der Vorführungen im Zoo. Solche Zeitpläne herzustellen, ist nicht einfach und wird in der Informatik als *Scheduling-Problem* bezeichnet. Natürlich möchte der Zoo seinen Besuchern ermöglichen, möglichst viele Vorführungen zu sehen, aber es müssen auch andere Bedingungen beachtet werden. Beispielsweise können Vorführungen nur angeboten werden, wenn die Tierpfleger Zeit haben, die verfügbaren Arenas frei sind und die Vorführungen sich mit den Lebensrhythmen der Tiere vereinbaren lassen.

Es gibt viele ähnliche Probleme im Leben, auf die sich dieselben Überlegungen anwenden lassen. Ein Beispiel ist die Erstellung eines Stundenplanes in der Schule, oder die Zuteilung von Kinofilmen zu Kinosälen. Die Erstellung dieser Zeitpläne ist so aufwändig, dass man dies schon für relativ kleine Beispiele (die Stundenpläne deiner Schule) oft nicht mehr von Hand erarbeiten kann. Auch die *Prozessoren* deines Computers müssen viele Aufgaben übernehmen und diese nacheinander abarbeiten. Der Zeitplan, wann welcher Prozessor was tut, wird vom *Betriebssystem* blitzschnell und ohne dass man es merkt erstellt. Scheduling ist eines der großen Themen der Informatik, mit welchen sich die Forschung auch heute noch beschäftigt.

Karotten pflanzen

Der Kaninchenroboter kann folgende Anweisungen ausführen:

	Springe nach **links** auf den nächsten Hügel.
	Springe nach **rechts** auf den nächsten Hügel.
	**Pflanze** einen Karottensamen auf dem Hügel, auf dem du stehst.

Der Kaninchenroboter hat diese Folge von Anweisungen ausgeführt:



Dabei ist der Roboter auf vier Hügeln gewesen. Wir wissen aber nicht, auf welchem Hügel er angefangen hat.

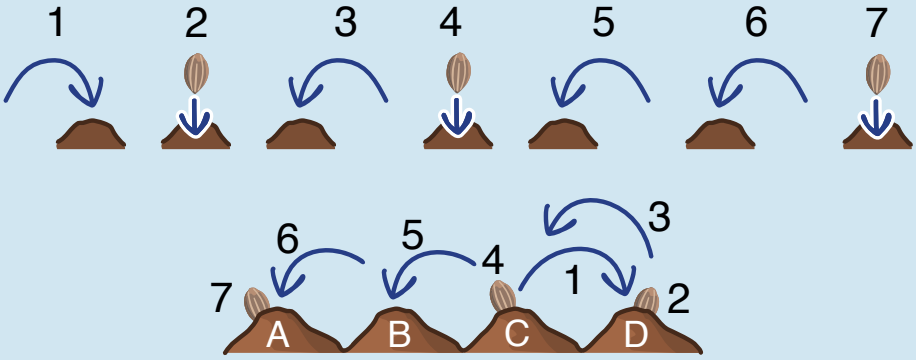
Auf welche Hügel hat der Roboter die Karottensamen gepflanzt?



So ist es richtig:



Um die richtige Antwort besser erklären zu können, geben wir den Hügeln Buchstaben (siehe oben) und den Anweisungen Nummern:



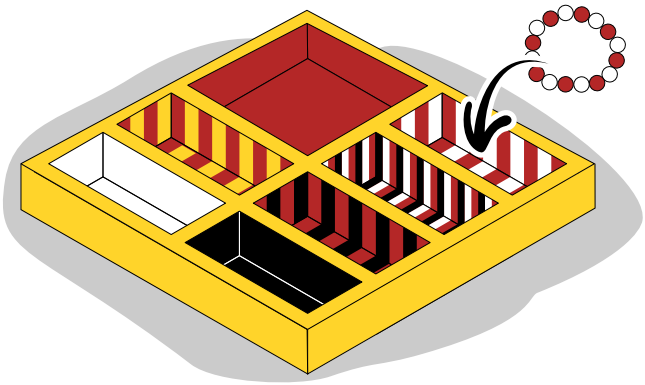
Zuerst bestimmen wir den Startpunkt des Roboters: Da der Roboter dreimal hintereinander nach links springt (Anweisungen 3, 5, 6), muss er vorher auf Hügel D stehen. Bevor er dreimal nach links springt, springt er einmal nach rechts (Anweisung 1). Der Roboter hat also auf Hügel C angefangen. Folglich werden die Karottensamen – den Anweisungen 2, 4 und 7 entsprechend – zuerst auf Hügel D, dann auf Hügel C und zuletzt auf Hügel A gepflanzt.

Das ist Informatik!
Echte Roboter haben eingebaute Computer, und die werden so ähnlich *programmiert* wie der Kaninchenroboter. Ein Computerprogramm besteht aus vielen einzelnen *Anweisungen.*

In unserem Fall wird die Abfolge der Anweisungen für den Roboter-Computer mit Hilfe von Bildblöcken angegeben. Das Ergebnis *(Output)* des Programms hängt nicht nur von der Startposition *(Input),* sondern auch von der Folge und Reihenfolge der Anweisungen ab.

Diese Biberaufgabe zeigt ein Beispiel für den Einsatz von Robotern in der Landwirtschaft. Roboter können nicht nur pflanzen, sondern auch bewässern, bestäuben oder Pflanzenschutzmittel gezielt verteilen.

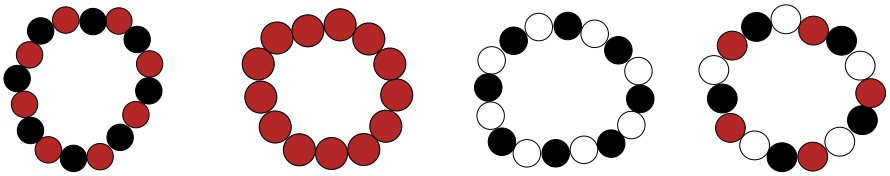
Passendes Fach



Viktoria hat Armbänder mit farbigen Perlen. Für die Bänder hat sie eine Schachtel mit sieben Fächern. Sie legt ihre Armbänder nur in Fächer mit passendem Farbmuster. Hier ist ein Beispiel für ein Band, das zu einem Fach passt:

Welches Armband passt zu keinem Fach?

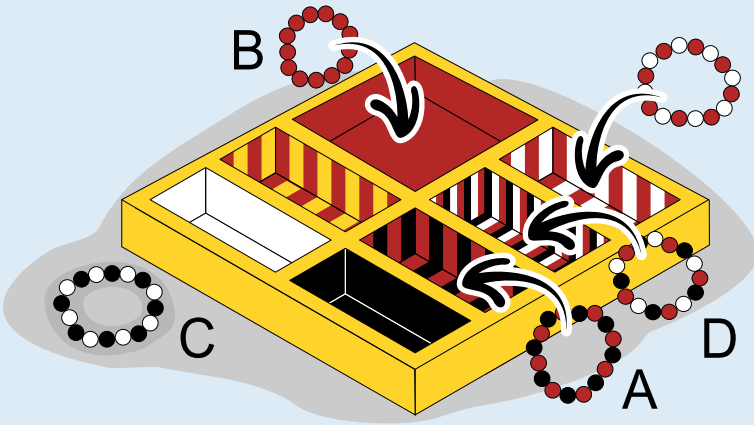
A) **B)** **C)** **D)**



Antwort C ist richtig:

Dieses Bild zeigt, zu welchen Fächern die Armbänder A, B und D passen. Das Farbmuster des Fachs passt jeweils zum Farbmuster des Armbands.

Es gibt aber kein Fach mit dem Farbmuster schwarz und weiß wie bei Armband C.



Das ist Informatik!
Viktoria ist ordentlich. Sie legt ihre Armbänder in eine Schachtel mit Fächern. Das Farbmuster eines Fachs muss zu dem Armband passen, das in dieses Fach gelegt wird. So kann Viktoria mit einem Blick jedes Band aus dem richtigen Fach nehmen. Wäre Viktoria weniger ordentlich und würde ihre Armbänder ungeordnet in eine Schublade legen, müsste sie in der Schublade kramen, wenn sie ein bestimmtes Band herausnehmen will. Noch schlechter wäre, wenn in der Schublade auch andere Dinge lägen oder wenn die Bänder in eine große Kiste kämen, die viel mehr Dinge enthalten als eine Schublade.

Computer verwalten viele Daten und legen sie in Speichern mit sehr viel Platz ab. Wenn die Daten ungeordnet in die Speicher kämen, wäre das wie bei einer riesigen Wühlkiste. Es würde dann sehr lange dauern, bestimmte Daten wiederzufinden. Deshalb müssen Informatikerinnen und Informatiker wie Viktoria sein, nämlich sehr ordentlich – zumindest wenn es um die Computerspeicher geht, um die sie sich kümmern. Für Daten gibt es viele verschiedene Ablage-Methoden, mit denen ein Computerprogramm die benötigten Daten schnell finden kann. Am schnellsten geht es mit *Hashing:* Wenn ein Datenelement gespeichert werden soll, wird ein Wert für das Element berechnet und an eine Stelle im Speicher gelegt, die man mit diesem Wert direkt finden kann. Das ist wie bei Viktoria und der Schachtel mit den Farbmustern in dieser Biberaufgabe.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

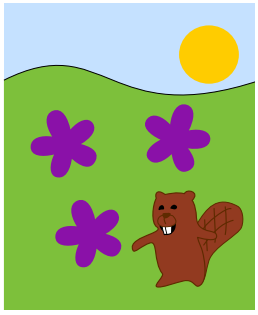
Zeichnung

Bea malt ein Bild. Sie hat fünf Flaschen mit Farbe. Am Anfang sind alle Flaschen voll. Für große Flächen auf dem Bild braucht sie mehr Farbe als für kleine Flächen. Als Bea fertig ist, enthalten die Flaschen noch so viel Farbe:

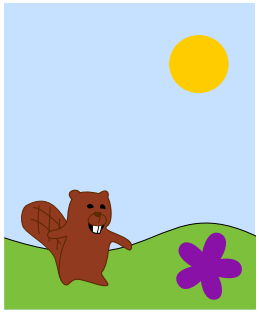


Welches Bild hat Bea gemalt?

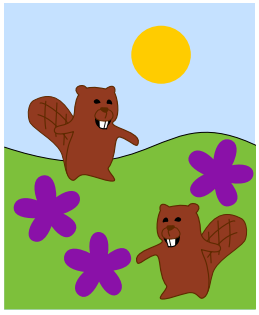
A)



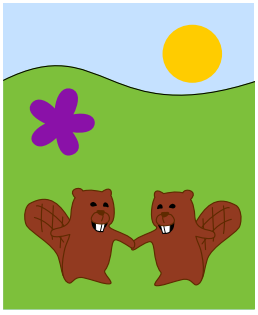
B)



C)



D)



Antwort D ist richtig:

Um die richtige Antwort zu finden, kannst du die Farbflächen auf dem Bild mit dem Inhalt der Farbflaschen vergleichen. Du erkennst an den Farbflaschen, dass am meisten Grün verwendet wurde und am zweitmeisten Blau. Das passt zu den Bildern A, C und D. Auf diesen Bildern ist die grüne Fläche am größten und die blaue Fläche am zweitgrößten. Bild B kannst du ausschließen, da dort die blaue Fläche viel größer ist als die grüne.

An den Farbflaschen kannst du außerdem erkennen, dass Bea genauso viel Gelb verwendet hat wie Violett. Auf den Bildern A und C ist die violette Gesamtfläche (3 Blüten) etwa 3-mal so groß wie die gelbe Fläche (Sonne). Nur in Bild D sind die gelben und violetten Flächen (Sonne und Blüte) etwa gleich groß. Auch die Verwendung der braunen Farbe passt zu Bild D: Bea hat genauso viel Braun verwendet wie Blau, und die braune Gesamtfläche (2 Biber) ist etwa gleich groß wie die blaue Fläche. Bea kann also nur Bild D gemalt haben.

Das ist Informatik!
In dieser Biberaufgabe hast du das richtige Bild herausgefunden, indem du die Verwendung der Farben untersucht hast. Dabei hast du dir z. B. folgende Fragen gestellt: Welche Farben kommen vor? Welche Farbe wird am meisten verwendet? Welchen Farben werden nur wenig verwendet? Wird eine Farbe mehr verwendet als eine andere Farbe?

Zur Beantwortung dieser Fragen musstest du auf dem Bild verschiedene Flächen unterscheiden, die Größe von Flächen abschätzen, Flächenwerte miteinander vergleichen und Entscheidungen treffen. Solche Untersuchungen können auch automatisch von Computerprogrammen durchgeführt werden. Computerprogramme können dir helfen, auf deinem Handy gespeicherte Fotos zu finden. Durch eine Farbuntersuchung kann ein Computerprogramm Landschaftsbilder, die oben viel Blau und unten viel Grün enthalten, von Portraitfotos unterscheiden. Durch Farbuntersuchungen von Satellitenfotos können Computer ermitteln, wie dicht eine Gegend bebaut ist, oder ob in einem Naturschutzgebiet illegal Wald gerodet wurde.

In der Informatik gibt es einen wichtigen Bereich, der sich mit der automatischen Untersuchung von Bildern befasst: *Computer Vision.*

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Im Park

Alia  ist bei der Biberstatue  im Park.

Auf ihrem Weg dorthin hat sie einige Dinge gesehen:



Zuerst hat sie blaue Blumen gesehen.

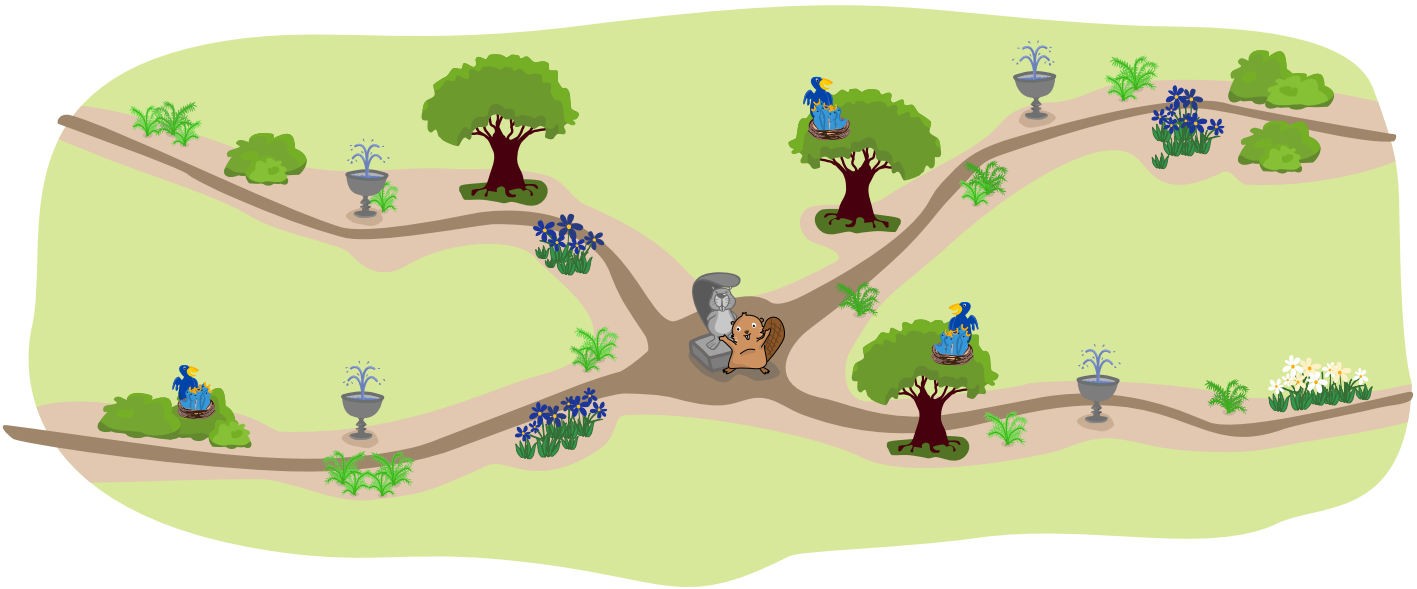


Danach hat sie einen Brunnen gesehen.



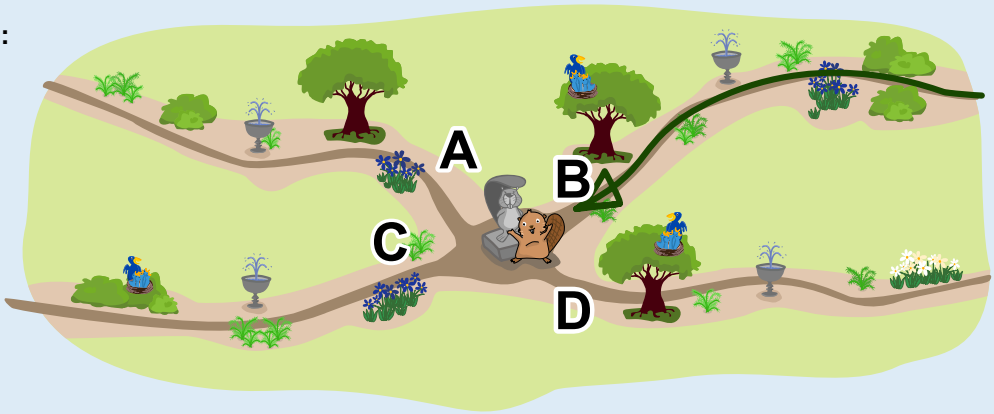
Zuletzt hat sie ein Nest entdeckt.

Auf welchem Weg ist Alia zur Biberstatue gegangen?



Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:

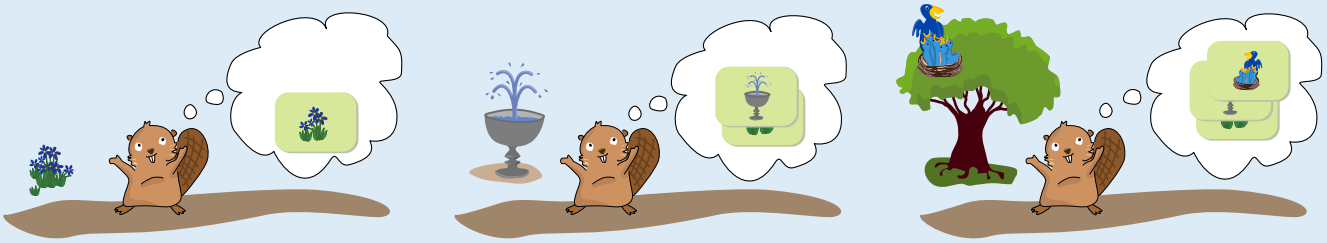


Um die richtige Antwort zu finden, prüft man zunächst, auf welchem Weg alle Dinge vorkommen, die Alia gesehen hat. Anschließend prüft man ausgehend von der Biberstatue, auf welchem Weg die Dinge, die Alia gesehen hat, in der umgekehrten Reihenfolge vorkommen.
Auf ihrem Weg zur Biberstatue hat Alia zuerst die blauen Blumen, dann den Brunnen und zuletzt das Nest gesehen. Auf dem korrekte Weg müssen die gesehenen Dinge also ausgehend von der Statue in der folgenden Reihenfolge vorkommen: Zuerst das Nest, dann der Brunnen und zuletzt die blauen Blumen.

Weg A ist nicht richtig, weil kein Nest vorkommt.
Weg C ist nicht richtig, weil die Dinge in umgekehrter Reihenfolge vorkommen.
Weg D ist nicht richtig, weil die Blumen am Ende nicht blau, sondern weiß sind.

Der einzige Weg, auf dem alle Dinge von der Statue aus in dieser Reihenfolge vorkommen, ist Weg B.

Das ist Informatik!
Auf ihrem Weg in den Park sieht Alia verschiedene Dinge in einer bestimmten Reihenfolge: zuerst die Blumen, dann den Brunnen und zuletzt das Vogelnest:



Wenn Alia sich auf dem Hinweg diese Informationen in genau dieser Reihenfolge merken will, um den Rückweg leichter zu finden, kann sie sie wie auf einem *Stapel* ablegen – zuletzt das Vogelnest. Auf dem richtigen Rückweg wird sie zuerst wieder das Vogelnest sehen. Das löscht sie dann aus ihrem „Stapel-Merkspeicher“ und geht weiter. Kurz gesagt: Was auf dem Hinweg zuletzt auf ihren Stapel gekommen ist und nun dort zuoberst liegt, nimmt sie auf dem Rückweg zuerst heraus. Das könnte man auf Englisch noch kürzer so ausdrücken: „last in, first out“ (zuletzt hinein, zuerst heraus), kurz: *LIFO.*

Die Informatik kennt eine Struktur zur Speicherung von Daten, die nach diesem LIFO-Prin
zip funktioniert. Diese Struktur heißt *Stack,* auf Deutsch: Stapel. Natürlich kann man sie nur dann sinnvoll verwenden, wenn man auf die gespeicherten Daten nach dem LIFO-Prinzip zugreifen will. Das scheint eine starke Einschränkung zu sein. Aber weil Stacks in Computern sehr einfach zu realisieren sind, werden sie dennoch recht häufig verwendet. Bei einem Computerspiel zum Beispiel speichert der Computer deine letzten Spielzüge in einem Stapel, so dass du bei Bedarf Zug um Zug rückgängig machen kannst.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

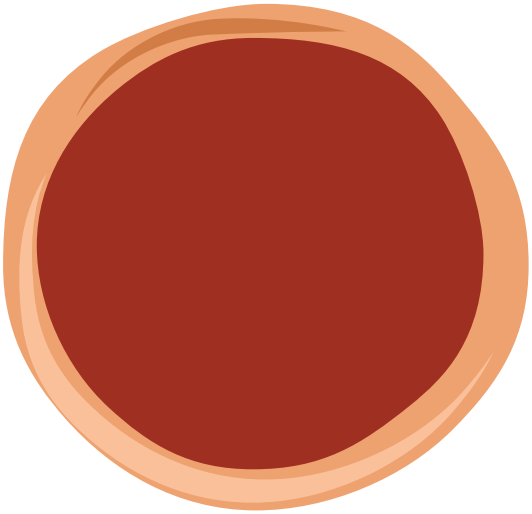
Pizza-Party

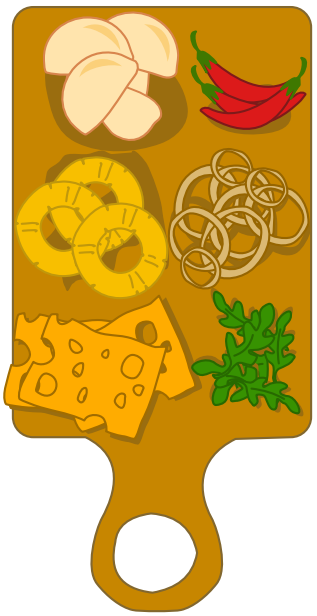
John feiert eine Pizzaparty. Er weiß, welche Beläge sich seine Gäste auf der Pizza wünschen.

Alice	
Bob	
Cem	
Dana	

John möchte eine Pizza mit ingesamt drei Belägen backen. Er wählt drei Beläge so aus, dass er möglichst viele Wünsche seiner Gäste erfüllt.

Mit welchen drei Belägen erfüllt John die meisten Wünsche?





So ist es richtig:

Um die richtige Antwort zu finden, könnte man alle Kombinationen von drei Belägen ausprobieren und für jede Kombination zählen, wie viele Wünsche sie erfüllt. Es gibt aber 20 solcher Kombinationen, sodass dieses Vorgehen mühsam und fehleranfällig ist.

Man muss aber gar nicht über Kombinationen nachdenken. Die Kombination, die die meisten Wünsche erfüllt, besteht aus den drei Belägen, die einzeln am meisten gewünscht werden. Diese Beläge kann man herausfinden, indem man die Tabelle aus der Aufgabenstellung anders organisiert. Für jeden Belag gibt es nun eine eigene Spalte. In der Spalte steht eine 1, wenn der Belag von einem Gast gewünscht wird, und sonst eine 0. Dann addiert man die Zahlen in jeder Spalte.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

						
Alice	1	1	0	0	1	0
Bob	0	1	1	0	0	1
Cem	0	1	0	1	1	0
Dana	0	1	0	0	1	1
Summe	1	**4**	1	1	**3**	**2**

Pilze sind am beliebtesten (4), dann kommen Käse (3) und Zwiebeln (2). Wenn John diese beliebtesten drei Beläge auf die Pizza legt, erfüllt er die meisten Wünsche, nämlich insgesamt 9. Mit keiner anderen Zusammenstellung von Belägen kann er mehr Wünsche erfüllen.



Das ist Informatik!
Das Pizza-Belag-Auswahl-Problem in dieser Biberaufgabe ist ein *Optimierungsproblem.* John sucht nicht irgendeine Belag-Kombination für seine Pizza, sondern die beste – oder, besser gesagt: eine beste, denn es wäre ja denkbar, dass es mehrere Belag-Kombination gibt, welche die meisten Wünsche erfüllen.

Um die beste Lösung eines Optimierungsproblems zu finden, muss man
- Daten erfassen (hier: die Wünsche der Freunde),
- neue Daten berechnen (die Häufigkeiten der Wünsche),
- Bedingungen berücksichtigen (es soll nur eine Pizza mit genau drei Belägen gebacken werden) und
- eine Entscheidung fällen (die Auswahl der Beläge).

Für die Entscheidung bei der Lösung eines Optimierungsproblems muss man mögliche Antworten bzw. Lösungen miteinander vergleichen können. Dazu benötigt man ein *Maß* dafür, wie gut eine Lösung ist. In dieser Biberaufgabe ist die Anzahl der erfüllten Wünsche das Optimierungsmaß: Je mehr Wünsche erfüllt sind, desto besser. Der Optimierungsalgorithmus konnte die Lösung mit dem besten Wert für das Optimierungsmaß direkt auswählen, durch Addition der Wünsche für die einzelnen Beläge. In diesem Fall war es also nicht nötig, unterschiedliche Lösungen miteinander zu vergleichen.

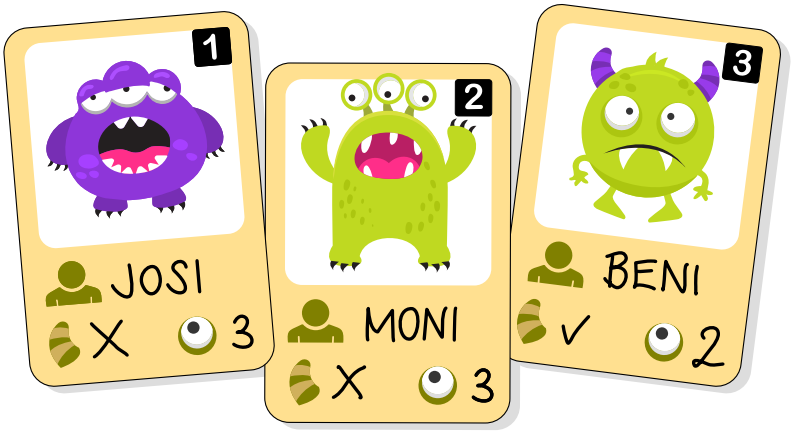
Optimierungsalgorithmen werden in der Informatik für viele Zwecke entwickelt, zum Beispiel für Produktionsprozesse in der Industrie oder zur Berechnung von Routen für die Auslieferungsfahrzeuge von Paket- oder anderen Lieferdiensten. Dabei kann es ganz unterschiedliche Optimierungsmaße geben, etwa die Geschwindigkeit des Produktionsprozesses oder die Länge der Route. Eines ist aber immer gleich: Man muss sich überlegen, wie man möglichst geschickt aus der Menge der möglichen Lösungen eine beste Lösung auswählt. So leicht wie in dieser Biberaufgabe geht das nur selten.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Ricca-Karten 1

Barbara sammelt Karten mit Monstern, den „Riccas“.
Auf jeder Karte sind die Eigenschaften eines Riccas angegeben: der Name , die Anzahl der Augen und ob das Ricca Hörner hat .

Hier sind drei Ricca-Karten, auf denen die Werte der Eigenschaften stehen:



Für jede Eigenschaft ist festgelegt, welche Werte erlaubt sind: Barbara bekommt vier Karten für ein weiteres Ricca. Aber nur auf einer Karte haben alle Eigenschaften erlaubte Werte.

Auf welcher?

Symbol	Eigenschaft	erlaubte Werte
	Name	Text (mehrere Buchstaben)
	hat Hörner	✓ oder X
	Anzahl der Augen	Zahlen

A)



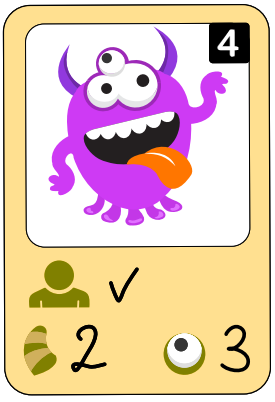
B)



C)



D)



Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Antwort B ist richtig!

Die Tabelle zeigt, welche Werte die Eigenschaften auf den vier Karten haben:

	A)	B)	C)	D)
	LORI	LORI	LORI	✓
	2	✓	2	2
	✓	3	3	3

Nur auf der Karte von Antwort B haben alle Eigenschaften erlaubte Werte, nämlich den Text LORI bei  , einen ✓ bei  und eine Zahl bei  .

Auf den Karten von den Antworten A, C und D steht bei  jeweils eine Zahl, nämlich 2. Das Ricca hat zwar zwei Hörner, aber die Eigenschaft  soll nur angeben, ob das Ricca Hörner hat oder nicht. Deshalb sind nur die Werte ✓ und X erlaubt.

Auf der Karte von Antwort A ist bei  der Wert ✓angegeben. Das Ricca hat Augen, aber die Eigenschaft  soll die Anzahl der Augen angeben. Deshalb ist nur eine Zahl erlaubt.

Das ist Informatik!
Die Eigenschaften und deren Werte auf den Karten beschreiben die Riccas. Die Bilder enthalten jedoch viel mehr Informationen über die Riccas, als auf den Karten notiert ist. So ist zum Beispiel nicht nur zu sehen, ob ein Ricca Hörner hat oder nicht, sondern auch wie viele Hörner es hat. Es ist wichtig, sich gut zu überlegen, welche Werte man für die Eigenschaften erlaubt.

Wenn man zum Beispiel herausfinden will, wie viele Hörner die Riccas auf den Karten insgesamt haben, muss man sie auf den Bildern zählen. Damit das schneller geht, wäre es gut, für die Eigenschaft  Zahlen als Werte zu erlauben, so wie bei den Augen. Dann gibt die Eigenschaft  die Anzahl der Hörner an.

Auch bei Computer-Programmen werden als Daten oft Werte für Eigenschaften gespeichert, wie bei den Ricca-Karten in dieser Biberaufgabe. Zum Beispiel werden im Computer-Katalog einer Bücherei für jedes Buch unter anderem der Titel, die Autorin oder der Autor und die Anzahl der Ausleihen gespeichert. Je nachdem, welche Werte für diese Eigenschaften erlaubt sind, kann das Programm unterschiedliche Berechnungen oder Operationen ausführen. Die Informatik nennt die Menge der für eine Eigenschaft erlaubten Werte den *Datentyp* dieser Eigenschaft. Wenn die Anzahl der Ausleihen den Datentyp Zahl hat, kann das Computer-Programm der Bücherei leicht herausfinden, welche Bücher am häufigsten ausgeliehen wurden.

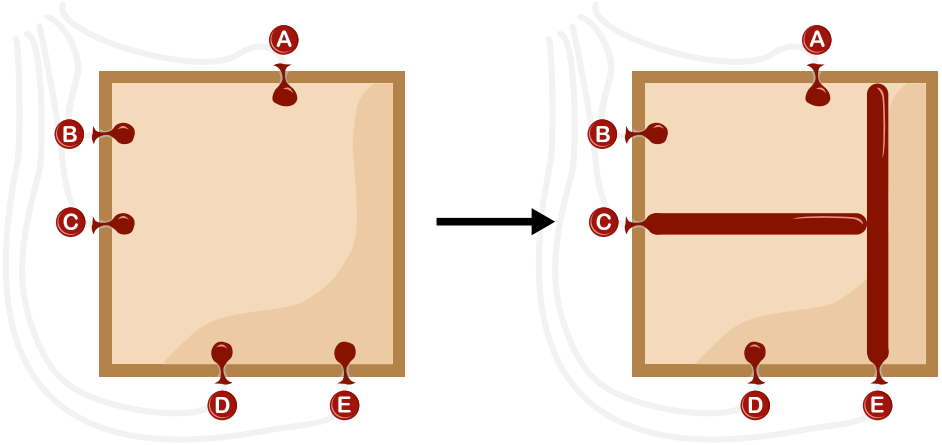
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Ballon-Maschine 1

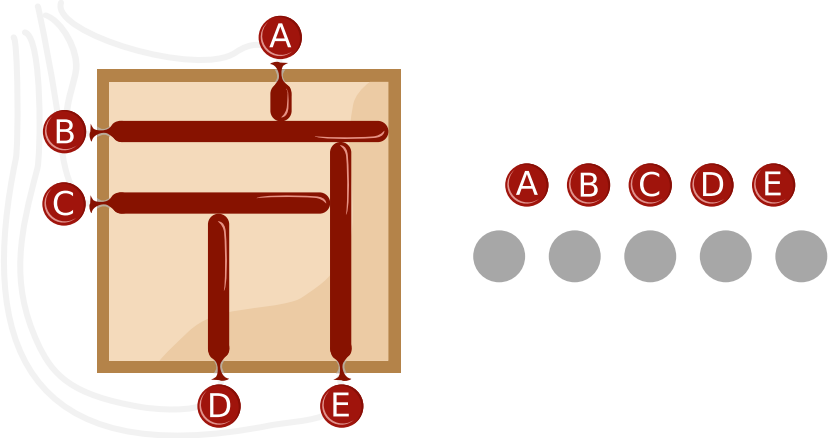
Eine Ballonmaschine kann Bilder erstellen, indem sie Ballons in einem quadratischen Rahmen aufbläst. Die Ballons sind mit den Buchstaben A, B, C, D und E beschriftet. Die Maschine liest nacheinander eine Folge von Buchstaben, von links nach rechts. Vor dem ersten Buchstaben sind alle Ballons leer.

Wenn die Maschine einen Buchstaben gelesen hat, macht sie Folgendes:
Der Ballon mit diesem Buchstaben wird aufgeblasen bis er
- entweder einen anderen Ballon berührt
- oder den gegenüberliegenden Rand des Rahmens berührt.

Wenn die Maschine zum Beispiel die Buchstabenfolge E C liest, macht sie dies:

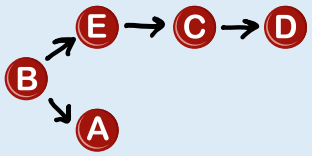


Die Maschine liest eine Folge von fünf Buchstaben. Am Ende hat sie dieses Bild erstellt. Gib die Folge an:



Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:



Aus dem Bild, das die Maschine erstellt hat, kann man einige Bedingungen für die richtige Reihenfolge beim Aufblasen der Ballons ablesen. Denn außer Ballon B wurde jeder Ballon aufgeblasen, bis er einen anderen Ballon berührt hat:

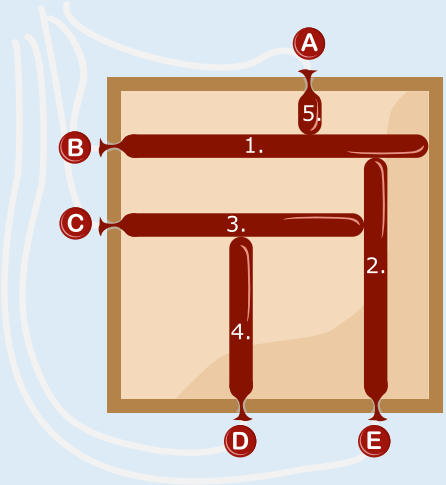
- Ballon B wurde vor A und E aufgeblasen.
- E wurde vor C aufgeblasen.
- C wurde vor D aufgeblasen.

Diese „Vorrangbeziehungen" zwischen den Ballons fasst das folgende Bild zusammen. Ein Pfeil vom (zum Beispiel) Ballon-Buchstaben B zum Ballon-Buchstaben A gibt an, dass Ballon B vor Ballon A aufgeblasen werden muss.

Jede richtige Antwort, also jede richtige Reihenfolge für das Aufblasen der Ballons, muss diese Vorrangbeziehungen einhalten. Das heißt, dass bei jeder richtigen Antwort am Anfang B sein muss. Zudem muss E vor C und C vor D sein. Das A kann jedoch irgendwann nach dem B kommen. Damit kann es genau vier mögliche richtige Folgen geben:

- B A E C D
- B E A C D
- B E C A D
- B E C D A

Hier sieht man, wie die Maschine mit der letzten Folge das gewünschte Bild erstellt:



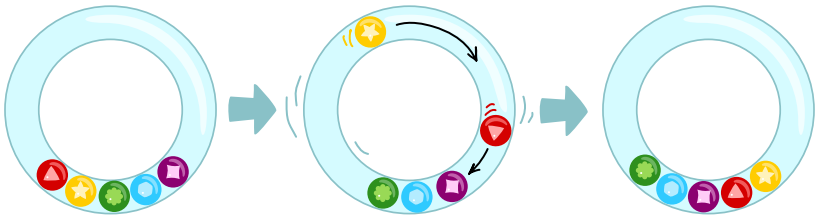
Das ist Informatik!

Die Buchstabenfolge ist wie ein Programm, das die Ballonmaschine steuert. Jeder Buchstabe ist eine Anweisung, die die Maschine dazu bringt, einen Ballon aufzublasen. Die „Buchstaben-Anweisungen" können zu einer Anweisungsfolge kombiniert werden. So entsteht ein längeres Programm, mit dem die Maschine komplexere Bilder erstellen kann. Dabei ist die Reihenfolge der Anweisungen wichtig. Zum Beispiel erzeugt die Anweisungsfolge B E ein anderes Bild als die Anweisungsfolge E B. Das liegt daran, dass die Ausführung einer Anweisung Auswirkungen auf die Ausführung folgender Anweisungen haben kann. Bei der Ballonmaschine hat die Ausführung einer Anweisung insbesondere Auswirkungen auf die Länge der später aufgeblasenen Ballons.

Auch Computerprogramme bestehen aus Anweisungen, die zu größeren Programmen kombiniert werden. Die einfachste Form der Kombination nennt man in der Informatik eine *Sequenz,* also eine Folge von Anweisungen. Kompliziertere Kombinationen entstehen durch spezielle *Kontrollstrukturen* wie Wiederholungsanweisungen und bedingte Anweisungen. Auch bei Computerprogrammen kann das Ausführen einer Anweisung Auswirkungen auf die spätere Ausführung einer anderen Anweisung haben – zum Beispiel, wenn beide Anweisungen auf die gleichen Daten zurückgreifen und diese verändern. Es ist eine wichtige Aufgabe für Informatikerinnen und Informatiker, bei der Entwicklung von Programmen solche Auswirkungen sorgfältig zu beachten, damit die Programme so funktionieren wie gewünscht. So wie bei der Ballonmaschine in dieser Biberaufgabe die Reihenfolge der Anweisungen stimmen muss, damit die Maschine das gewünschte Bild erstellt.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Olivers Rassel



Oliver hat eine durchsichtige Rassel mit bunten Kugeln.
Wenn er die Rassel schüttelt, bewegen sich einige Kugeln.
Danach liegen die Kugeln anders in der Rassel als vorher.
Oliver schüttelt die Rassel noch einmal.



Wie liegen die Kugeln nun? Fülle die leeren Plätze in der Rassel.

So ist es richtig:

Nach dem Schütteln können die Kugeln anders liegen. Aber ihre Reihenfolge ändert sich nicht: Wenn man von einer bestimmten Kugel aus zur nächsten Kugel geht, ist das immer dieselbe – auch wenn man einmal rundherum durch die Rassel gehen muss.

Schauen wir uns die Reihenfolge im ersten Bild oben an: Nach der roten Kugel kommt die gelbe. Wenn wir das mit einem Pfeil schreiben – → – können wir die gesamte Reihenfolge so aufschreiben: → → → → →

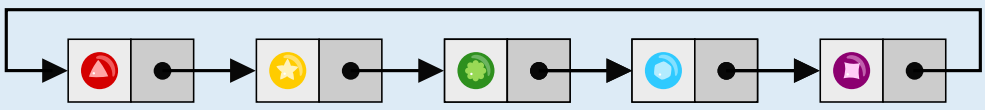
Nach der violetten kommt also wieder die rote.
So schliesst sich der Kreis in Olivers Rassel.



Mit Hilfe der Reihenfolge finden wir leicht die richtige Antwort:
Nach der violetten Kugel kommt die rote und danach die gelbe.
Die grüne Kugel liegt schon da, und danach kommt die blaue :

Das ist Informatik!
In Olivers Rassel bleibt die Reihenfolge der Kugeln immer gleich. Egal wo eine Kugel liegt, die nächste Kugel (in einer bestimmten Richtung) ist immer dieselbe. Aber die Kugeln in der Rassel haben noch eine Eigenschaft: Wenn man bei einer Kugel anfängt und zur nächsten Kugel geht und von dort wieder zur nächsten und so weiter, dann kommt man irgendwann wieder bei der Anfangs-Kugel an. Das ist so ähnlich wie bei Perlen an einer Kette, die jemand am Arm oder um den Hals trägt.

Auch in der Informatik kennt man Ketten. Wenn man Datenelemente in einer festen Reihenfolge bearbeiten will, können sie in einer *verketteten Liste* gespeichert werden. In einer verketteten Liste wird jedes Datenelement in einem eigenen *Knoten* gespeichert. Zusätzlich enthält jeder Knoten einen Verweis auf den nächsten Knoten in der Liste. Enthält der letzte Knoten einen Verweis auf den ersten Knoten, ist die Kette *zirkulär.* Die Kugeln in Olivers Rassel bilden so eine zirkuläre verkettete Liste, mit den Kugeln als Knoten und der Nachbarschaft innerhalb der Rassel (gegen den Uhrzeigersinn gesehen) als Verweise:



Verkettete Listen in Computerprogrammen kommen im Alltag selten zum Vorschein – aber gelegentlich doch: Wenn die Musik-App auf dem Smartphone Songs einer Wiedergabeliste der Reihe nach spielt und am Ende wieder von vorne anfängt, bilden auch die Songs eine zirkuläre verkettete Liste.

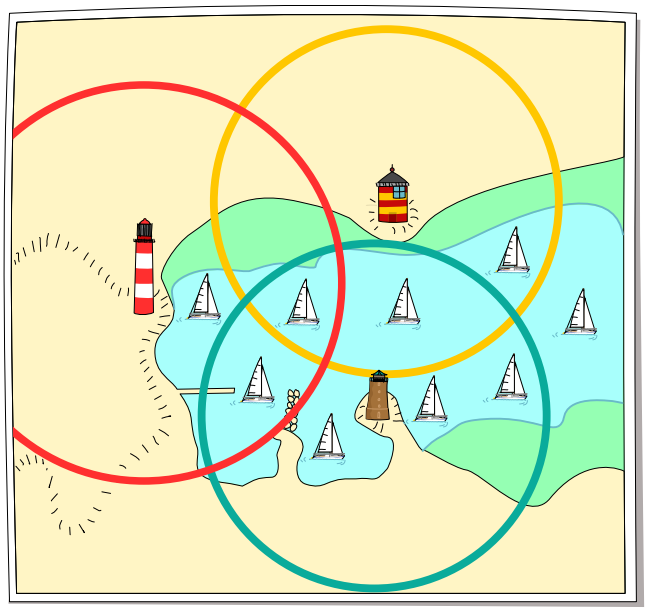
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Leuchttürme

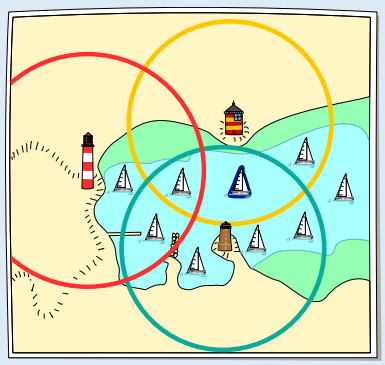
Ben ist auf seinem Segelboot.
Ben hat eine Karte. Sie zeigt die Leuchttürme.
Um jeden Leuchtturm ist ein Kreis gezeichnet.
Wenn Bens Boot in dem Kreis ist, kann Ben den Leuchtturm sehen.

Ben sieht die Leuchttürme  und  . Den Leuchtturm  sieht er nicht.

Welches ist Bens Boot?

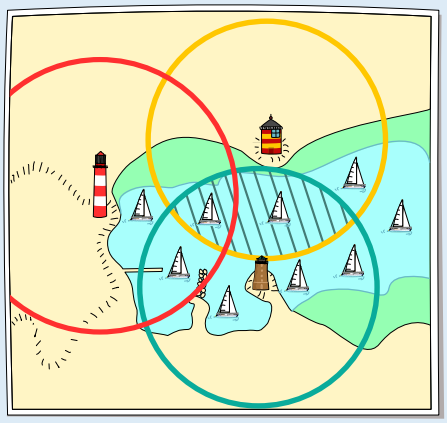


So ist es richtig:

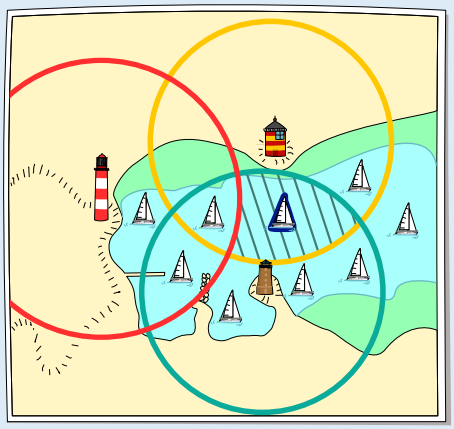


Ben sieht die Leuchttürme  und  . Bens Boot ist also in dem Gebiet, in dem sich die Kreise dieser Leuchttürme überschneiden.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0



Ben sieht den Leuchtturm  nicht. Also ist Bens Boot nicht im Kreis dieses Leuchtturms.



Dies ist das einzige Boot, das in den beiden Kreise um die Leuchttürme  und  ist, aber nicht im Kreis um  .
Also ist dieses Boot Bens Boot.

Das ist Informatik!
In der Informatik ist die exakte Beschreibung eines Problems und des Lösungsverfahrens von großer Bedeutung. Ein nützliches Werkzeug dafür ist die Aussagenlogik, bei der mit Ausssagen gearbeitet wird, die jeweils wahr oder falsch sein können. In dieser Biberaufgabe kann ein Gebiet, in dem ein Boot ist, mit diesen drei Aussagen beschrieben werden:

- Aussage A lautet: „Leuchtturm  ist sichtbar.“
- Aussage B lautet: „Leuchtturm  ist sichtbar.“
- Aussage C lautet: „Leuchtturm  ist sichtbar.“

Für Bens Boot sind die Aussagen B und C wahr, aber die Aussage A ist falsch:

Ben sieht die Leuchttürme  und  , aber nicht den Leuchtturm  . In der Aussagenlogik wird das so aufgeschrieben: „B UND C UND NICHT A“.

Diese Aussage kombiniert die „einfachen“ Aussagen A, B und C mit Hilfe von UND und NICHT. Sie beschreibt das in der Erklärung gezeigte Gebiet, in dem genau Bens Boot ist.

In der Karte entspricht zum Beispiel „A UND B UND C“ dem Gebiet, in dem sich alle drei Kreise überschneiden, denn nur dort kann man alle drei Leuchttürme sehen. „NICHT A UND NICHT B UND NICHT C“ entspricht dem Gebiet, das außerhalb der drei Kreise liegt. Dort ist keiner der drei Leuchttürme sichtbar.

In der Informatik kommt die Aussagenlogik unter anderem bei der Wissensrepräsentation, bei der Verifizierung von Programmen und vor allem bei der Modellierung digitaler Schaltungen zur Anwendung.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

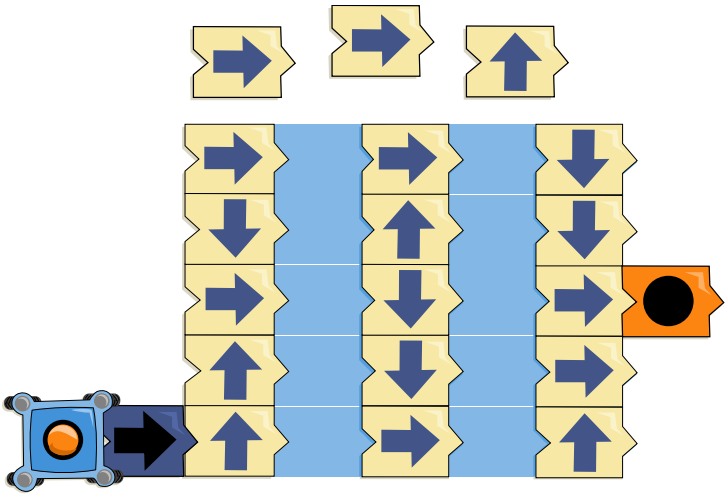
Robertas Roboter

Robertas Roboter  kann nur über besondere Kärtchen fahren. Mit den Kärtchen kann man einen Wegeplan legen.

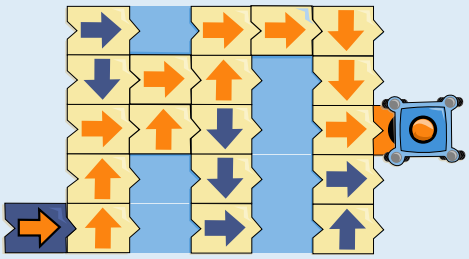
Der Roboter beginnt immer auf dem Start-Kärtchen  und soll das Ziel-Kärtchen  erreichen. Von einem Kärtchen mit Pfeil fährt er in Pfeilrichtung zum nächsten Kärtchen.

Unten siehst du einen Wegeplan. Darauf fährt der Roboter zunächst nach rechts, dann hoch und noch einmal hoch. Dann würde er gerne nach rechts fahren, aber dort ist kein Kärtchen! Der Wegeplan ist noch nicht fertig. Es gibt noch drei Kärtchen und zehn Stellen, an die sie gelegt werden können.

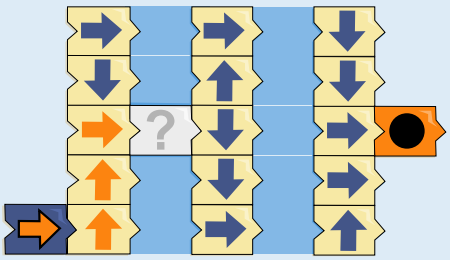
Lege die drei Kärtchen so, dass Robertas Roboter das Ziel erreicht.



So ist es richtig:



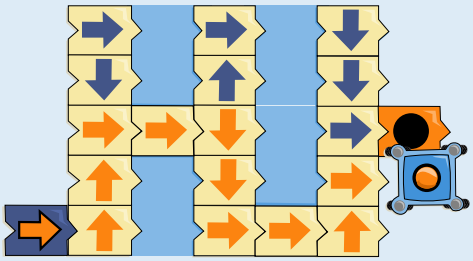
Wie bereits in der Aufgabe angedeutet, wird Robertas Roboter zunächst nach rechts, zweimal hoch und einmal wieder nach rechts fahren. Damit muss ein Kärtchen an die mit ? markierte Stelle gelegt werden:

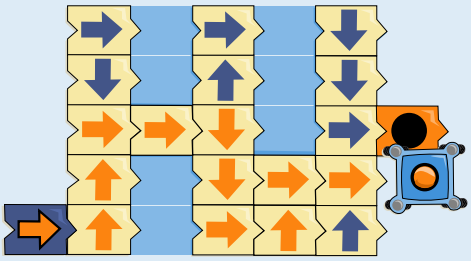


Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Die beiden Arten von Kärtchen, die zur Verfügung stehen, sind „hoch“  oder „rechts“  . Falls  gewählt würde, würde der Roboter also nach rechts fahren, danach zweimal runter und wieder rechts. Nun könnte man entweder wieder  oder  und darüber  wählen.

In beiden Fällen würde der Roboter unter dem Ziel landen:





Also muss  gewählt werden. Als zweites Kärtchen kann nun nur noch  gewählt werden, weil das einzige  bereits verwendet wurde. Dann würde der Roboter also nach rechts, hoch, nach rechts und von dort hoch und nach rechts fahren. An dieser Stelle ist das letzte verfügbare Kärtchen  zu legen, so dass der Roboter also nach rechts, zweimal runter und einmal nach rechts fahren kann und im Ziel landet.

Es gibt also nur eine richtige Lösung.

Man kann sich diese Lösung übrigens auch schneller rückwärts überlegen: Zum Ziel kommt man nur von den obersten drei möglichen Positionen in der rechten hellblauen Spalte. Dorthin kommt man jedoch nur ganz oben, damit muss ganz oben rechts schon mal  liegen. Dorthin kommt man jedoch nur von den obersten beiden Kärtchen in der Mitte, also muss man in der linken hellblauen Spalte noch einmal  legen, um dort hinzukommen.

Das ist Informatik!
In der Regel wird man dieses Problem so lösen wie in der Antworterklärung beschrieben: Man wird mal einen Weg ausprobieren, und wenn er nicht zum Ziel führt, wird man zurückgehen und einen anderen Weg ausprobieren. Dies nennt man in der Informatik eine *Tiefensuche,* weil zunächst ein Weg vollständig ausprobiert wird und dann erst ein nächster probiert wird. Weil man dazu oftmals einen oder mehrere Schritte zurückgehen muss, um eine andere Lösung auszuprobieren, macht man *Backtracking.* In jedem Fall jedoch löst man das Problem vorwärts. Manche Probleme jedoch lassen sich besser rückwärts lösen, indem man vom Ziel her schaut, wie man dort hinkommen könnte. Eigentlich ist diese Art von Problem nicht gut geeignet, dass man es rückwärts löst, denn während man vorwärts immer weiß, welches das nächste Kärtchen ist, kann es rückwärts sein, dass man von mehreren Kärtchen kommt (der Pfeil zeigt vorwärts und rückwärts muss man für alle Kärtchen drum herum schauen, ob sie in Richtung des Kärtchens zeigen). Aber in diesem speziellen Fall ist gleichwohl schnell zu erkennen, welche Lösung die einzig richtige sein kann.

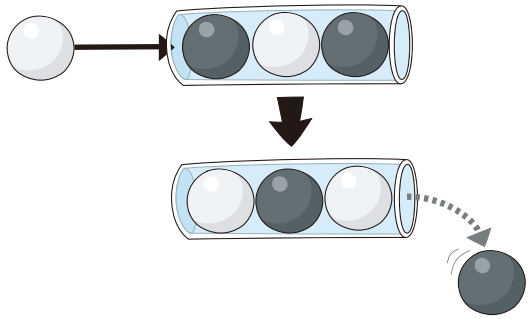
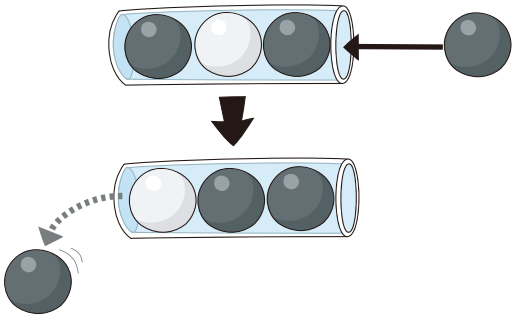
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Röhre

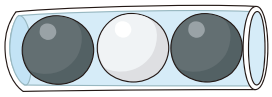
In eine durchsichtige Röhre passen genau drei Kugeln. Die Röhre ist an beiden Seiten offen.



Wenn in die volle Röhre von einer Seite eine Kugel hinein geschoben wird, fällt auf der anderen Seite eine Kugel heraus. Hier sind zwei Beispiele:

Weiße Kugel von links	Schwarze Kugel von rechts
	

Die Röhre ist mal wieder voll:



Nun werden nacheinander vier Kugeln in die Röhre geschoben: Zuerst eine schwarze und dann eine weiße Kugel von rechts, danach eine schwarze und dann eine weiße Kugel von links:



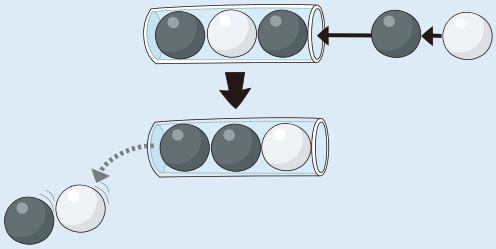
Welche drei Kugeln sind am Ende in der Röhre?



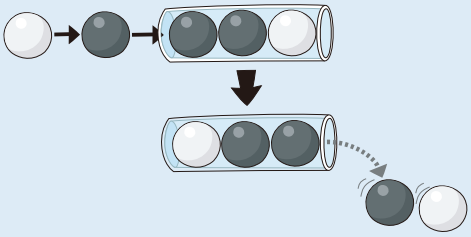
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:

Es werden zwei Kugeln von rechts in die Röhre geschoben, zuerst eine schwarze und dann eine weiße Kugel. Daher fallen links zwei Kugeln raus, zuerst die schwarze, dann die weiße.



Danach werden zwei Kugeln von links in die Röhre geschoben, zuerst eine schwarze und dann eine weiße Kugel. Daher fallen diesmal rechts zwei Kugeln raus, zuerst die weiße, dann die schwarze.



Am Ende sind also (von links nach rechts) eine weiße, eine schwarze und noch eine schwarze Kugel in der Röhre.

Man kann die richtige Antwort auch einfacher bestimmen: Da zuerst von rechts zwei Kugeln in die Röhre geschoben werden und danach von links zwei Kugeln, ist die ursprünglich rechts in der Röhre liegende Kugel wieder rechts. Links davon sind (von rechts nach links) die beiden zuletzt in die Röhre geschobenen Kugeln.

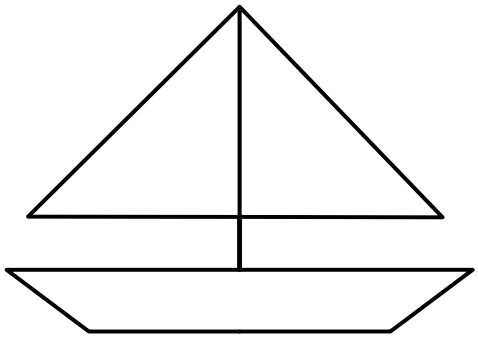
Das ist Informatik!
Die Röhre in dieser Biberaufgabe ist ein spezielle Form der Warteschlange, die in der Informatik *Deque* genannt wird (kurz für engl. *Double-Ended Queue, „Warteschlange mit zwei Enden“).* Im Unterschied zu einer herkömmlichen Warteschlange können wir an beiden Enden einer Deque ein Element hinzufügen und von beiden Enden der Deque ein Element entfernen. Eine Deque ist eine *Datenstruktur.*

Deques sind vielfältig einsetzbar, z. B. als verallgemeinerter Ersatz sowohl von *Stapeln* (vorne hinzufügen und vorne entfernen) als auch einfachen *Warteschlangen* (hinten hinzufügen und vorne entfernen). Man kann sich eine Deque wie ein Rangiergleis vorstellen: vorne und hinten können Waggons an- oder abgehängt werden, aber nicht einfach ein Waggon aus der Mitte herausgenommen werden. Bei einem einfachen Kopfgleis hingegen könnte man nur an einer Seite Waggons an- und abhängen (wie bei einem Stapel) und auf einem so genannten Ablaufberg bewegen sich die Waggons immer nur in einer Richtung (wie bei einer Warteschlange).

So wie die Röhre dieser Aufgabe nur bis zu drei Bälle enthalten kann, hat auch eine Deque praktisch eine begrenzte Kapazität. Wenn einer bereits vollkommen ausgelasteten Deque ein Element hinzugefügt wird, muss am anderen Ende Platz gemacht werden. Dies geschieht, indem dort ein Element entfernt wird und die entsprechende Information verloren geht. Speicherplatz ist eine endliche Ressource und die Menge der Daten, die man speichern kann, hängt davon ab. Gleichwohl werden Deques häufig als hypothetisch unbegrenzt verwendet, insbesondere wenn es eher um theoretische Fragestellungen als um praktische Anwendungen geht.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

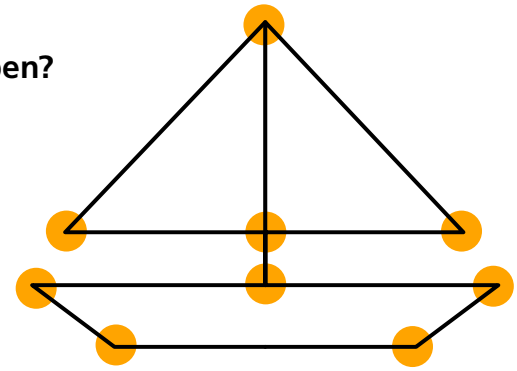
Segelboot



Sophie hat das Segelboot gezeichnet.

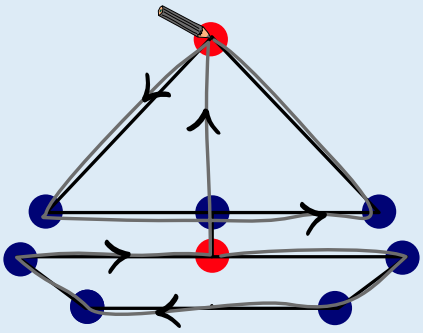
Sie hat
- das Segelboot in einem Zug gezeichnet, den Stift also niemals angehoben, und
- jede Linie nur genau einmal gezeichnet.

An welchen der unten markierten Punkte kann Sophie mit dem Zeichnen begonnen haben?



So ist es richtig:

Sophie kann an beiden roten Punkten beginnen. Wenn sie beim unteren roten Punkt beginnt, zeichnet sie zuerst den Rumpf des Bootes nach, danach den Mast und das Segel und kommt schließlich beim oberen roten Punkt an (siehe Bild). Beginnt sie beim oberen roten Punkt, kann sie genau umgekehrt vorgehen.

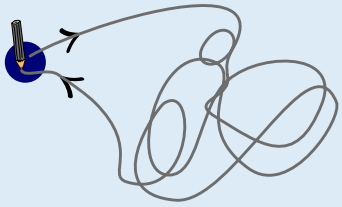


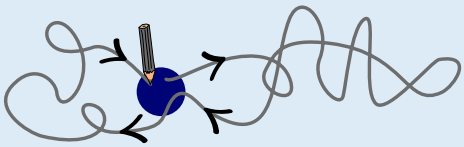
Die beiden Möglichkeiten kann man durch Ausprobieren finden. Man kann aber auch systematisch bestimmen, an welchem Punkt Sophie beginnen kann. Man kann zwischen zwei Arten von Punkten unterscheiden:
- Von den blauen Punkten geht jeweils eine **gerade** Anzahl Linien aus.
- Von den roten Punkten geht jeweils eine **ungerade** Anzahl Linien aus.

Sophie kann entweder bei einem blauen oder bei einem roten Punkt beginnen. Wir probieren beide Alternativen aus:

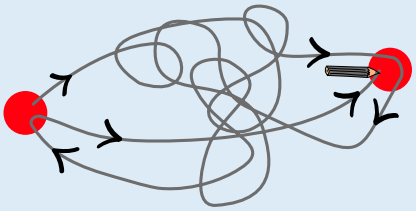
Sophie beginnt bei einem blauen Punkt und zeichnet die erste Linie nach. Um alle von diesem Punkt ausgehenden Linien nachzuzeichnen, muss sie zu diesem Punkt zurückkehren - einmal oder mehrmals. Das ist sicher dann möglich, wenn es nur blaue Punkte gibt: Bei den weiteren blauen Punkten gibt es für jede Linie, über die Sophie beim Nachzeichnen zu diesem Punkt kommt, immer eine noch unbenutzte Linie, über die Sophie den Punkt auch wieder verlassen kann.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0





Sophie beginnt bei einem roten Punkt. Wegen der ungeraden Anzahl an ausgehenden Linien muss Sophie diesen Punkt irgendwann über die letzte nicht nachgezeichnete Linie verlassen und kann dann nicht mehr zu diesem Punkt zurück. Sie muss also einen anderen Endpunkt finden. Hier bietet sich der zweite rote Punkt an: Wegen der ungeraden Anzahl an Linien kann Sophie diesen Punkt irgendwann nicht mehr verlassen. Es gibt also einen Zusammenhang zwischen den zwei roten Punkten: Sophie muss in einem roten Punkt beginnen und im anderen roten Punkt enden.



Sophie konnte das Segelboot so – also in einem Zug und jede Linie genau einmal – nachzeichnen, weil es genau zwei rote Punkte hat. In der richtigen Antwort müssen beide roten Punkte angegeben sein, obwohl es auch schon eine gute Leistung ist, genau einen richtig zu erkennen.

Das ist Informatik!
Die Zeichnung vom Segelboot besteht aus Punkten und aus Linien, die einige der Punkte verbinden. In der Informatik sind Strukturen aus Dingen, von denen manche miteinander verbunden sind, als *Graph* bekannt. Ein Graph besteht aus *Knoten* und *Kanten.* Die Segelboot-Zeichnung ist also die Darstellung eines Graphen: Die Punkte sind die Knoten, und die Linien sind die Kanten, die jeweils zwei Knoten miteinander verbinden. Ein Knoten kann mehrere Kanten haben, die von ihm ausgehen, also ihn mit anderen Knoten verbinden. Ein Weg in einem Graphen ist durch eine Folge von Knoten bestimmt und geht über Kanten, die jeweils zwei im Weg aufeinanderfolgende Knoten miteinander verbinden.

In dieser Biberaufgabe will Sophie einen Weg durch den Segelboot-Graphen finden, der durch alle Knoten geht und genau einmal über jede Kante. Die Knoten dürfen dabei beliebig oft besucht werden. Damit das Problem lösbar ist, darf es keine Knoten geben, die man gar nicht erreichen kann. Ein Graph, der diese Voraussetzung erfüllt, ist *zusammenhängend.* Das Haus vom Nikolaus ist ein bekanntes Rätsel, bei dem man einen solchen Weg in einem zusammenhängenden Graph finden muss.

Der Schweizer Mathematiker Leonhard Euler (1707-1783) hat massgeblich dazu beigetragen, diese Art von Problemen zu verstehen und zu lösen. In Anlehnung an seine Forschungsarbeit ist ein Weg, wie ihn Sophie durch den Segelboot-Graphen finden will, in der Informatik als *Eulerweg* bekannt. Ein bekannter Algorithmus, der festellt, ob es in einem zusammenhängenden Graphen einen Eulerweg gibt, überprüft, ob ein Graph Knoten mit einer *ungeraden Anzahl* ausgehender Kanten hat. Einen Eulerweg gibt es in genau zwei Fällen:

- Wenn es *genau zwei* solcher Knoten gibt, dann startet der Eulerweg in einem dieser Knoten und endet im anderen Knoten.
- Hat ein Graph nur Knoten mit einer geraden Anzahl ausgehender Kanten, dann kann man von *jedem Knoten* aus einen Eulerweg konstruieren. Alle diese Eulerwege haben eine kreisartige Form, weil sie im selben Knoten starten und enden. In der Informatik redet man dann von einem *Eulerkreis.*

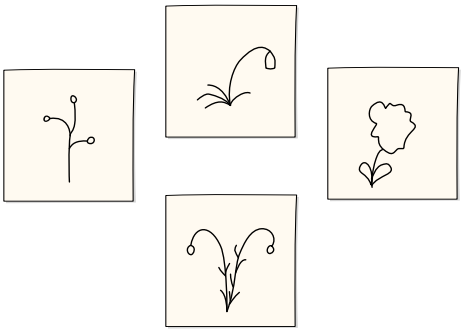
Eulerwege und Eulerkreise haben zahlreiche Anwendungen im alltäglichen Leben. Wenn man einen Spaziergang durch ein Dorf plant, kann man versuchen einen Eulerweg oder einen Eulerkreis zu konstruieren. Findet man einen, dann kann man alle Straßen nacheinander besuchen ohne eine Straße jemals doppelt zu benutzen.

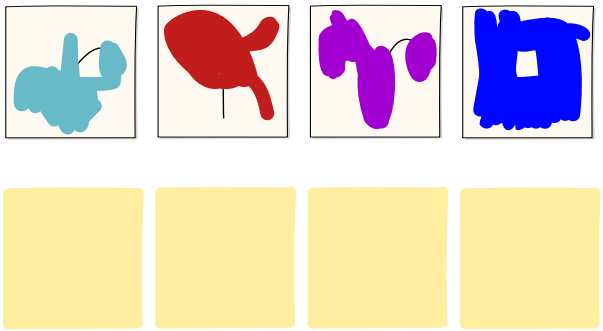
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Fingerfarben

Lars hat Blumen gezeichnet. Seine kleine Schwester Carlotta findet die Zeichnungen und bemalt sie mit Fingerfarben.

Wie sahen die bemalten Zeichnungen vorher aus?



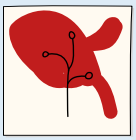


So ist es richtig:
Hier siehst Du, was sich auf den bemalten Zeichnungen (Bilder A bis D) hinter den Fingerfarben verbirgt.

A)



B)



C)



D)



Um die richtige Antwort zu finden, vergleichen wir
1. die schwarzen Linien jeder bemalten Zeichnung mit den Linien aller unbemalten Zeichnungen und
2. die weißen Bereiche jeder bemalten Zeichnung mit den weißen Bereichen aller unbemalten Zeichnungen.

Dabei stellen wir fest:
- Es gibt nur eine unbemalte Zeichnung mit einem weißen Bereich in der oberen linken Ecke wie in Bild A.
- Es gibt nur eine unbemalte Zeichnung mit einem Stiel in der unteren Hälfte wie in Bild B.
- Es gibt nur eine unbemalte Zeichnung mit einem weißen Bereich in der Mitte wie in Bild D.
- Damit bleibt genau eine unbemalte Zeichnung für Bild C übrig.

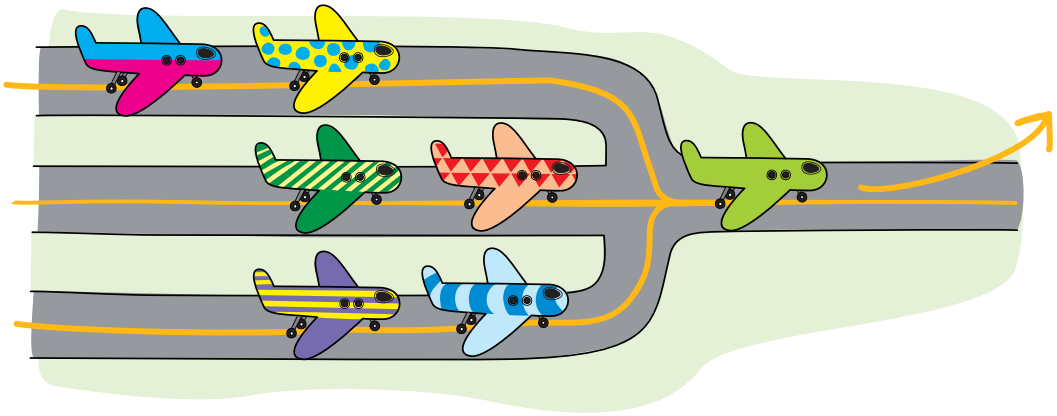
Das ist Informatik!
Die kleine Carlotta hat die Zeichnungen ihres Bruders mit Fingerfarben bemalt. Danach konnte man kaum noch erkennen, wie die unbemalten Zeichnungen aussahen. Bei der Restaurierung von Kunstwerken in Museen bemühen sich Expertinnen und Experten, übermalte oder beschädigte Kunstwerke wieder in ihren ursprünglichen Zustand zu versetzen. Dabei müssen sie oft passende Referenzen aus anderen Quellen finden, um das Originalwerk genau wiederherzustellen. Dieser Prozess ähnelt dem, was du gerade in dieser Biberaufgabe gemacht hast.

In der Informatik werden Restaurierungstechniken in Systemen verwendet, die völlig neue Bilder erzeugen. Diese fortschrittlichen Systeme werden als *Diffusionsmodelle* bezeichnet. Ein Diffusionsmodell wird anhand zahlreicher Bilder trainiert, um zu lernen, wie beschädigte digitale Bilder restauriert werden können. Mit diesem Wissen kann das System dann aus Bildern, die aus vielen kleinen, verstreuten Punkten bestehen, neue Bilder erstellen. Es gibt jedoch einen wesentlichen Unterschied zu deinem Vorgehen bei dieser Aufgabe. Du hast die Aufgabe mit logischem Denken gelöst, während das Diffusionsmodell nicht mit Logik arbeitet. Stattdessen erlangt es seine Restaurierungsfähigkeiten, indem es mithilfe von *maschinellem Lernen* viele Beispiele analysiert.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Flugzeuge

Heute Morgen wollen sieben Flugzeuge starten. Alle starten auf derselben Startbahn rechts. Die Flugzeuge fahren auf den Linien vorwärts und können einander nicht überholen.



Der Startplan zeigt, in welcher Reihenfolge die sieben Flugzeuge starten. Einige Flugzeuge fehlen aber noch.

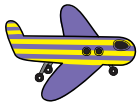
Ergänze die fehlenden Flugzeuge im Startplan.

Zeit	
10:45	
10:52	
10:55	
10:59	
11:00	
11:10	
11:11	









Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:

Zeit	
10:45	
10:52	
10:55	
10:59	
11:00	
11:10	
11:11	

1. Als erstes (um 10:45 Uhr) startet das Flugzeug ; es steht schon auf der Startbahn.
2. Als zweites (um 10:52 Uhr) startet das Flugzeug .
3. Als drittes (um 10:55 Uhr) startet das Flugzeug : Es steht vor dem Flugzeug and . Damit das als nächstes starten kann, wie im Plan angegeben, muss vorher das Flugzeug gestartet sein.
4. Als viertes (um 10:59 Uhr) startet das Flugzeug and .
5. Als fünftes (um 11:00 Uhr) startet das Flugzeug : Es steht vor dem Flugzeug and . Damit das als nächstes starten kann, wie im Plan angegeben, muss vorher das Flugzeug gestartet sein.
6. Als sechstes (um 11:10 Uhr) startet das Flugzeug and .
7. Als siebtes und letztes (um 11:11 Uhr) startet das Flugzeug . Alle anderen Flugzeuge sind schon gestartet.

Das ist Informatik!
Die Startreihenfolge der Flugzeuge hängt auch von der Reihenfolge beim Warten ab. Einige Flugzeuge können erst dann starten, wenn andere, die beim Warten vor ihnen stehen, bereits gestartet sind. An einem echten Flughafen müssen bei der Planung der Startreihenfolge noch viele andere Bedingungen berücksichtigt werden, zum Beispiel Verspätungen, technische Probleme oder Wetteränderungen. Für die Planung werden meist Computerprogramme verwendet, die auf Änderungen bei den Bedingungen schnell reagieren und dennoch gute Pläne erstellen können.

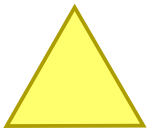
Die Erstellung von Zeitplänen, wie dem Startplan in dieser Biberaufgabe, fällt in der Informatik in den Bereich *Scheduling*. Scheduling kann kompliziert werden, zum Beispiel wenn nur eine Ressource (z. B. eine Startbahn) zur Verfügung steht, wenn bestimmte Ereignisse voneinander abhängig sind oder wenn eine Reihenfolge eingehalten werden muss, um Probleme zu vermeiden.

In der Informatik gilt Scheduling als besonders schwieriges Problem. Die Herausforderung besteht darin, für jede denkbare Situation einen optimalen Plan zu berechnen. Wenn viele Parameter und Bedingungen berücksichtigt werden müssen, kann selbst ein sehr schneller Computer dafür enorm viel Zeit benötigen. Informatik-Fachleute sagen: Scheduling ist *NP-schwer.* Das bedeutet: Es ist zwar einfach zu überprüfen, ob ein gegebener Plan korrekt ist, aber extrem schwierig, den besten Plan überhaupt zu finden. Das ist wie bei einem besonders schwierigen Rätsel: Die Lösung zu prüfen ist leicht, sie zu finden, kann dagegen sehr aufwändig sein.

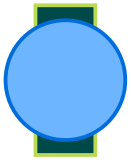
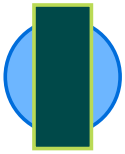
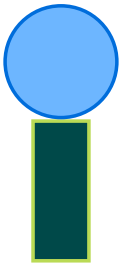
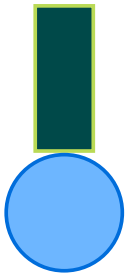
Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Hivobu

Im Land Hivobu heißen diese drei Formen:

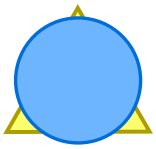
RAH	OH	TEH
		

Wenn man in Hivobu zwei Formen hintereinander oder untereinander legt, heißt das so:

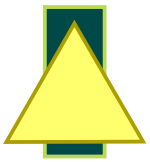
OH RAH CO	RAH OH CO	OH RAH DU	RAH OH DU
			

Was heißt in Hivobu: TEH OH CO?

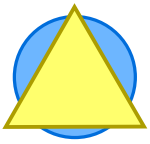
A)



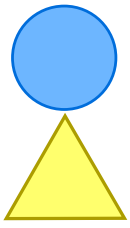
B)



C)



D)



Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Antwort C ist richtig:

Wir wissen, wie die einzelnen Formen heißen:

• Das Rechteck  heißt: RAH;

• der Kreis  heißt: OH; und

• das Dreieck  heißt: TEH.

Wir wissen auch, wie es heißt, wenn man zwei Formen hintereinander oder untereinander legt: Zuerst kommt der Name der ersten Form, dann der Name der zweiten Form, und dann heißt es,

- wenn die zweite Form hinter der ersten Form liegt: CO;
- wenn die zweite Form unter der ersten liegt: DU.

Wenn wir also, wie in Antwort C, ein Dreieck (TEH) haben und einen Kreis (OH), der hinter (CO) dem Dreieck liegt, heißt das: TEH OH CO

Aber wie heißen die anderen Antworten in Hivobu?

- Antwort A heißt: OH TEH CO – ein Kreis (OH) und ein Dreieck (TEH), das hinter (CO) dem Kreis liegt.
- Antwort B heißt: TEH RAH CO – ein Dreieck (TEH) und ein Rechteck (RAH), das hinter (CO) dem Dreieck liegt.
- Antwort D heißt: OH TEH DU – ein Kreis (OH) und ein Dreieck (TEH), das unter (DU) dem Kreis liegt.

Das ist Informatik!
Computer sind schlau? Stimmt, sie können sehr schnell komplizierte Dinge ausrechnen, Informationen im Internet finden und vieles mehr. Stimmt aber auch nicht, denn damit sie das können, muss man ihnen sehr genau sagen, wie sie das tun sollen. Auch den KI-Systemen, mit denen wir scheinbar ganz normal reden können, muss man erst einmal mühsam sagen, wie das geht.

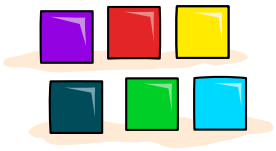
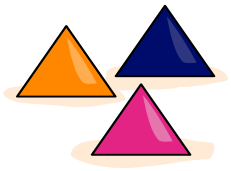
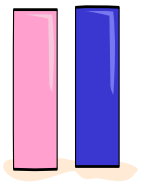
Im Grunde verstehen Computer nämlich nur genaue Anweisungen, die man in Sprachen mit klaren Strukturen formulieren muss. In der Informatik heißen solche Sprachen auch *formale Sprachen,* und Computer verstehen nur diejenigen formalen Sprachen gut, die eine eher einfache Struktur haben.

Das ist so wie in dieser Biberaufgabe: Das, was wir von der Sprache in Hivobu kennengelernt haben, wenn es um den Umgang mit Formen geht, hat eine sehr einfache Struktur. Zuerst werden zwei Formen genannt, und dann sagt ein Kommando, was mit den beiden Formen passiert. Bringt man diese *Syntax,* also die Struktur der Sprache, durcheinander – wenn man zum Beispiel das Kommando in der Mitte sagt statt am Schluss –, weiß niemand in Hivobu, was gemeint ist. In der Informatik kennt man diese Art der Syntax (erst die Dinge sagen, dann das Kommando) als *Postfix-Notation.* Auch ein Computer, der eine Anweisung in Postfix-Notation erwartet, käme bei einem Fehler durcheinander.

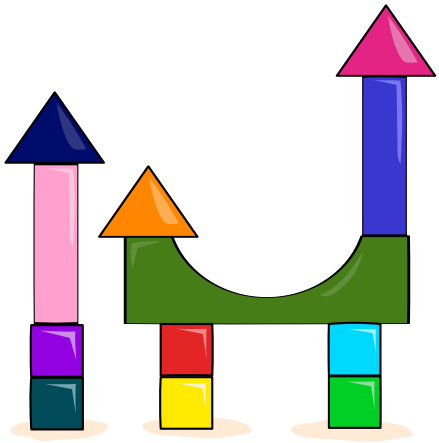
Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Bauklötze 1

Ali hat diese Bauklötze:

6 Würfel	1 Brücke	3 Pyramiden	2 Quader
			

Alis Schwester Zaha gibt ihm nach und nach Anweisungen, was er mit den Bauklötzen tun soll. Ali erledigt jede Anweisung sofort. Am Ende entsteht dieses Bauwerk:



In welcher Reihenfolge hat Zaha die Anweisungen gegeben?

Stelle beide Quader auf dein Bauwerk.

Stelle die Würfel-Türme in eine Reihe.

Lege die Pyramiden auf dein Bauwerk.

Baue drei Türme mit jeweils 2 Würfeln.

Lege die Brücke auf dein Bauwerk.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:
Zaha hat die Anweisungen in dieser Reihenfolge gegeben:

Baue drei Türme mit jeweils 2 Würfeln.

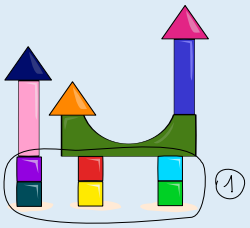
Stelle die Würfel-Türme in eine Reihe.

Lege die Brücke auf dein Bauwerk.

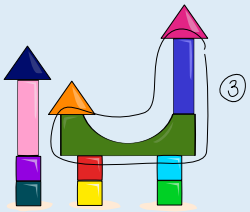
Stelle beide Quader auf dein Bauwerk.

Lege die Pyramiden auf dein Bauwerk.

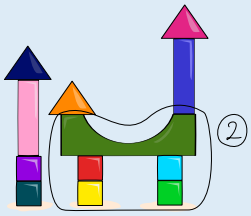
Wir erklären, warum genau diese Reihenfolge Alis Bauwerk entstehen lässt.



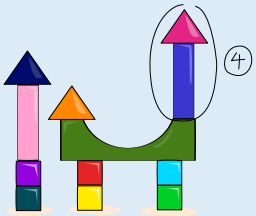
Weil die Würfel-Türme die einzigen Teile sind, die direkt auf dem Boden stehen, müssen sie als erste gebaut (1. Anweisung) und danach in eine Reihe gestellt werden (2. Anweisung).



Danach müssen die Quader aufgestellt werden (4. Anweisung). Sie stehen nämlich direkt auf der Brücke, sind aber unter den Pyramiden.



Danach muss die Brücke auf das bisherige Bauwerk gelegt werden (3. Anweisung). Sie muss auf die Würfel-Türme gelegt werden, aber unter den Quadern und Pyramiden sein.



Zuletzt werden die Pyramiden gelegt (5. Anweisung). Sie liegen auf den Quadern, und es gibt keinen anderen Klotz, der auf die Pyramiden gebaut werden soll.

Da die meisten Anweisungen sagen, dass Klötze auf das (bisherige) Bauwerk gesetzt werden sollen, hängt die Reihenfolge der Anweisungen direkt damit zusammen, welche Klötze auf bzw. unter welchen anderen stehen.

Es ist nicht möglich, das Bauwerk mit einer anderen Reihenfolge der Anweisungen zu bauen.

Das ist Informatik!
Wenn eine Menge von Bauklötzen einzeln nebeneinander auf dem Boden stehen, kann man im Nachhinein nicht sagen, in welcher Reihenfolge die Klötze auf den Boden gestellt wurden. Die Handlungen, also das Legen oder Stellen eines Klotzes auf den Boden, sind voneinander unabhängig. Wenn man Bauklötze aufeinander stellt, geht das nur der Reihe nach, und der unterste Klotz wurde zuerst, der oberste Klotz zuletzt benutzt. Das Ablegen des untersten Klotzes ist Voraussetzung dafür, dass der nächste Klotz darauf abgelegt werden kann.

Computerprogramme bestehen aus einer Folge einzelner Anweisungen, so wie die von Zaha in dieser Biberaufgabe. Dabei können die einzelnen Anweisungen einfach sein und entsprechend einfache Auswirkungen haben, sie können aber auch sehr kompliziert sein. Auf jeden Fall ist es wichtig, dass Informatikerinnen und Informatiker sich beim Programmieren sehr gut überlegen, welche Wirkungen die einzelnen Anweisungen haben – und welche Wirkungen anderer Anweisungen eine Anweisung voraussetzt. Wenn man sich das vor dem Programmieren genau überlegt, hat man keine Schwierigkeiten, die Anweisungen im Programm in die richtige Reihenfolge zu bringen – so wie du das in dieser Biberaufgabe gemacht hast.

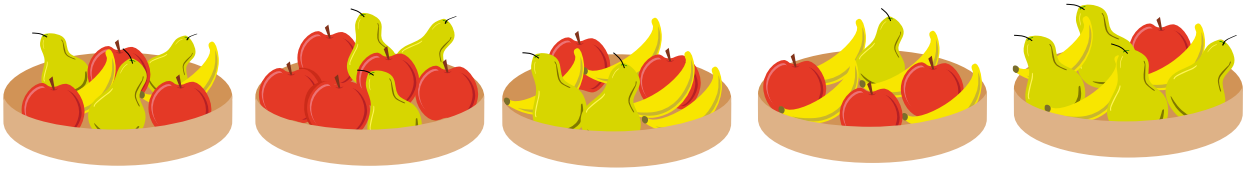
Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Obstkörbe

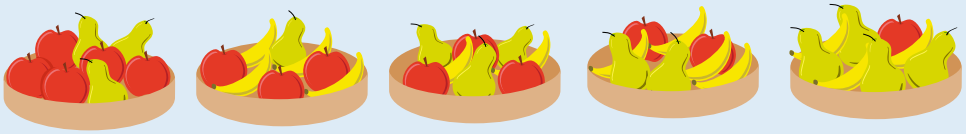
Bella mag Obst. Am liebsten mag sie Äpfel, dann Bananen und am wenigsten Birnen.

Bella hat fünf Körbe mit Obst. Sie möchte diese Körbe nach ihrer Vorliebe anordnen. Je mehr Äpfel ein Korb hat, desto weiter links soll er stehen. Wenn zwei Körbe gleich viele Äpfel haben, soll der Korb mit mehr Bananen weiter links stehen.

Ordne die Körbe so an, wie Bella das möchte.



So ist es richtig:



Wir benennen die Körbe mit den Buchstaben A bis E. Dann zählen wir die Obstsorten in jedem Korb.

Korb	Bild	Anzahl Äpfel	Anzahl Bananen	Anzahl Birnen
A)		3	2	3
B)		5	0	3
C)		2	4	2
D)		3	3	1
E)		1	3	4

In Korb B sind die meisten Äpfel, nämlich 5; er muss ganz links stehen. Danach kommen die Körbe A und D mit jeweils 3 Äpfeln. Weil Korb D mehr Bananen hat, kommt er als nächster in die Reihe, danach Korb A. Von den beiden übrigen Körben kommt Korb C mit 2 Äpfeln vor Korb E mit nur 1 Apfel. Insgesamt lautet die Reihenfolge also: B, D, A, C, E

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Das ist Informatik!
Bella möchte in dieser Biberaufgabe die Obstkörbe nach ihren Vorstellungen *sortieren,* also in eine ganz bestimmte Ordnung bringen. Da sie Äpfel am liebsten mag, ist es für sie sinnvoll, den Korb mit den meisten Äpfeln ganz vorne in der Reihe zu haben: Wenn die Körbe so sortiert sind, weiß sie genau, wie sie am schnellsten an viele Äpfel kommt.

Man kann Dinge sortieren, wenn man für jeweils zwei Dinge immer entscheiden kann, in welcher Ordnung sie zueinander stehen. Dazu braucht es mindestens ein Merkmal, das alle zu sortierenden Dinge haben, und nach dem man sie unterscheiden kann. Für die Werte, die das Merkmal haben kann, muss es eine klare Ordnung geben. Zum Beispiel hat jedes Buch mindestens einen Autor oder eine Autorin, und wir können den Namen des ersten Autors bzw. der ersten Autorin als Merkmal nehmen. Namen kann man alphabetisch ordnen, wenn man alle Buchstaben und auch die Länge der Namen berücksichtigt. Dann kommt Meier vor Meyer, und Schach kommt vor Schacht. So können wir die Bücher nach dem Namen des ersten Autors bzw. der ersten Autorin sortieren.

Für die Informatik ist Sortieren eine ganz wichtige Angelegenheit. Computer sortieren ständig Daten, insbesondere weil sie in sortierten Daten schneller suchen können. Die Informatik kennt sehr viele Verfahren (in der Fachsprache *Algorithmen* genannt), mit denen Daten sortiert werden können. Es ist selbst für Informatikerinnen und Informatiker nicht immer einfach zu entscheiden, welches davon für eine gegebene Sortieraufgabe am besten geeignet ist.



Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Nach Hause

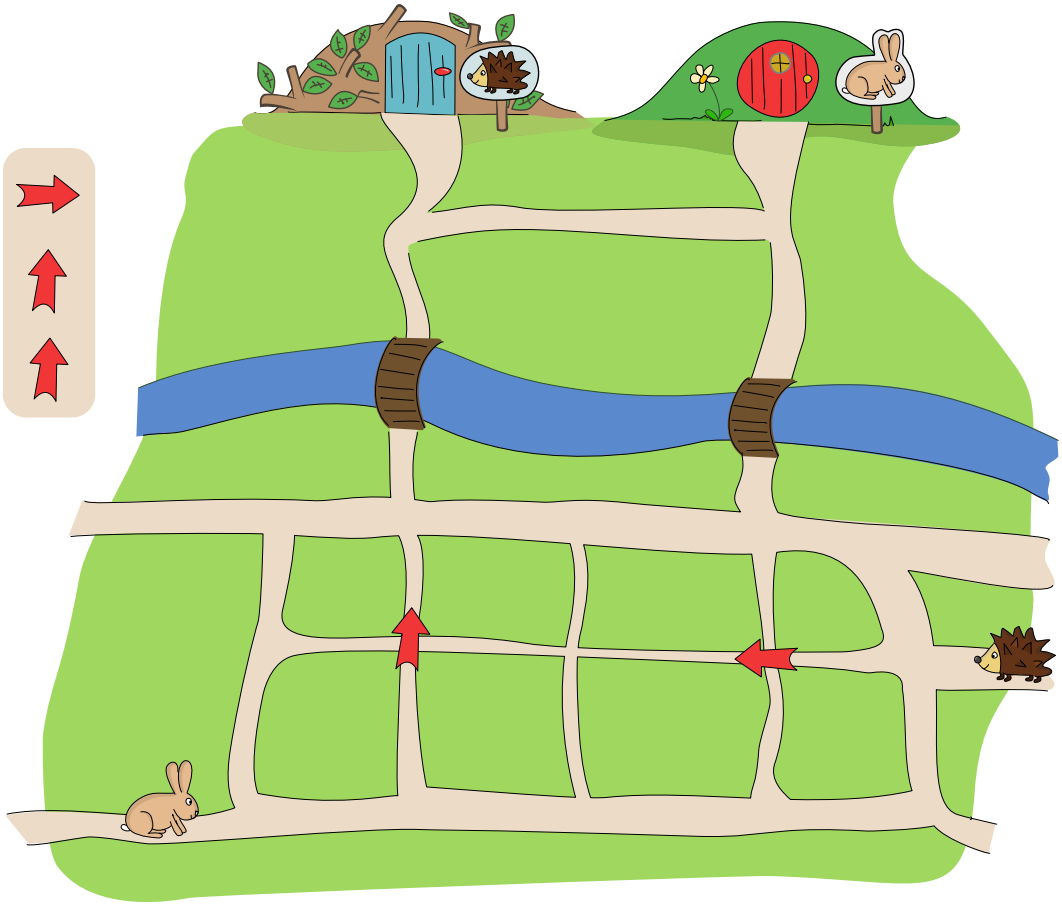
Ein Hase und ein Igel wollen nach Hause.

Jeder hat ein eigenes Haus:  und  .

Hase und Igel laufen auf den Wegen, und zwar geradeaus. Nur wenn sie zu einer Kreuzung mit Pfeil kommen, folgen sie der Richtung des Pfeils.

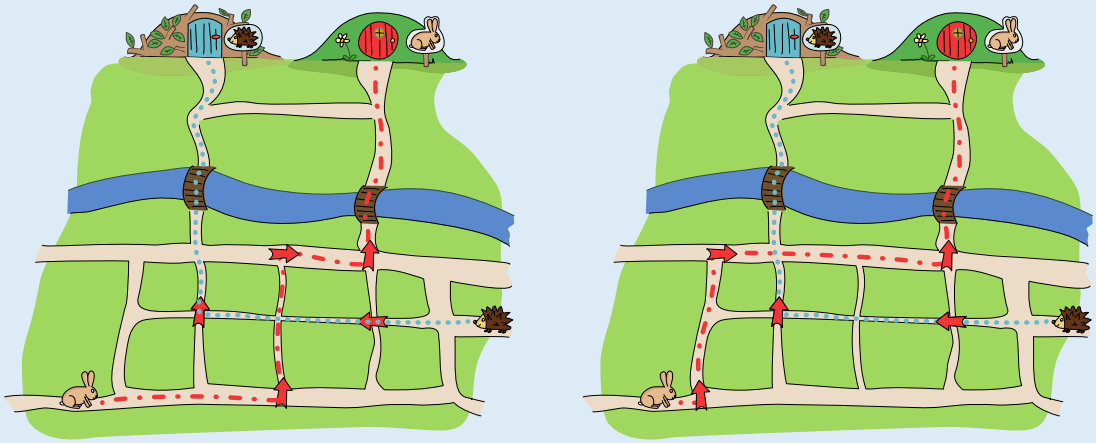
An einigen Kreuzungen liegen schon Pfeile. Entlang der Pfeile findet der Igel nach Hause, der Hase aber nicht. Zum Glück sind noch drei Pfeile übrig.

Zeichne die drei übrigen Pfeile auf Kreuzungen, so dass Hase und Igel beide nach Hause finden.



Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Es gibt zwei richtige Antworten:



Um nach Hause zu kommen, muss der Hase irgendwann nach „oben“ (in Richtung der Häuser) gehen. Das kann er an der ersten oder der dritten Kreuzung machen, an die er kommt.

Wichtig ist, dass Hase und Igel auf ihren Wegen nie zu demselben Pfeil kommen, weil sie sonst beide von dort aus denselben Weg laufen und somit beide am selben Haus ankommen würden. Das würde zum Beispiel passieren, wenn der Hase an der zweiten oder vierten Kreuzung nach oben geht und zu Pfeilen kommt, die dem Igel den Weg nach Hause weisen. Auch an der fünften Kreuzung kann der Hase nicht nach oben gehen; er müsste dann später nach links, aber es ist kein Pfeil übrig, der nach links zeigt.

Der Hase kann also nur an der ersten oder dritten Kreuzung nach oben gehen. Die anderen Pfeile müssen dann jeweils so gelegt werden wie oben abgebildet. Nur so kommt der Hase nach Hause, und der Weg des Igels nach Hause wird nicht gestört.

Das ist Informatik!
Auf ihrem Weg folgen Hase und Igel an den Kreuzungen dieser Vorschrift: „Wenn an der Kreuzung ein Pfeil liegt, folge der Richtung des Pfeils. Sonst gehe geradeaus.“

In der Informatik nennt man solch eine Vorschrift *Algorithmus.* Der Algorithmus bleibt immer gleich: Er sagt, ob und wie man die Richtung ändert, wenn man an eine Kreuzung kommt. Die Richtungsänderungen werden von den Pfeilen bestimmt. Sie sind Daten, die der Algorithmus *verarbeitet;* in der Informatik spricht man auch von *Eingaben* für den Algorithmus. Im Hase-Igel-Wege-Algorithmus dieser Biberaufgabe bestimmt die Eingabe, also Lage und Art der Pfeile, ob die Ausgabe des Algorithmus richtig ist, also ob die Wege, welche die Tiere gehen, sie beide nach Hause führen.

Eine ähnliche Vorschrift könnte es beim Waschen eines Kleidungsstücks geben: „Wähle die Waschtemperatur so wie auf dem Waschzettel angegeben.“ Nur wenn die Eingabe, also die Temperatur-Angabe auf dem Waschzettel, stimmt, ist auch die Ausgabe richtig, also ein sauberes Kleidungsstück, das noch genau so groß ist wie vor dem Waschen.

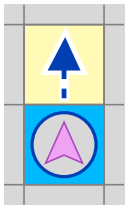
Diese Beispiele zeigen, dass es kritisch ist, wenn die Qualität der Ausgaben eines Algorithmus von der Qualität der Eingaben abhängt. Das ist zum Beispiel bei vielen KI-Systemen der Fall. KI-Systeme arbeiten mit Modellen der Wirklichkeit, die aus Daten gebildet werden. Sind diese Daten nicht gut ausgewählt, wird das Modell nicht gut funktionieren: es hat einen *Bias* (deutsch: Verzerrung). Ein KI-Modell von Krankheiten etwa wird für Frauen nicht gut funktionieren, wenn die Eingabedaten nur von Männern stammen.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

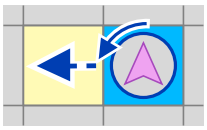
Lefty 1

Roboter Lefty bewegt sich über ein Raster mit quadratischen Feldern. Zwischen Feldern kann es rote Mauern geben. Lefty soll das grüne Ziel erreichen.
Lefty kann sich auf genau zwei Arten bewegen:

Ein Feld vorwärts fahren

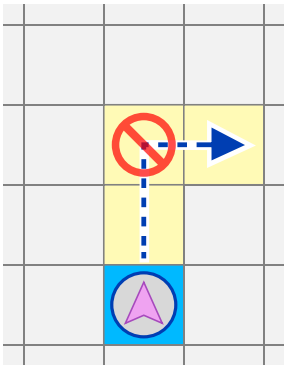


Nach links drehen und dann sofort ein Feld vorwärts fahren

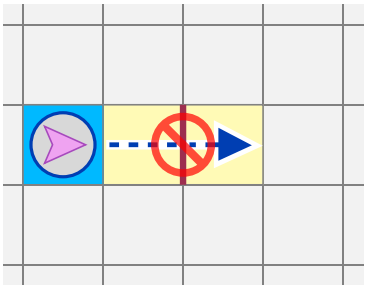


Lefty kann aber nicht alles. Zum Beispiel kann er ...

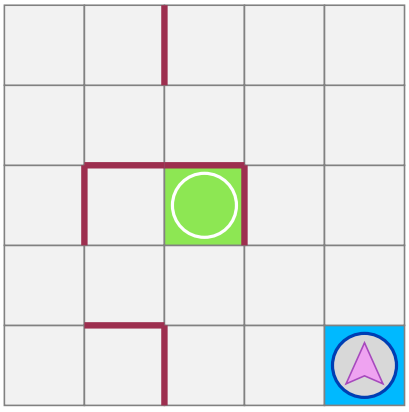
... nicht einfach rechts abbiegen und ...



… **nicht** durch Mauern fahren.

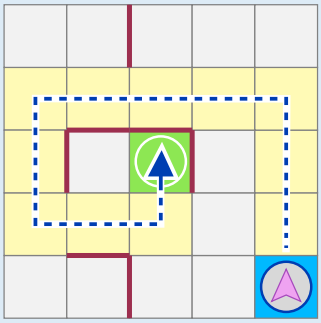


Über welche Felder muss Lefty fahren, um das Ziel zu erreichen?
Wähle so wenige Felder wie möglich aus.



Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

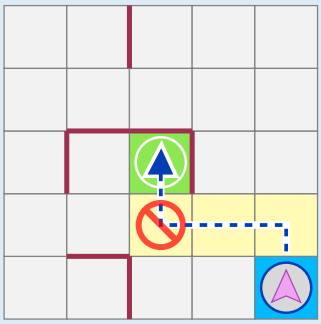
So ist es richtig:



Wenn Lefty über diese Felder fährt, erreicht er das Ziel. Dabei bewegt er sich nur auf die beiden Arten, die er kann.

Es gibt keinen anderen Weg mit weniger oder gleich vielen Feldern, auf dem Lefty das Ziel erreicht.

Den direkten Weg kann Lefty nicht nehmen, weil er dafür rechts abbiegen müsste.



Das ist Informatik!
Armer Lefty! Seine Funktionsweise ist stark eingeschränkt. Wenn er doch nur andere Bewegungen machen könnte! Wenn er sich nach rechts drehen und vielleicht sogar über Mauern klettern könnte, wäre sein Leben im Raster viel einfacher. Lefty würde sich viel selbstbewusster fühlen, wenn er einen *komplexeren Befehlssatz* hätte. (Bei einem Roboter werden die grundlegenden Dinge, die er tun kann, durch entsprechende Befehle in der Software des Roboters gesteuert).

Aber ist das wirklich notwendig? Lefty könnte sich zum Beispiel nach rechts drehen, indem er sich dreimal hintereinander nach links dreht. Wir müssen nur die Regel abschaffen, dass sich Lefty nach der Linksdrehung sofort nach vorne bewegen muss. Dann könnte er sich in alle Richtungen drehen und fahren. Und anstatt über eine Mauer zu klettern, könnte er um sie herumfahren, wenn dafür genug Platz ist. Hey Lefty, du kannst mit deinem *reduzierten Befehlssatz* glücklich und zufrieden leben – solange die, die dich programmieren, die Befehle klug einsetzen.

In der Informatik sind genau diese beiden Ansätze, den Befehlssatz eines *Prozessors* zu gestalten, am weitesten verbreitet: Manche Prozessoren sind ein CISC (Complex Instruction Set Computer/deutsch: Computer mit komplexem Befehlssatz), andere sind ein RISC (Reduced Instruction Set Computer/Computer mit reduziertem Befehlssatz) – wie jener von Lefty in dieser Biberaufgabe. Ein CISC hat in der Regel viele verschiedene Befehle, die sehr mächtig sein können (wie das Klettern über eine Mauer), aber dafür seltener verwendet werden. Ein RISC hingegen hat nur wirklich nötige Befehle mit eher einfacher Wirkung, die dann häufig verwendet werden.

Beide Arten von Architekturen haben ihre Vor- und Nachteile. Die Prozessoren bekannter Marken sind entweder CISC- oder RISC-Prozessoren, wobei RISC-Prozessoren in letzter Zeit etwas beliebter geworden sind.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Ein besonderer Baum

Jona hat einen besonderen Apfelbaum im Garten:

- Landet ein Vogel  auf dem Baum, wachsen sofort zwei neue Äpfel.

- Klettert ein Eichhörnchen  auf den Baum, fällt ein Apfel runter.
  Wenn kein Apfel am Baum hängt, passiert nichts.

- Besucht eine Schlange  den Baum, verschwinden alle Äpfel sofort.

Heute Morgen hängen 25 Äpfel am Baum.
Dann besuchen einige Tiere nacheinander den Baum, zuletzt ein Eichhörnchen.
Jona hat ihre Reihenfolge genau aufgeschrieben:



Wie viele Äpfel hängen danach am Baum?

A) 3 Äpfel **B)** 7 Äpfel **C)** 17 Äpfel **D)** 31 Äpfel

Antwort B ist richtig.
Nachdem das letzte Eichhörnchen auf den Baum klettert, hängen noch 7 Äpfel am Baum.

Da alle Äpfel sofort verschwinden, wenn eine Schlange den Baum besucht, genügt es, nur die Tiere zu betrachten, die nach der zweiten (und letzten) Schlange den Baum besuchen. Nach dem Besuch dieser Schlange hängen 0 Äpfel am Baum. Dann landen nacheinander vier Vögel auf dem Baum; anschließend hängen am Baum $4 \times 2 = 8$ Äpfel. Zum Schluss klettert noch ein Eichhörnchen auf den Baum: Ein Apfel fällt herunter, und $8 - 1 = 7$ Äpfel bleiben übrig.

Das ist Informatik!
Der Besuch eines Tieres verändert den Zustand des besonderen Apfelbaums – aber nur auf ganz bestimmte Weise: Nur die Anzahl der Äpfel, die am Baum hängen, wird geändert. Auf andere Eigenschaften des Baumes, etwa die Anzahl der Blätter, die Länge einzelner Äste oder die Form der Baumkrone, hat der Besuch eines Tieres keinen Einfluss. Für diese Biberaufgabe ist es also ausreichend, die Anzahl der Äpfel zu betrachten.
Auch ein Computerprogramm hat einen Zustand, der von den einzelnen Anweisungen des Programms verändert wird. Als Zustand werden meist die von einem Programm gespeicherten Daten betrachtet; diese Daten speichert das Programm in den bei der Programmierung eingeführten *Variablen*.
Die Folge der Tierbesuche auf dem Baum in dieser Biberaufgabe ist wie ein Computerprogramm: Jeder Tierbesuch ist eine Anweisung, die den Zustand des Apfelbaums verändert. Dieser Zustand – also die Anzahl der Äpfel, siehe oben – kann in einer einzigen Variable gespeichert werden.
Beim Lösen der Aufgabe ist dir vielleicht aufgefallen, dass du nicht das ganze „Programm“ anschauen musstest, sondern nur den Teil nach dem letzten Vorkommen der Schlange. Durch genaue Betrachtung der Auswirkungen der einzelnen Anweisungen auf den Zustand des Programms konntest du eine besondere Eigenschaft des Programms herausfinden. Eine solche Analyse von (Computer-)Programmen gehört zu den häufigen Tätigkeiten von Informatikerinnen und Informatikern.

Riccas

Evelyn hat fünf Bilder von Riccas. Sie beschreibt in Sätzen, wie Riccas aussehen.



Ihre Freundin Lydia zeigt ihr ein sechstes Bild von einem Ricca:



Nun stellt Evelyn fest: Einer ihrer Sätze über Riccas ist sicher falsch.

Welcher dieser Sätze über Riccas ist nun sicher falsch?

A) Alle Riccas haben Zähne.

B) Einige Riccas haben Flügel.

C) Riccas haben entweder Hörner oder drei Augen.

D) Wenn Riccas genau zwei Arme haben, dann haben sie auch genau zwei Beine.

Antwort D ist richtig:

Antwort A enthält eine Aussage, die für alle Riccas gelten muss. Wenn auch nur ein Ricca keine Zähne hätte, wäre sie falsch. Jedoch haben alle sechs Riccas, die Evelyn nun kennt, Zähne. Also kann Evelyns Satz nicht sicher falsch sein.

Antwort B enthält eine Aussage, die nur für einige Riccas gelten soll. Da eines der sechs Riccas, die Evelyn nun kennt, Flügel hat, ist dieser Satz für die sechs Riccas richtig. Aber selbst wenn keines der sechs Riccas Flügel hätte, könnten andere, weitere Riccas Flügel haben, und der Satz könnte noch richtig sein. Dieser Satz kann nur dann sicher falsch sein, wenn Evelyn alle Riccas kennen würde und keines Flügel hätte.

Antwort C verknüpft zwei Aussagen mit „entweder – oder". Diese verknüpfte Aussage ist genau dann wahr, wenn genau eine der beiden Aussagen wahr ist. Das ist für alle sechs Bilder der Fall: vier Riccas haben Hörner, aber keine drei Augen, die übrigen beiden Riccas haben keine Hörner, aber dafür drei Augen. Damit der Satz falsch wäre, müsste es mindestens ein Ricca mit drei Augen und Hörnern oder ein Ricca ohne Hörner und mit einer anderen Zahl als drei Augen geben. Unter den sechs Riccas, die Evelyn nun kennt, ist kein solches Ricca. Also ist der Satz für die sechs Riccas richtig, und nicht sicher falsch.

Bleibt der Satz von Antwort D. Er ist in Form einer „Wenn – Dann"-Aussage formuliert. Nur wenn die Wenn-Bedingung stimmt, muss auch die Dann-Aussage wahr sein. Die Bedingung ist wahr für alle sechs Riccas, die Evelyn nun kennt: sie alle haben genau zwei Arme. Alle Riccas auf Evelyns ersten fünf Bildern haben ebenfalls genau zwei Beine; für sie ist Evelyns Satz also richtig. Jedoch hat das Ricca auf Lydias Bild mehr als zwei Beine, nämlich fünf. Deshalb ist dieser Satz nun sicher falsch.

Das ist Informatik!
Die Anzahl der Arme, Beine und Augen, und ob Riccas Zähne, Hörner oder Flügel haben, sind *Eigenschaften* von Riccas. Wenn man Riccas beschreibt, formuliert man *Aussagen* über diese Eigenschaften. Das führt zu einem *Modell* davon, was Riccas sind.

Computer haben ebenfalls viele Modelle. Einige sind explizit formuliert wie beispielsweise ein Modell von Schülerinnen und Schülern, das aus Name, Geburtsdatum und Wohnadresse in einer Datenbank besteht. Andere Modelle bilden Computer aus Daten, wenn man ihnen beispielsweise Bilder zum Vergleich beim Training eines neuronalen Netzes gibt.

Evelyns Sätze – also ihr Modell der Riccas in dieser Biberaufgabe – sind als *logische Ausdrücke* formuliert. Einige haben *Quantoren* („(für) alle" oder „es gibt"/„einige"), andere nutzen *logische Operatoren* („entweder – oder" bzw. „wenn – dann"). Diese logischen Ausdrücke sind *formalisiert:* Das heißt, dass es eine Festlegung gibt, wie man sie verwendet und was sie bedeuten.

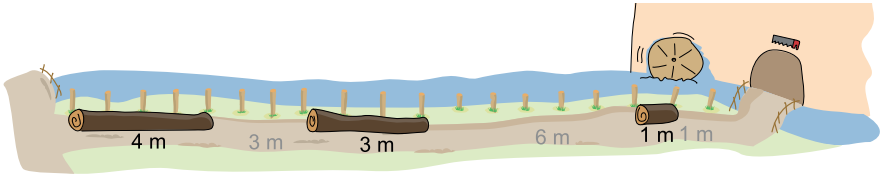
Anhand dieser Festlegungen
- können (einfache) Ausdrücke mit Hilfe von Quantoren und Operatoren zu komplexeren Ausdrücken verknüpft werden.
- kann die Bedeutung der komplexeren Ausdrücke aus den einfacheren Ausdrücken berechnet werden.

Logische Ausdrücke sind eine in der Informatik verbreitete Methode, Modelle zu beschreiben.

Noahs Sägerei

Biber Noah schneidet Holzstämme in verschiedenen Längen zu und verkauft sie dann. Sobald er einen Stamm zugeschnitten hat, legt er ihn auf dem 18 Meter langen Weg ab. Dabei beachtet Noah folgende Regel: Er legt den Stamm in die erste Lücke von links, in die der Stamm passt.

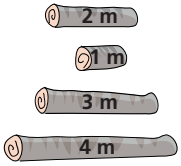
Noah verkauft einige Stämme. Danach gibt es drei Lücken auf dem Weg:



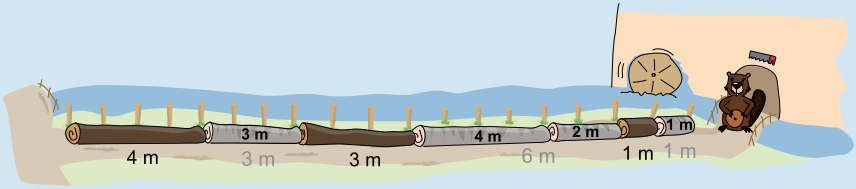
Nun will Noah vier Stämme zuschneiden, mit Längen von 1, 2, 3 und 4 Metern.

In welcher Reihenfolge muss Noah die Stämme zuschneiden, damit er alle vier in die Lücken legen kann?

1	2	3	4



So ist es richtig:



Schneidet Noah die Stämme in der Reihenfolge (3 m, 4 m, 2 m, 1 m) zu, passen sie alle auf den Weg: Für den 3 m-Stamm ist die 3 m-Lücke ganz links die erste freie Lücke von links, in die der Stamm passt; dort legt Noah den Stamm ab. Der 4 m-Stamm kommt dann in die 6 m-Lücke links. Dann ist die verbleibende 2 m-Lücke die erste freie Lücke von links; da rein passt der nächste Stamm, und den letzten Stamm legt Noah in die 1 m-Lücke ab.

Weitere richtige Reihenfolgen sind (3 m, 2 m, 4 m, 1 m) und (4 m, 3 m, 2 m, 1 m).

Alle anderen Reihenfolgen führen dazu, dass Noah nicht in der Lage ist, alle Baumstämme abzulegen: Der 1 m lange Stamm muss immer als letztes an der Reihe sein, weil nur dieser Stamm den letzten freien Platz ausfüllen kann. Der 2 m-Stamm darf nicht vor dem 3 m-Stamm kommen, weil er sonst in die 3 m-Lücke gelegt würde und dadurch eine zweite 1 m-Lücke entsteht. Außer den drei genannten Reihenfolgen gibt es keine Reihenfolgen, die diese Bedingungen erfüllen.

Das ist Informatik!
Diese Biberaufgabe ist ein Spezialfall des *Behälterproblems* (Englisch auch *bin packing problem*). Beim Behälterproblem müssen Objekte unterschiedlicher Größen in einer bestimmten Anzahl von Behältern untergebracht werden, die selbst auch wieder unterschiedliche Größen haben können. Die Objekte sind hier die Baumstämme, die Behälter die Lücken auf dem Weg.

Das Behälterproblem kommt in ganz unterschiedlichen Lebensbereichen vor. Einige Beispiele:
(a) In einem Möbellager müssen kleine und große Möbel platzsparend untergebracht werden.
(b) Eine Spedition kann Geld sparen, wenn sie zum Transport von Gütern durch geschicktes Packen weniger Lastwagen braucht.
(c) Das Betriebssystem eines Computers muss Dateien unterschiedlicher Größe auf der Festplatte speichern. Wenn Dateien gelöscht werden, entstehen Lücken auf der Festplatte. Diese Lücken müssen gefüllt werden, damit kein Speicherplatz verschwendet wird, ganz ähnlich wie auf der Straße bei dieser Biberaufgabe.

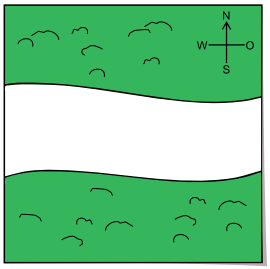
In der Informatik gilt das Behälterproblem als eines der schwersten Probleme; garantiert optimale Lösungen können auch von Computerprogrammen nur für kleine Fälle mit wenigen Objekten und wenigen Behältern gelöst werden. Es gibt aber verschiedene Verfahren und Strategien, mit denen gute Lösungen des Behälterproblems bestimmt werden können. In dieser Biberaufgabe ist die Strategie durch Noahs Regel vorgegeben. Sie legt jeden Baumstamm immer in die erste Lücke von links, in die er passt. Diese Strategie nennt man *First Fit.* Man sieht am Beispiel dieser Aufgabe, dass diese Strategie zu schlechten Ergebnissen führen kann: Nur wenn die Stämme in einer bestimmten Reihenfolge abgelegt werden, können alle Lücken gefüllt werden.

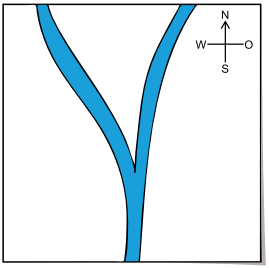


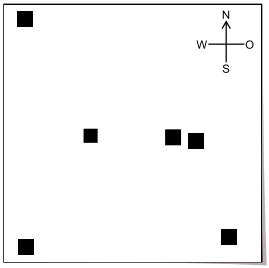
Karlas Traumhaus

Karla hat drei Karten, die alle genau das gleiche Gebiet zeigen. Eine Karte zeigt die Wälder, eine die Flüsse und eine die Häuser in diesem Gebiet. Karlas Traumhaus liegt im Wald und in der Nähe eines Flusses.

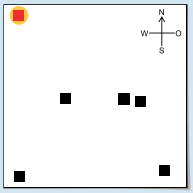
Welches ist Karlas Traumhaus?



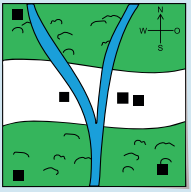




So ist es richtig:
Das Haus oben links auf der Hauskarte ist Karlas Traumhaus.



Um Karlas Traumhaus zu finden, müssen die Informationen aus allen drei Karten ausgewertet werden. Das Traumhaus muss sich in einem Waldgebiet und in der Nähe eines Flusses befinden. Das trifft nur auf das Haus oben links zu. Dies ist leicht zu erkennen, wenn die Karten übereinander gelegt werden:



Das ist Informatik!
Wenn die Informationen über die Wälder, die Flüsse und die Häuser auf einer einzigen Karte dargestellt sind, ist es einfach, das gesuchte Haus zu finden.

Ein *Geoinformationssystem* (GIS) führt eine Vielzahl räumlicher Informationen (z.B. Wälder, Straßen, Landesgrenzen, Tankstellen, Rathäuser, Überschwemmungsgebiete usw.) zusammen und stellt diese auf einer Karte dar. Ein GIS dient also der Visualisierung und Analyse sogenannter *Geodaten.* Mit Hilfe eines GIS ist es z.B. für Katastrophenschutzbeauftragte möglich, Informationen für Evakuierungspläne zusammenzustellen.

Die Verwendung mehrerer Ebenen mit unterschiedlichen Bildinformationen ist auch aus Grafikprogrammen bekannt. Eine wichtige Frage ist immer, welche Ebene mit den darin enthaltenen Objekten die oberste ist und deshalb im Vordergrund dargestellt wird. Im Beispiel sollte die Hauskarte die oberste Ebene sein, damit die Häuser nicht von den Waldflächen verdeckt werden.

Nebeneinander

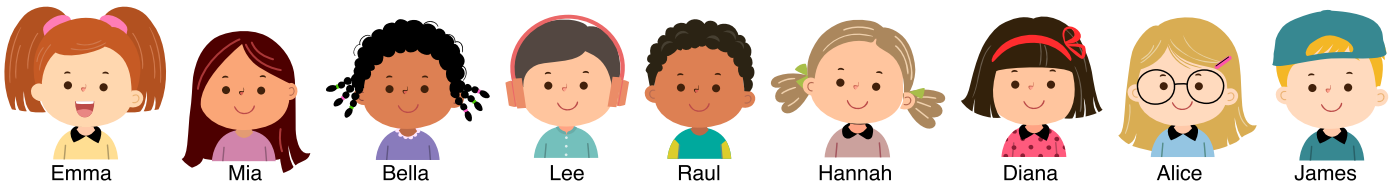
Ava hat die Schule gewechselt. Heute trifft sie ihre Freunde von der alten Schule online. Ihre Freunde sitzen alle nebeneinander in einer Reihe.

Jedes Kind sitzt vor einer eigenen Kamera. In jedem Kamerabild sind aber auch die Kinder zu sehen, die direkt daneben sitzen.

So sieht Ava ihre Freunde auf ihrem Bildschirm:



Wie sitzen die Freunde nebeneinander in der Reihe?



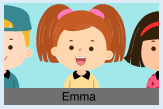
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:

Die Kamerabilder verraten, wer neben wem sitzt.



Im Kamerabild rechts unten auf dem Bildschirm ist links neben James kein Kind zu sehen. Deshalb sitzt James an einem Ende der Reihe. Das Kind, das rechts neben James zu erkennen ist ...



... ist im Bild links oben auf dem Bildschirm zu sehen: Emma. Das Kind, das rechts neben Emma zu erkennen ist, …

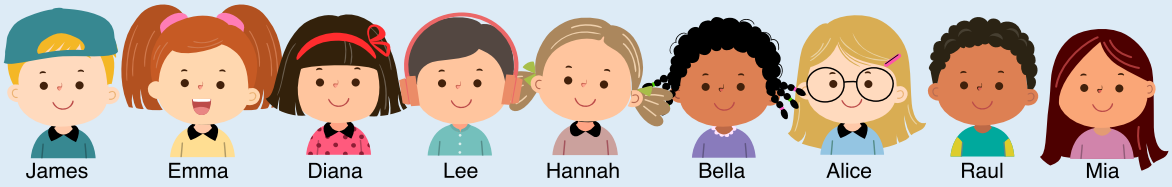


… ist im Bild links unten zu sehen: Diana. Das Kind, das rechts …



… ist im Bild oben mitte zu sehen: Mia. Rechts neben Mia ist kein Kind zu sehen. Deshalb sitzt Mia am anderen Ende der Reihe.

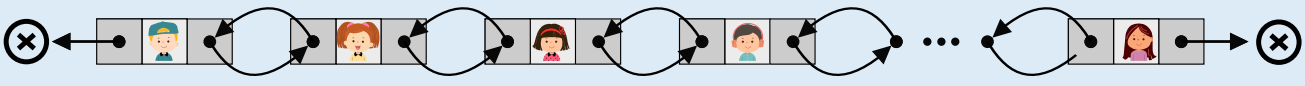
Insgesamt zeigen die Kamerabilder, dass die neun Freunde so nebeneinander sitzen:



Das ist Informatik!
Auf den Kamerabildern sieht man nicht nur ein Kind, sondern kann auch erkennen, welche Kinder links und rechts daneben sind. Informatikerinnen und Informatiker würden sagen: Die Bilder enthalten *Verweise* auf die Kinder daneben, und die Bilder sind (überwiegend) in zwei Richtungen miteinander *verkettet.*

Die Kamerabilder in dieser Biberaufgabe bilden aus Informatik-Sicht eine *doppelt verkettete* Liste. Eine Liste ist allgemein eine Datenstruktur mit Knoten und Verweisen zwischen den Knoten; die Knoten enthalten Daten bzw. *Datenelemente.* In einer doppelt verketteten Liste enthält jeder Knoten ein Element, einen Verweis auf den vorherigen Knoten und einen Verweis auf den nächsten Knoten. In dieser Aufgabe zeigt jedes Kamerabild (Knoten) das Kind (Element) und Verweise auf die Kamerabilder und damit auch die Elemente rechts und links vom Kind.

Um die richtige Antwort zu finden, durchläufst du die Kinder-Liste und folgst den Verweisen, um die Kinder in die richtige Reihenfolge zu bringen. Im Gegensatz zu der zirkulär verketteten Liste in der Biberaufgabe „Olivers Rassel“ kannst du in beide Richtungen laufen. Es gibt jedoch einen Anfang *(Kopf)* und ein Ende der Liste. In unserer Liste ist James am Kopf und Mia am Ende der Liste. Ihre Verweise nach links bzw. nach rechts sind leer.



Ein Beispiel für eine doppelt verkettete Liste ist dein Browser-Verlauf: Du kannst beliebig rückwärts oder dann wieder vorwärts springen. Die aktuell betrachtete Seite (von der aus du im Verlauf zurückgehst), ist am Kopf der Liste.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Berukone

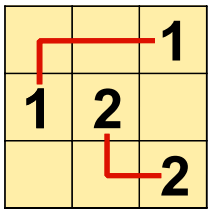
Lasst uns Berukone spielen!

in Berukone-Rätsel besteht aus einem Raster mit Zahlen auf den Feldern.
Zwei gleiche Zahlen bilden ein Paar.

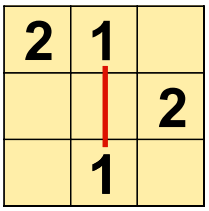
Um das Rätsel zu lösen, musst du für alle Paare die beiden Zahlen durch eine Linie verbinden. Diese Verbindungslinie muss waagrecht oder senkrecht von Feld zu Feld gehen, darf sich aber auf einem Feld um 90 Grad drehen. Eine Linie darf nicht durch eine andere Zahl und nicht durch eine andere Linie gehen.

Leider gibt es Berukone-Rätsel, die nicht gelöst werden können:

Dieses Berukone kann gelöst werden.



Dieses Berukone kann nicht gelöst werden.



Genau eines dieser Berukones kann nicht gelöst werden. Welches?

A)

			3
3			
	2	1	2
1			

B)

3			3
		1	
	2		2
1			

C)

3			
		1	3
	2		
1			2

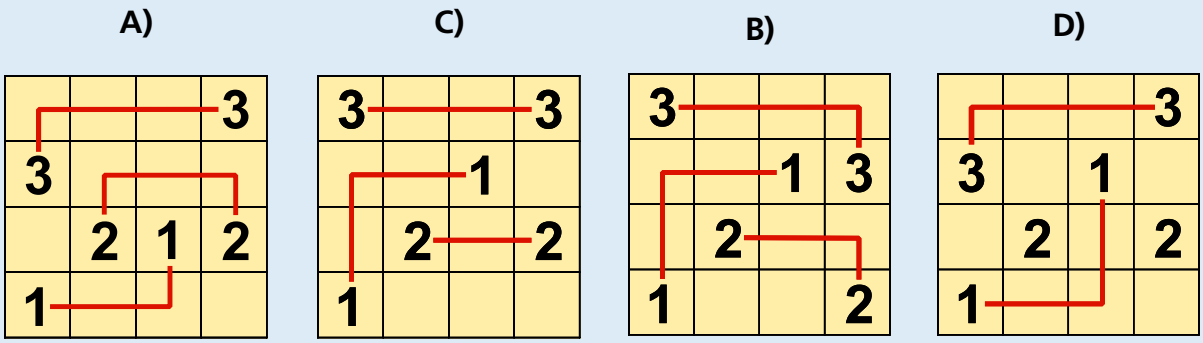
D)

			3
3		1	
	2		2
1			

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Antwort D ist richtig:

Berukone D ist das einzige der vier Berukone-Rätsel, das nicht gelöst werden kann. So kannst du die Rätsel A, B und C lösen und versuchen Rätsel D zu lösen:



Im Rätsel D gibt es keine Möglichkeit, für alle Paare die beiden Zahlen nach den Berukone-Regeln zu verbinden. Insbesondere kann es nur für eines der beiden Paare aus 1en und 2en eine regelgemäße Verbindungslinie geben: Die einzige Linie, mit der man die 1en verbinden kann, ohne dass die Linie durch eine andere Zahl geht, ist oben dargestellt. Alle möglichen Linien zwischen den 2en, die nicht durch mindestens eine andere Zahl gehen, würden die Linie zwischen den 1en kreuzen. Es gibt also keine Möglichkeit mehr, auch noch die 2en mit einer Linie zu verbinden, ohne die Regeln zu verletzen.

Das ist Informatik!
Für die kleinen Berukone-Rätsel in dieser Biberaufgabe ist relativ einfach zu erkennen, ob sie gelöst werden können oder nicht und wie sie gelöst werden können. Aber stell dir größere Rätsel vor, mit viel mehr Zahlen: Dann ist es vielleicht nicht mehr so einfach, eine Lösung zu finden. Es könnte viele Möglichkeiten geben, die Paar-Zahlen zu verbinden, und es würde viel Zeit in Anspruch nehmen, alle auszuprobieren.

Aber du kannst versuchen, das schlauer anzugehen. Du kannst Schritt für Schritt vorgehen, wenn du ein Rätsel löst: Verbinde die Paare nacheinander oder versuche, die Rasterfelder nacheinander zu füllen. Sobald du voraussehen kannst, dass dein aktueller Weg nicht mehr zu einer Lösung führt, war mindestens ein vorheriger Schritt falsch. Dann kannst du den letzten Schritt rückgängig machen und vielleicht sogar noch mehr Schritte, bis du zu einer Stelle kommst, an der ein alternativer Schritt möglich ist, der die bessere Wahl sein könnte. Das heißt, du gehst auf der Spur (engl.: *track*), die du mit deinen bisherigen Schritten gelegt hast, zurück (engl.: *back*), um dann einen anderen Weg zu verfolgen.

Diese Methode heißt in der Informatik *Backtracking.* Sie wird oft verwendet, um Computerprogramme Probleme – ob reale Probleme oder nur Rätsel – automatisch lösen zu lassen. Backtracking-Algorithmen funktionieren wie oben beschrieben: Sie versuchen, Lösungen Schritt für Schritt zu konstruieren und Schritte rückgängig zu machen, wenn sie voraussehen können, dass der aktuelle Weg nicht zu einer Lösung führt. Ihre Fähigkeit, (a) gute Entscheidungen beim nächsten Schritt zu treffen und (b) vorauszusehen, ob der aktuelle Weg zu einer Lösung führen kann oder nicht, ist entscheidend für ihren Erfolg. Backtracking-Algorithmen müssen oft mit schwierigen Problemen umgehen, für die spezifische Lösungsansätze nicht bekannt sind. Im Jahr 2010 fanden japanische Forscher heraus, dass das Lösen von Arukone-Rätseln (eine Variante von Berukone, auch bekannt als Numberlink, bei der in einer Lösung kein Feld frei bleiben darf) zu den schwierigsten bekannten Problemen in der Informatik gehört.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Ricca-Karten 2

Barbara sammelt Karten mit Monstern, den „Riccas“. Hier sind ihre Ricca-Karten:



Auf jeder Karte sind die Eigenschaften eines Riccas angegeben, zum Beispiel der Name ( ) oder ob das Ricca Zähne hat ( ). Die Eigenschaften haben Werte.
Auf Karte 2 zum Beispiel hat den Wert MONI, und hat den Wert ✓: Das Ricca auf der Karte heißt also MONI und hat Zähne.

Barbara erkennt, dass es nur drei verschiedene Arten von Werten auf den Karten gibt. Sie nennt sie „Typen“:

Text	mehrere Buchstaben hintereinander
Zahlen	also 1, 2, 3 usw.
ja/nein	die Zeichen ✓ und X, die „ja“ und „nein“ bedeuten

Ordne den Eigenschaften die richtigen Typen zu.

ja/nein ja/nein Text Zahlen Zahlen

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:

				
Text	Zahlen	Zahlen	ja/nein	ja/nein

Diese Tabelle enthält für alle sechs Karten die Werte der fünf Eigenschaften:

Karte					
1	JOSI	3	2	X	✓
2	MONI	3	2	X	✓
3	KILI	2	2	✓	✓
4	BENI	2	2	X	✓
5	LORI	2	5	X	✓
6	PHIL	2	2	X	✓

Aus der Tabelle können wir nun ablesen:

- Alle Werte der Eigenschaft  sind vom Typ Text.
- Alle Werte der Eigenschaften  und  sind vom Typ Zahlen.
- Alle Werte der Eigenschaften  und  sind vom Typ ja/nein.

Das ist Informatik!
Barbara hat festgestellt, dass auf ihren Ricca-Karten verschiedene Typen von Werten verwendet werden. Diese Typen zu unterscheiden, kann bei der Organisation der Karten helfen. Weil Barbara weiß, dass alle Werte der Eigenschaft  Zahlen sind, kann sie die Anzahlen der Beine der Riccas miteinander vergleichen und die Karten nach der Anzahl der Beine sortieren.

Eine Ricca-Karte ist wie ein Datensatz, der im Computer gespeichert ist. Die Elektronik von Computer-Speichern kann aber nur 1en und 0en speichern. Auch Computer-Programme verwenden deshalb „Wert-Typen“, die in der Informatik *Datentypen* heißen, um festzulegen, wie der Computer diese 1en und 0en interpretieren und manipulieren soll. Außerdem kann mit Hilfe von Datentypen geprüft werden, ob und wie gespeicherte Daten miteinander kombiniert werden können.

Die Wert-Typen aus dieser Biberaufgabe entsprechen wichtigen Datentypen aus der Programmierung:

Text wird in Programmiersprachen häufig als „string“ bezeichnet: Daten dieses Typs sind Folgen von Zeichen (Buchstaben, Ziffern und andere Zeichen).

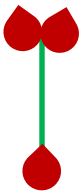
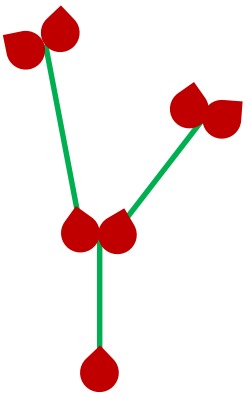
Zahlen entspricht dem Typ „integer“: Daten dieses Typs sind ganze Zahlen, also … -2, -1, 0, 1, 2, … .

ja/nein entspricht dem Typ „boolean“: Dieser Datentyp kennt nur zwei verschiedene Werte, die häufig `true` und `false` (wahr und falsch) heißen.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

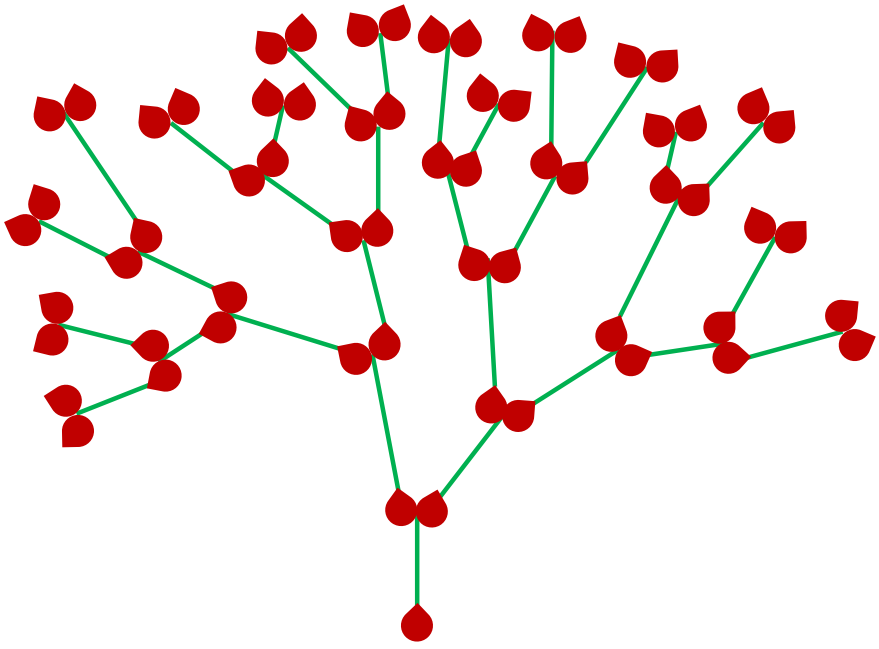
Wunderblume

Bei Sonnenaufgang wächst aus jeder neuen Knospe der Wunderblume ein Stiel. Den ganzen Tag lang wächst der Stiel weiter. Bei Sonnenuntergang hört der Stiel auf zu wachsen, und es kommen zwei neue Knospen heraus. So geht es Tag für Tag weiter, und die Wunderblume wird immer prächtiger.

Neue Knospe vor Sonnenaufgang	Tag 1 nach Sonnenuntergang	Tag 2 nach Sonnenuntergang
		

Wie viele Tage ist diese Wunderblume gewachsen?

A)	B)	C)	D)
5 Tage	11 Tage	16 Tage	32 Tage



Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Antwort A ist richtig:

Die Wunderblume hat Knospen, Stiele – und auch Äste. Ein Ast besteht aus den Stielen, an denen entlang man von der ersten Knospe zu einer äußeren Knospe geht, aus der noch kein Stiel gewachsen ist. Am Ende von Tag 1 gibt es nur einen Ast, und der ist 1 Stiel lang. Am Ende von Tag zwei gibt es schon zwei Äste, und die sind 2 Stiele lang. Durch das Wachstum der Wunderblume werden die Äste an jedem Tag um einen Stiel länger.

Man muss also nur einem Ast folgen, z. B. dem ganz rechts, und zählen, wie viele Stiele der Ast lang ist. Der Ast ist 5 Stiele lang, und so weiß man, dass die Wunderblume 5 Tage gewachsen ist.

Die anderen Antworten sind also falsch. Die Zahlen 11, 16 und 32 passen trotzdem zu dieser in 5 Tagen gewachsenen Wunderblume:
- Jeder Ast hat 11 Knospen.
- Die äußeren Knospen sind aus 16 Stielen herausgewachsen.
- Diese Wunderblume hat 32 äußere Knospen.

Das ist Informatik!
Nach nur fünf Tagen ist die Wunderblume schon sehr groß geworden. Sie hat viele Stiele und noch mehr schöne rote Knospen bekommen. Alleine von den äußeren Knospen gibt es nach 5 Tagen schon 32 Stück. Andersherum kann man feststellen, dass jede der 32 äußeren Knospen an einem Ast gewachsen ist, der nur 5 Stiele lang ist.

Diese Tabelle zeigt für die ersten 10 Tage, wie viele äußeren Knospen eine Wunderblume nach wie vielen Tagen hat und wie lang die Äste dann sind:

Tage	1	2	3	4	5	6	7	8	9	10
Länge der Äste	1	2	3	4	5	6	7	8	9	10
äußere Knospen	2	4	8	16	32	64	128	256	512	1024

Schon nach 10 Tagen hat die Wunderblume sehr viele Knospen. Die Äste sind aber noch recht kurz: Um nach 10 Tagen von der ersten Knospe aus eine der über tausend äußeren Knospen zu erreichen, muss man an nur 10 Stielen entlang gehen.

Damit Computerprogramme auch mit vielen Daten schnell umgehen können, wurden in der Informatik Datenstrukturen entwickelt, die der Wunderblume in dieser Biberaufgabe ähneln. In diesen Strukturen kann man sehr viele Daten speichern, aber dennoch jedes einzelne Datenelement vom Ausgangspunkt der Struktur aus in wenigen Schritten erreichen. Wenn eine solche Struktur genauso schön ausgewogen ist wie die Wunderblume, kann man in 5 Schritten $2 \times 2 \times 2 \times 2 \times 2 = 32$ Datenelemente erreichen, in 10 Schritten $2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 1.024$ und in nur 20 Schritten mehr als eine Million. Die Anzahl der benötigten Schritte wächst also sehr viel langsamer als die Menge der damit erreichbaren Datenelemente. In der Informatik sagt man, dass die Anzahl der Schritte „logarithmisch“ wächst, im Verhältnis zur Anzahl der Datenelemente. Umgekehrt wächst die Anzahl der Datenelemente „exponentiell“ im Verhältnis zur Anzahl der Schritte.

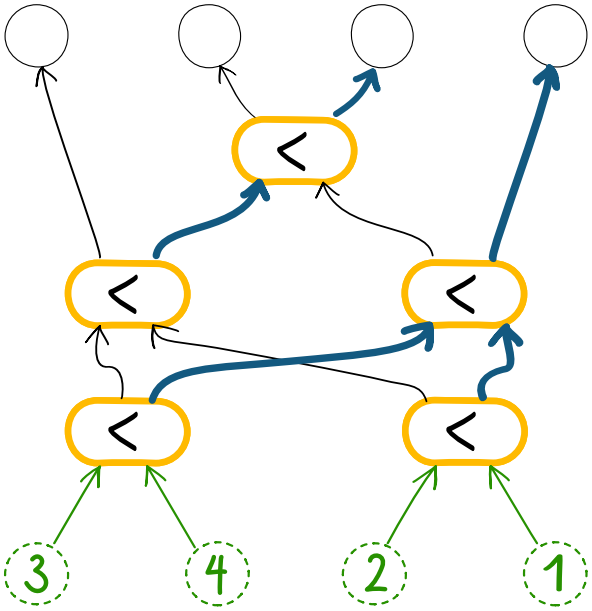
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Zahlenmaschine

Die Biber haben eine Zahlenmaschine.

Vier Zahlen werden unten in die Eingabefelder eingegeben, zum Beispiel 3, 4, 2 und 1.

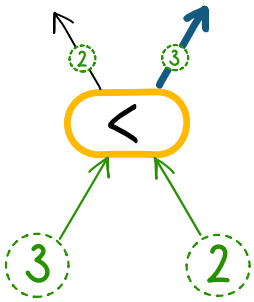
Entlang von Pfeilen und Schaltern < wandern die Zahlen durch die Maschine nach oben bis zu den Ausgabefeldern.



Jeder der fünf Schalter vergleicht die beiden eingehenden Zahlen und leitet …

- … die kleinere Zahl nach links und
- … die größere Zahl nach rechts weiter.

Etwa so:



Welche Aufgabe führt die Maschine aus?

A) Sie gibt die Zahlen in unveränderter Reihenfolge aus. Im Beispiel: 3, 4, 2, 1

B) Sie sortiert die Zahlen in absteigender Reihenfolge. Im Beispiel: 4, 3, 2, 1

C) Sie gibt die Zahlen in umgekehrter Reihenfolge aus. Im Beispiel: 1, 2, 4, 3

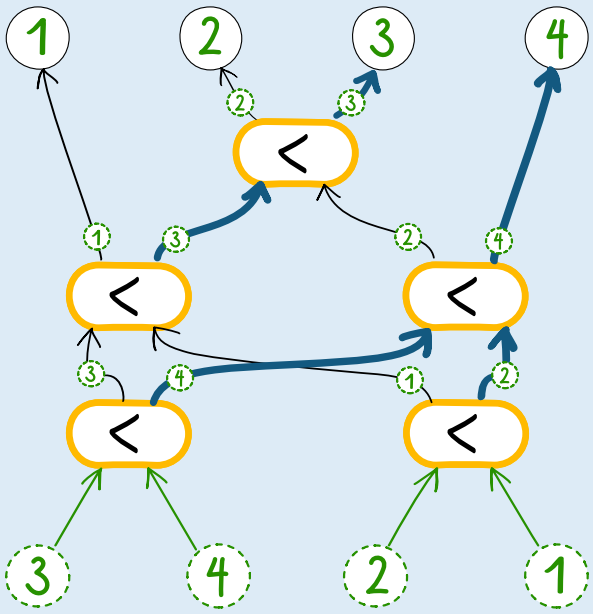
D) Sie sortiert die Zahlen in aufsteigender Reihenfolge. Im Beispiel: 1, 2, 3, 4

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Antwort D ist richtig:
Die Zahlenmaschine sortiert die Zahlen in aufsteigender Reihenfolge. Im Beispiel: 1, 2, 3, 4

Durch Ausprobieren der Maschine kann man sowohl die richtige Antwort feststellen als auch alle anderen Antworten ausschließen.

Die Zahlenmaschine macht in einem ersten Schritt zwei Vergleiche von je zwei Zahlen. Danach vergleicht sie zum einen die beiden grösseren und zum anderen die beiden kleineren Zahlen aus den ersten beiden Vergleichen miteinander, um den ingesamt größten Wert (Maximum) bzw. insgesamt kleinsten Wert (Minimum) der vier Zahlen zu ermitteln. Durch einen Vergleich der übrigen beiden Zahlen ist die Sortierung dann vollständig.



Das ist Informatik!
In der Informatik ist die Zahlenmaschine aus dieser Biberaufgabe als *Sortiernetzwerk* bekannt. Ein Sortiernetzwerk besteht aus einer Reihe identischer, sehr einfacher Komponenten, den Komparatoren. Jeder *Komparator* empfängt zwei numerische Werte auf zwei Eingangsleitungen und vergleicht sie. Dann leitet er die Werte auf zwei Ausgangsleitungen weiter: den kleineren Wert auf der linken Leitung und den größeren Wert auf der rechten Leitung weiter. (Häufig werden Sortiernetzwerke auch quer dargestellt, mit den Eingängen links und Ausgängen rechts und den Komparatoren als Brücken zwischen zwei Leitungen; dann wird die kleinere Zahl in der Regel nach oben und die größere Zahl nach unten geleitet.)

Durch die Kombination einer ausreichenden Anzahl von Komparatoren kann jede Zahlenfolge sortiert werden. Die Zahlenmascheine in dieser Biberaufgabe zeigt, dass vier Zahlen mit fünf Komparatoren sortiert werden können. Für fünf Zahlen sind mindestens neun und für sechs Zahlen mindestens zwölf Komparatoren erforderlich.

Sortiernetzwerke sind in der Informatik von praktischer Bedeutung, weil Komparatoren als sehr einfache und daher kostengünstige elektronische Bauteile und damit Sortiernetzwerke auch insgesamt gut in Hardware realisiert werden können. Außerdem können die Komparatoren zum Teil gleichzeitig arbeiten, was die Sortierung beschleunigt: Im Allgemeinen ist die Anzahl der nicht-parallelen Schritte eines Sortiernetzwerks kleiner als die Anzahl der zu sortierenden Zahlen. Ein Nachteil von Sortiernetzwerken ist, dass sie jeweils nur für Eingaben von fester Länge ausgelegt sind.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Verrückte Lampe

Viktoria Volt hat eine verrückte Lampe gebaut. Die Lampe hat zwei Lichtschalter: einer links und einer rechts. Jeder Lichtschalter kann entweder **an** (,) oder **aus** (,) sein.



Die verrückte Lampe hat aber noch einen dritten, geheimen Schalter: ein Bild!

Je nachdem, wie das Bild hängt ( ,  oder ), funktionieren die Lichtschalter anders.

Diese Tabelle sagt, wie die Schalter jeweils das Licht an oder aus machen:

Bild	Lichtschalter	Licht
	genau einer ist **an:** oder	**an**
	sonst	**aus**
	beide sind **aus:**	**aus**
	sonst	**an**
	beide sind **an**	**an**
	sonst	**aus**

Der linke Lichtschalter ist aus , der rechte Lichtschalter ist an .
Wie muss das Bild hängen, damit das Licht aus ist?

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:

Wenn das Bild so hängt  , ist das Licht aus.

Wir schauen alle drei Möglichkeiten, wie das Bild hängen kann, genauer an:

 Weil genau ein Schalter **an** ist (nämlich der rechte), ist das Licht **an.** Diese Antwort ist falsch.

 Weil nicht beide Lichtschalter **aus** sind, ist das Licht **an.** Auch diese Antwort ist falsch.

 Weil nicht beide Lichtschalter **an** sind, ist das Licht **aus.** Diese Antwort ist richtig.

Das ist Informatik!
Veronikas Lichtschalter können jeweils zwei Werte haben: **an** oder **aus.** Und auch das Licht hat nur diese beiden Werte. In jeder Position des Bildes lässt sich der Wert des Lichts aus den Werten der Schalter ausrechnen, und zwar so, wie in der Tabelle angegeben.

Auch die kleinste Speichereinheit in Computern, nämlich das *Bit,* kann nur zwei Werte annehmen. In der Informatik heißen diese Werte häufig auch **an** und **aus,** manchmal aber auch **wahr** und **falsch** oder auch **1** und **0.** So wie man Zahlen mit Operatoren (+, –, ×, :) kombinieren und daraus neue Werte berechnen kann, kann man Bits mit *logischen Operatoren* kombinieren. In Computern sind diese Operatoren als *logische Gatter* eingebaut. Mit den Gattern kann man Bits auf nur alle erdenklichen Arten kombinieren und verrechnen. Deshalb können Computer Daten auf fast beliebige Weise verarbeiten und verändern.

Das Bild in dieser Biberaufgabe entscheidet, welche logische Operation die beiden Lichtschalter umsetzen. Hängt das Bild gerade, funktionieren die Schalter wie ein XOR („entweder oder“): Wenn entweder der linke oder der rechte Schalter an ist, ist das Licht an. Hängt das Bild nach rechts gekippt, funktionieren sie wie ein OR („oder“): Wenn der linke Schalter oder der rechte (also beide oder einer von beiden) an ist, ist das Licht an. Hängt das Bild nach links gekippt, funktionieren die Schalter wie ein AND („und“): Wenn der linke und der rechte Schalter (beide gleichzeitig) an ist, ist das Licht an.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Holz für den Damm

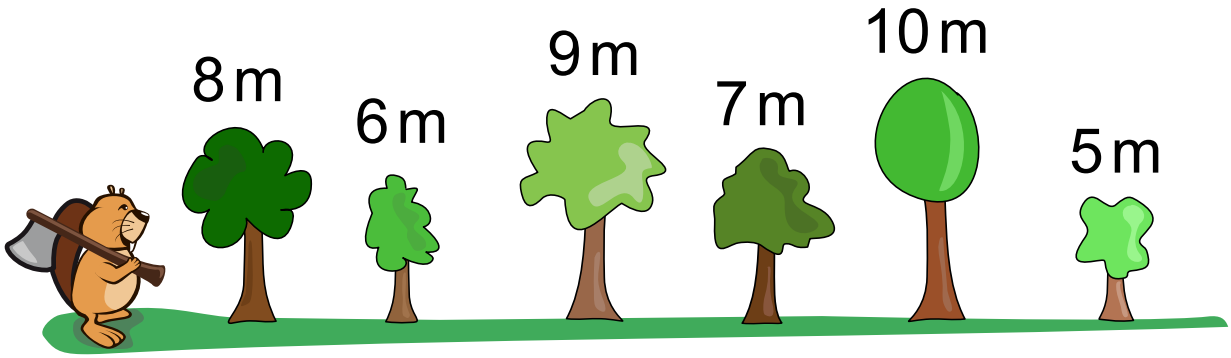
Für ihren nächsten Dammbau müssen die Biber einige Bäume fällen.

Sechs Bäume kommen in Frage. Die Biber wissen, wieviele Meter Holz jeder Baum hat. Sie wollen insgesamt möglichst viele Meter Holz haben. Den ersten Baum können sie frei wählen. Immer wenn sie danach einen nächsten Baum fällen wollen, müssen sie zwei Regeln befolgen:

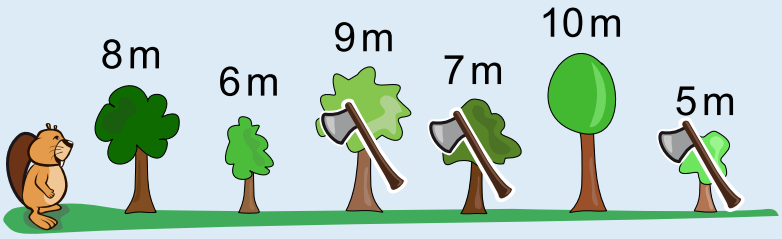
- Regel 1: Der nächste Baum muss weiter rechts stehen als der vorherige.
- Regel 2: Der nächste Baum muss kleiner sein, also weniger Meter Holz haben als der vorherige.

Ein Beispiel: Wenn sie den 6m-Baum fällen, dürfen sie danach nur noch den 5m-Baum fällen. Dann haben sie am Ende insgesamt 11 Meter Holz.

Welche Bäume können die Biber nach ihren Regeln fällen, damit sie am Ende möglichst viele Meter Holz haben?



So ist es richtig:



Die Aufgabe besteht darin, von links nach rechts gehend eine nach Holzmetern absteigende Teilfolge der sechs Bäume zu finden, sodass die Summe der Holzmeter in dieser Teilfolge maximal ist. Da wir die maximale Gesamthöhe suchen, müssen wir nur Teilfolgen berücksichtigen, die nicht mehr erweitert werden können, weil das Hinzunehmen egal welchen Baumes zur Teilfolge die Regeln verletzen würde. Wir nennen solche Teilfolgen vollständig.

Wenn wir zum Beispiel mit dem 8m-Baum (kurz: 8) beginnen, gibt es zwei vollständige Teilfolgen: 8, 6, 5 und 8, 7, 5. Die Teilfolgen 8, 7 (kann um 5 erweitert werden) und 8, 5 (kann um 6 oder 7 erweitert werden) sind nicht vollständig.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Die Tabelle zeigt alle vollständigen Teilfolgen. Außerdem gibt sie für jede Teilfolge an, wieviele Meter Holz die Biber am Ende haben, wenn sie die Bäume dieser Teilfolge fällen.

vollständige Teilfolge	Meter Holz
8, 6, 5	19
8, 7, 5	20
6, 5	11
9, 7, 5	21
10, 5	15

Die Biber können also die Bäume 9, 7 und 5 nach ihren Regeln fällen, damit sie am Ende möglichst viel Holz haben, nämlich 21 Meter.

Das ist Informatik!
Die Biber versuchen, im Rahmen ihrer Regeln eine Menge von Bäumen zu finden, so dass sie die größtmögliche Anzahl an Metern Holz bekommen. In dieser Biberaufgabe sind 21 Meter Holz das *optimale,* das heißt beste Ergebnis, das sie erzielen können. Bei der Beantwortung dieser Biberaufgabe hast du also ein *Optimierungsproblem* gelöst.

Allgemein geht es bei einem Optimierungsproblem darum, einen bestmöglichen Wert zu finden. Das kann ein größter Wert sein, wie in dieser Biberaufgabe, oder auch ein kleinster Wert, etwa wenn du den schnellsten Weg suchst, den du zur Schule nehmen kannst. Es liegt in der Natur der meisten Menschen, für ein Vorhaben nach der schnellsten, kürzesten, günstigsten oder unterhaltsamsten Variante zu suchen: Das ist Optimierung. Es ist aber auch Optimierung, wenn ein Unternehmen versucht, möglichst wenig Gehalt an seine Mitarbeiterinnen und Mitarbeiter auszuzahlen, oder bei der Vermietung von Wohnungen versucht wird, möglichst hohe Mieteinnahmen zu erzielen.

Viele Optimierungsprobleme sind so komplex, dass sie mit Hilfe von Informatiksystemen, also Computerprogrammen bzw. „Apps“ gelöst werden. Wenn ein Paketauslieferer nach Möglichkeiten sucht, die Routen der Auslieferungsfahrzeuge so zu planen, dass der Energieverbrauch möglichst gering ist, oder ein Energieversorger seine Anlagen zur Produktion erneuerbarer Energie möglichst wirtschaftlich einsetzen will, müssen so viele Daten und Bedingungen berücksichtigt werden bzw. die Steuerung so flexibel sein, dass mit Hilfe von Computern oft sehr viel bessere Ergebnisse erzielt werden als ohne. Da ist es gut, dass die Informatik viele Methoden zur Lösung von Optimierungsproblemen kennt und sagen kann, welche Methoden für welche Arten von Problemen gut eingesetzt werden können.

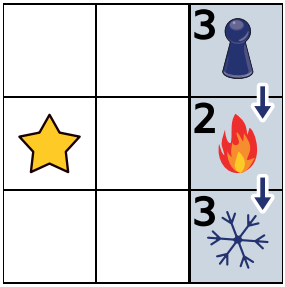
Für *besonders Interessierte:* Das Problem in dieser Aufgabe kennt die Informatik auch als „Maximum Sum Decreasing Subsequence“. Schau dir einmal diese Webseite an: https://www.geeksforgeeks.org/dsa/maximum-sum-increasing-subsequence-dp-14/. Dort kannst du nachverfolgen, wie man von einfachen, aber schlechten Ansätzen zur Lösung eines Optimierungsproblems nach und nach immer bessere Ansätze finden und diese in Programme umsetzen kann.

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Schatzsuche

Nina und Daniel spielen Schatzsuche.
Auf einem Spielbrett mit quadratischen Feldern wählt Nina im Kopf ein Feld aus.
Dort ist der Schatz versteckt.

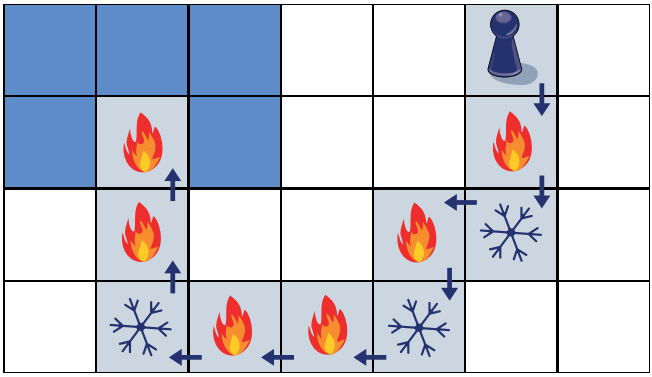
Daniel wählt ein Startfeld aus. Von dort geht er schrittweise mit seiner Spielfigur um je ein Feld weiter: nach links, rechts, oben oder unten.



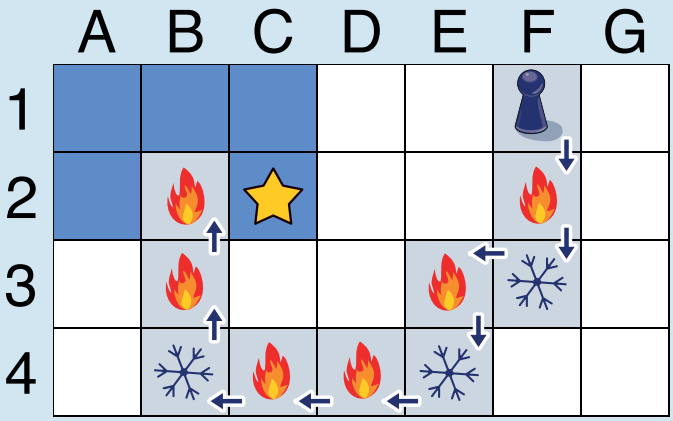
Beim ersten Versuch nehmen sie ein kleines Spielbrett.
Nina versteckt den Schatz auf dem Feld mit dem Stern.
Daniel startet rechts oben und macht zwei Schritte entlang der Pfeile. Nach jedem Schritt sagt Nina, ob Daniel nun näher (❄) am Schatz oder weiter weg (🔥) vom Schatz ist als vor dem Schritt – siehe Bild. Die Zahlen zeigen Daniels Entfernungen vom Schatz: jeweils die kleinste Anzahl Schritte, mit denen Daniel aktuell zum Schatz gehen könnte.

Nun nehmen sie ein größeres Spielbrett.
Nina versteckt den Schatz auf einem der blau markierten Felder.
Das Bild zeigt Daniels Schritte und was Nina nach jedem Schritt sagt.

Wo ist der Schatz versteckt?



So ist es richtig:



Wir verfolgen Davids Weg und Ninas Rückmeldungen. David startet in Zeile 1 des Spielbretts. Nach dem ersten Schritt ist er in Zeile 2 und näher am Schatz als in Zeile 1. Nach dem nächsten Schritt ist er in Zeile 3 und wieder weiter weg vom Schatz. Da er in der gleichen Spalte geblieben ist, muss der Schatz auf einem Feld in Zeile 2 sein. Denn egal, in welcher Spalte der Schatz versteckt ist: Den kürzesten Weg zum Schatz von einer anderen Spalte aus hat man, wenn man in der gleichen Zeile ist.

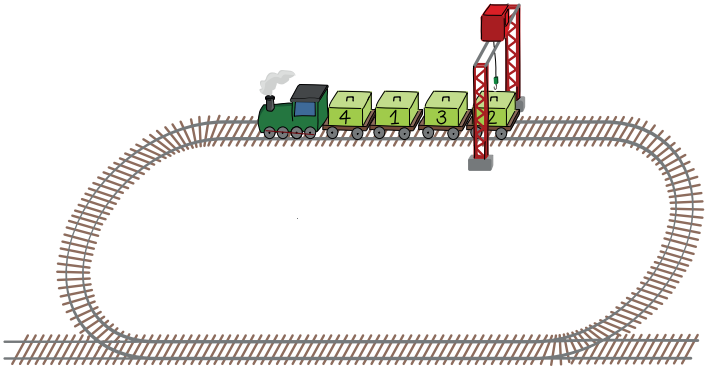
Aber in welcher Spalte ist der Schatz versteckt? Auf seinem weiteren Weg kommt Daniel in Zeile 4 mit einigen Schritten nach links dem Schatz zunächst näher; insbesondere ist er in Spalte C näher am Schatz als in Spalte D. Aber nach dem letzten Schritt in der Zeile ist Daniel in Spalte B weiter weg vom Schatz als in Spalte C. Der Schatz muss also auf einem Feld in Spalte C sein. Denn was wir oben für Spalten gesagt haben, gilt auch für Zeilen: Den kürzesten Weg zum Schatz von einer anderen Zeile aus hat man, wenn man in der gleichen Spalte ist.

Das ist Informatik!
Daniel geht (mit seiner Spielfigur) über das Spielbrett. Von jedem Feld, auf dem er gerade steht, misst Nina die Entfernung zum Feld mit dem Schatz und nutzt dies für ihre Rückmeldung. Üblicherweise wird die Entfernung zwischen zwei Punkten als Länge der geradlinigen Verbindung zwischen den Punkten gemessen *(euklidische Distanz).* Doch die zwei Felder sind genau genommen keine Punkte. Deshalb bemisst Nina die Entfernung zwischen zwei Feldern in der Anzahl von Schritten, die Daniel für einen kürzesten Weg vom einen Feld zum anderen gehen müsste. Dieses *Maß* kann man generell bei Rastern anwenden und ist in der Informatik als *Manhattan-Distanz* bekannt – abgeleitet vom gerasterten Straßenplan des New Yorker Stadtteils Manhattan.

Informatikerinnen und Informatiker wählen die Art der Distanzberechnung zwischen zwei Objekten in Abhängigkeit von der Frage, die sie lösen möchten. Wenn man zum Beispiel die Entfernung zwischen zwei gleich langen Wörtern in einer natürlichen Sprache messen möchte, kann man die Anzahl Stellen, an welchen sich die Wörter unterscheiden, zählen; das ist dann der *Hamming-Abstand* oder die *Hamming-Distanz.* Sind die Wörter unterschiedlich lang, kann man die *Editierdistanz* verwenden. Entfernungen bzw. Distanzen spielen in der Informatik häufig eine Rolle, wenn es darum geht, optimale Lösungen eines Problems zu finden. Egal ob die Lösung eines Problems am schnellsten, am kürzesten oder am günstigsten sein soll: Man muss oft nicht den Algorithmus ändern, sondern nur das Entfernungsmaß: Dauer, Länge oder Kosten.

Zug entladen

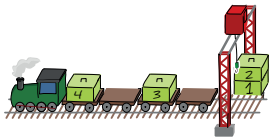
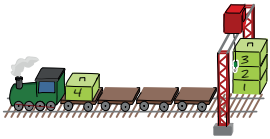
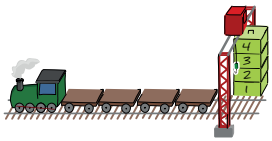
Ein Zug hat Wagen mit nummerierten Kisten. Zum Entladen der Kisten fährt der Zug eine Runde zum Kran. Der Kran ist unbeweglich, und der Zug kann nur vorwärts fahren.



Die Kisten sollen in der Reihenfolge ihrer Nummern entladen werden, Kiste 1 zuerst.

Der Zug fährt Wagen für Wagen die Kisten unter den Kran. Nur wenn eine Kiste an der Reihe ist, entlädt der Kran die Kiste. Wenn der Zug unter dem Kran durch ist, können noch Kisten auf den Wagen sein. Dann muss der Zug noch eine Runde zum Kran fahren.

Der obige Zug muss drei Runden fahren, bis alle Kisten in der richtigen Reihenfolge entladen sind:

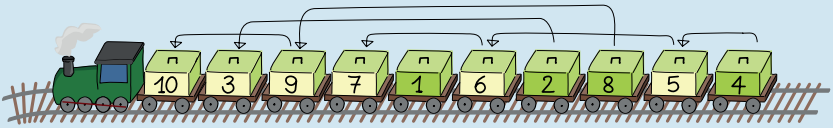
Runde 1	Runde 2	Runde 3
		
Kiste 4 ist nicht an der Reihe, Kiste 1 wird entladen, Kiste 3 ist nicht an der Reihe, Kiste 2 wird entladen.	Kiste 4 ist nicht an der Reihe, Kiste 3 wird entladen.	Kiste 4 wird entladen.

Wie viele Runden muss dieser Zug fahren, bis alle Kisten in der richtigen Reihenfolge entladen sind?

7 ist die richtige Antwort:

Die richtige Reihenfolge für das Entladen ist 1, 2, 3, 4, 5, 6, 7, 8, 9, 10; die Reihenfolge der Kisten auf dem Zug ist aber 10, 3, 9, 7, 1, 6, 2, 8, 5, 4. Nach der ersten Runde kann der Kran die Kisten 1 und 2 entladen. Nach der zweiten Runde kann der Kran die Kisten 3 und 4 entladen, nach weiteren Runden dann die Kisten 5, 6, 7 und 8, 9 und schließlich 10. Insgesamt fährt der Zug 7 Runden.

Zur Beantwortung kann man sich auch Folgendes überlegen: Der Zug muss immer dann eine weitere Runde fahren, wenn zwei in der richtigen Reihenfolge direkt hintereinander stehende Kisten auf dem Zug falsch herum stehen, also die Kiste mit der größeren Nummer vor der Kiste mit der kleineren Nummer kommt.



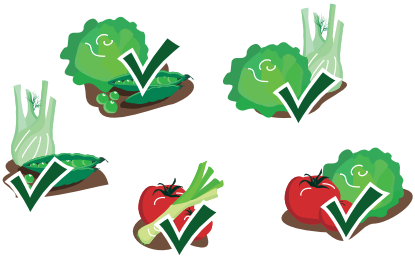
Da z. B. Kiste 3 vor Kiste 2 kommt, ist Kiste 3 nach der ersten Runde nicht an der Reihe und kann nicht entladen werden. Der Zug muss eine weitere Runde fahren, damit auch Kiste 3 entladen werden kann. Für das Nummernpaar (2,3) stehen also die Kisten „falsch" auf dem Zug. Beim gegebenen Zug gibt es sechs solcher Paare: (2,3), (4,5), (5,6), (6,7), (8,9) und (9, 10). Deshalb muss der Zug 6 zusätzliche Runden fahren, also insgesamt 7 Runden.

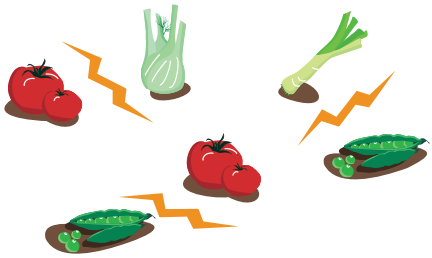
Das ist Informatik!
Wenn für eine beliebige Zahl in der Folge 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 die Kiste mit der nächstgrößeren Zahl weiter vorne auf dem Zug liegt, sprechen Informatikerinnen und Informatiker von einer *Inversion,* das bedeutet Umkehrung. Für jede solche Umkehrung ist eine zusätzliche Runde erforderlich. Wenn wir die Anzahl der Umkehrungen zählen, erhalten wir die richtige Antwort zu dieser Biberaufgabe.

Das Zählen von Inversionen in Bezug auf eine gewünschte Folge hat viele Anwendungen. Bei einigen *Sortieralgorithmen,* wie z. B. *Bubble-Sort,* gibt die Anzahl der Inversionen Aufschluss darüber, wie viele Vertauschungen erforderlich sind, um eine bestimmte Folge zu sortieren. Wenn zwei Kunden dieselbe Menge von Artikeln in eine Rangfolge bringen, sagt uns die Anzahl der Inversionen in ihren Ranglisten, wie sehr sich ihre Vorlieben gleichen. Dies wird von Online-Shops genutzt, um „ähnliche" Kunden zu identifizieren und ihnen Produktempfehlungen zu geben.

Gemüsebeet

Lisa legt ein Gemüsebeet an, mit sechseckigen Bereichen.
Darauf will sie Gemüse pflanzen, in jeden Bereich eines.
Es gibt fünf Sorten Gemüse.
Manche Gemüse vertragen sich gut miteinander (✓), andere nicht ( ):

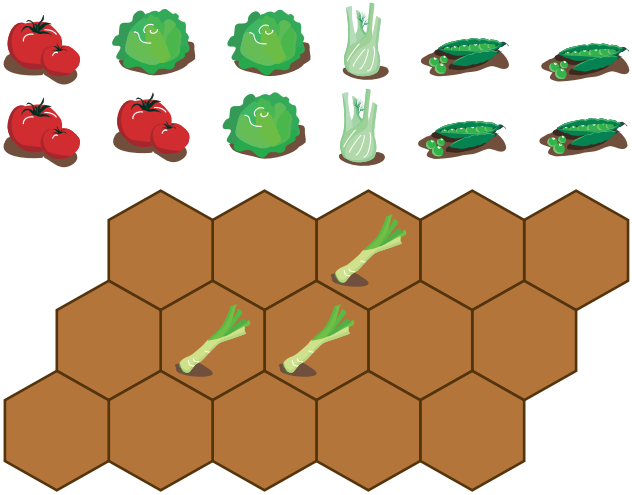




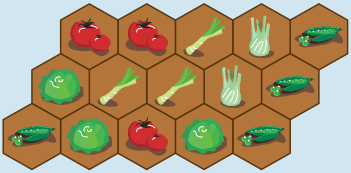
Beim Pflanzen beachtet Lisa folgende Regel: Gemüse, die sich nicht vertragen, dürfen nicht in Bereiche gepflanzt werden, die sich berühren.

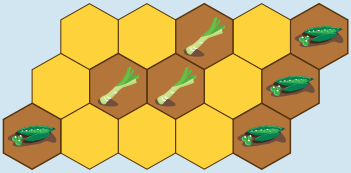
In drei Bereiche hat Lisa schon Lauch gepflanzt.

Bepflanze alle noch freien Bereiche und beachte Lisas Regel!



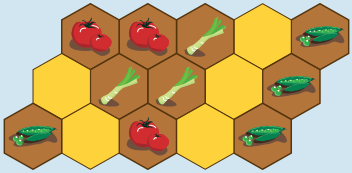
So ist es richtig:



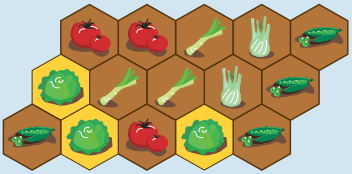


Weil Erbsen sich mit Lauch nicht vertragen, pflanzt Lisa keine Erbsen in die gelben Bereiche. Für die Erbsen bleiben nur die übrigen Bereiche.

Weil Tomaten sich mit Erbsen nicht vertragen, pflanzt Lisa keine Tomaten in die gelben Bereiche. In die übrigen Bereiche kann sie Tomaten pflanzen; Tomaten vertragen sich mit Lauch.

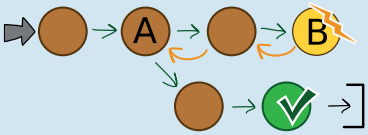


Weil Tomaten sich mit Fenchel nicht vertragen, pflanzt Lisa keinen Fenchel in die gelben Bereiche. Den Fenchel kann sie in die beiden Bereiche zwischen Lauch und Erbsen pflanzen. In die gelben Bereiche kann sie Salat pflanzen: Für Salat ist Lisa keine Unverträglichkeit bekannt.



Das ist Informatik!

Wer Gemüse so pflanzen will, dass die Ernte möglichst groß wird, muss viele Bedingungen beachten: Die einzelnen Sorten haben zum Beispiel unterschiedlichen Bedarf an Platz, Nährstoff und Licht. In dieser Biberaufgabe betrachten wir nur eine Art von Bedingung: die Verträglichkeit zwischen den Gemüsesorten. Um eine Bepflanzung von Lisas Beet zu finden, die alle Verträglichkeitsbedingungen beachtet, könnte man so vorgehen: Man probiert systematisch alle Kombinationen aus, die Gemüse auf dem Beet zu platzieren. Erst wenn das Beet voll ist, wird geprüft, ob diese Kombination alle Bedingungen erfüllt und eine Lösung für Lisas Problem ist. In der Informatik ist solch ein Ausprobieren aller Kombinationen als *Brute-Force*-Methode bekannt. Bei Problemen mit vielen Kombinationen und nur wenigen Lösungen kann ein Vorgehen nach dieser Methode sehr lange dauern. Deshalb ist es meist besser, schrittweise vorzugehen und bei jedem Schritt alle Bedingungen zu berücksichtigen. Auf diese Weise haben wir die Lösung für Lisas Problem gefunden, und eine „falsche" Kombination bzw. Bepflanzung des Beets konnte gar nicht entstehen.

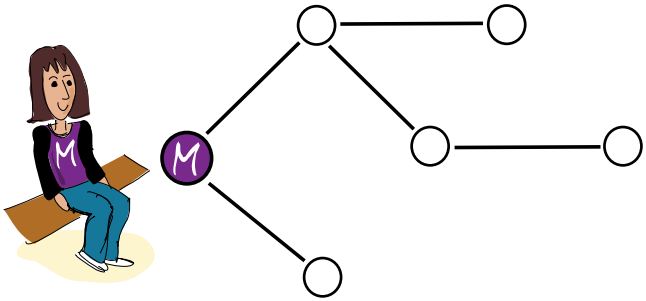


Zum Glück ließ sich die Lösung auf direktem Weg finden: Es gab immer Bereiche, in die wir einige der noch übrigen Gemüse pflanzen konnten. Das gelingt im Allgemeinen nicht immer. Wenn man versucht, die Lösung schrittweise zusammenzusetzen, kann es bei einem Schritt A mehrere Möglichkeiten geben, alle Bedingungen zu erfüllen. Je nach Wahl kann es bei einem späteren Schritt B keine Möglichkeit mehr geben. Dann nimmt man die letzten Schritte solange zurück, bis man beim Schritt A mit den mehreren Möglichkeiten wieder angekommen ist. Dort wählt man eine andere Möglichkeit und versucht damit eine Lösung zu finden. In der Informatik ist diese Rücknahme von Schritten als *Backtracking* bekannt.

Martinas Dorf

In Martinas Dorf gibt es sechs Häuser. Außerdem gibt es Wege, über die man von einem Haus zum nächsten gehen kann. Für alle diese Wege benötigt Martina die gleiche Zeit.

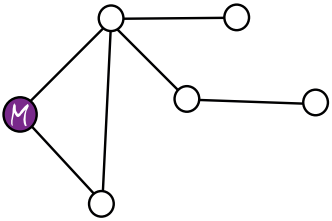
Martina hat eine besondere Karte des Dorfs gezeichnet. Sie hat darin Wege eingezeichnet, über die sie am schnellsten zu den anderen Häusern gehen kann.



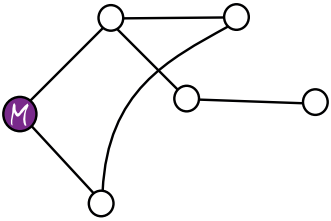
Natürlich gibt es auch eine richtige Karte des Dorfs, mit allen Wegen.

Welche dieser Zeichnungen kann nicht die richtige Karte sein?

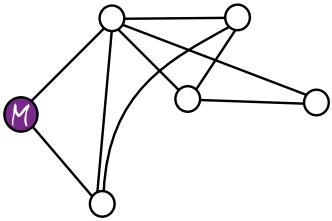
A



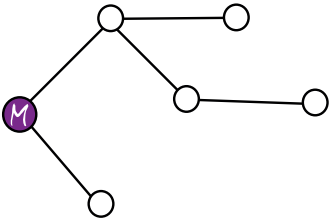
B



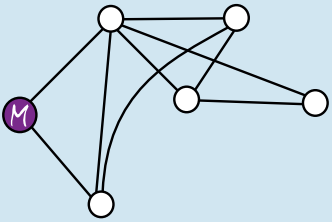
C



D



Antwort C ist richtig:



Martinas besondere Karte zeigt, dass sie zu dem Haus ganz rechts am schnellsten über drei Wege gehen kann. Wenn C die richtige Karte des Dorfes wäre, dann könnte Martina schneller zu diesem Haus gehen, nämlich über zwei Wege. Also kann C nicht die richtige Karte des Dorfes sein.

Bei den Karten A, B und D gibt es keine Möglichkeit, schneller zu einem der anderen Häuser zu gehen als über die Wege auf Martina besonderer Karte. Diese Karten könnten also richtige Karten des Dorfes sein.

Das ist Informatik!
Martina ist Informatikerin. Sie hat ihre Karte als *Graph* gezeichnet. Graphen bestehen aus Knoten (hier die Häuser), die durch Kanten (hier die Wege) verbunden sein können. Sie sind in vielen Bereichen der Informatik geeignet, die Realität zu modellieren – auch hier in dieser Biberaufgabe.

Martina weiß, dass es für Graphen eine ganze Reihe von Algorithmen gibt, beispielsweise die sogenannte Breitensuche, um Aufgaben zu lösen wie „Wie kommt man am schnellsten zu einem anderen Haus?". Vielleicht hat sie ihre besondere Dorfkarte mit Hilfe einer Breitensuche in einem größeren Graph, der richtigen Karte des Dorfes mit allen Wegen, erstellt.

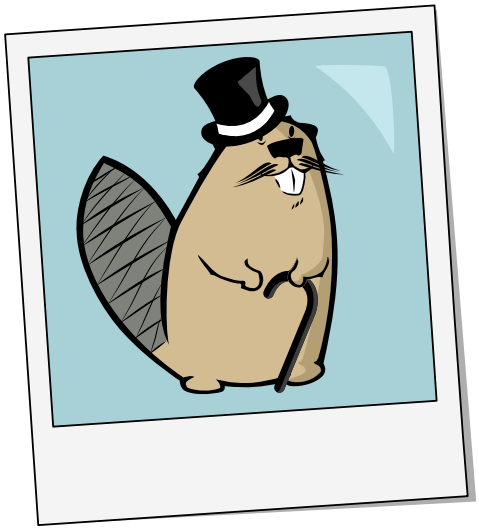
In der Graphentheorie, die sich mit Graphen und Graph-Algorithmen beschäftigt, entspricht Martinas Karte einem Teilgraph der Gesamtkarte des Dorfes. Martinas Teilgraph hat zwei Besonderheiten:

- Alle Knoten sind direkt (über eine Kante) oder indirekt (über mehreren Kanten) miteinander verbunden.
- Egal welche zwei Knoten man zufällig auswählt, es gibt immer nur genau einen Weg zwischen den beiden.

Ein Graph mit diesen Besonderheiten wird in der Informatik als *Baum* bezeichnet. Martinas Haus stellt die *Wurzel* des Baumes dar. Von der Wurzel aus kann Martina alle anderen Knoten (die anderen Häuser im Dorf) auf einem eindeutigen Weg erreichen. Martinas Teilgraph ist also ein Baum; außerdem enthält er alle Knoten des gesamten Graphen (der Gesamtkarte des Dorfes) – aber möglicherweise nicht alle Kanten. Ein Teilgraph mit diesen Eigenschaften wird als *Spannbaum* des gesamten Graphen bezeichnet.

In der Informatik gibt es viele Anwendungen für Graph-Algorithmen, vor allem im Zusammenhang mit Netzwerken (Verkehrsnetze, Telekommunikationsnetze …), etwa bei der Berechnung von Routen in Navigationssystemen. Spannbäume können beim Aufbau kostengünstiger Netze eingesetzt werden und hilfreich bei der Lösung besonders schwieriger Probleme sein.

Faktencheck



Als der berühmte Biber Alfred zu einem großen Fest kam, wurde er fotografiert.

Vier Influencer schrieben über diesen Moment, aber nicht alles in sozialen Medien ist wahr:

Nur eine der vier Aussagen ist wahr. Welche?

A) Alfred hatte einen Hut  auf **und** benutzte keinen Stock  .

B) Alfred trug eine Krawatte  **oder** eine Fliege  .

C) Alfred hatte einen Hut  auf **und** trug eine Fliege  .

D) Alfred hatte Flipflops  an **oder** sogar keine Schuhe  .

Antwort D ist richtig:

Um die richtige Antwort zu bestimmen, muss der Wahrheitsgehalt (wahr oder falsch) der vier Aussagen festgestellt werden, im Vergleich zum Foto. Jede Aussage besteht aus zwei Teilaussagen, die entweder mit **und** oder mit **oder** verknüpft sind. Der Wahrheitsgehalt der gesamten Aussage lässt sich aus dem Wahrheitsgehalt der Teilaussagen bestimmen:

und: Die gesamte Aussage ist *wahr,* wenn beide Teilaussagen *wahr* sind; sonst ist sie *falsch.*
oder: Die gesamte Aussage ist *wahr,* wenn mindestens eine der Teilaussagen *wahr* ist; sonst ist sie *falsch.*

In der deutschen Sprache sind manche der zweiten Teilaussagen verkürzt, damit Subjekt oder Prädikat nicht wiederholt werden. In der folgenden Analyse sind die Teilaussagen vollständig angeführt:

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Aussage A:
Alfred hatte einen Hut  auf *(wahr)* **und** Alfred benutzte keinen Stock  .
⟶ Diese Aussage ist *falsch,* weil nicht beide Teilaussagen *wahr* sind.

Aussage B:
Alfred trug eine Krawatte  **oder** Alfred trug eine Fliege  .
⟶ Diese Aussage ist *falsch,* weil nicht mindestens eine Teilaussage *wahr* ist.

Aussage C:
Alfred hatte einen Hut  auf *(wahr)* **und** Alfred trug eine Fliege  .
⟶ Diese Aussage ist *falsch,* weil nicht beide Teilaussagen *wahr* sind.

Aussage D:
Alfred hatte Flipflops  an *(falsch)* **oder** Alfred hatte sogar keine Schuhe  an.
⟶ Diese Aussage ist *wahr,* weil (mindestens) eine Teilaussage *wahr* ist.

Das ist Informatik!
In den sogenannten „sozialen Medien" ist nicht immer alles wahr. Und leider gibt es nicht immer ein Foto, mit dem wir den Wahrheitsgehalt einer Aussage dort überprüfen können.

Auch die Logik beschäftigt sich mit der Wahrheit von Aussagen: Aussagen können *wahr* oder *falsch* sein. Logische Operatoren können verwendet werden, um (einfachere) Aussagen zu komplexeren Aussagen zu verknüpfen. Der Wahrheitsgehalt der gesamten Aussage lässt sich dann aus dem Wahrheitsgehalt der Teilaussagen berechnen. In dieser Biberaufgabe werden zwei Operatoren verwendet, **und** und **oder.**

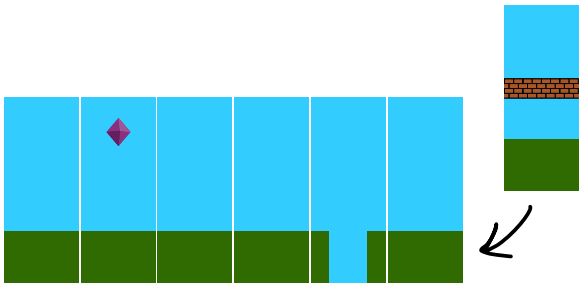
Die *Wahrheitswerte* der gesamten Aussagen können für beide Operatoren in einer *Wahrheitstabelle* dargestellt werden:

Teilaussage A	Teilaussage B	A und B	A oder B
wahr	*wahr*	*wahr*	*wahr*
falsch	*wahr*	*falsch*	*wahr*
wahr	*falsch*	*falsch*	*wahr*
falsch	*falsch*	*falsch*	*falsch*

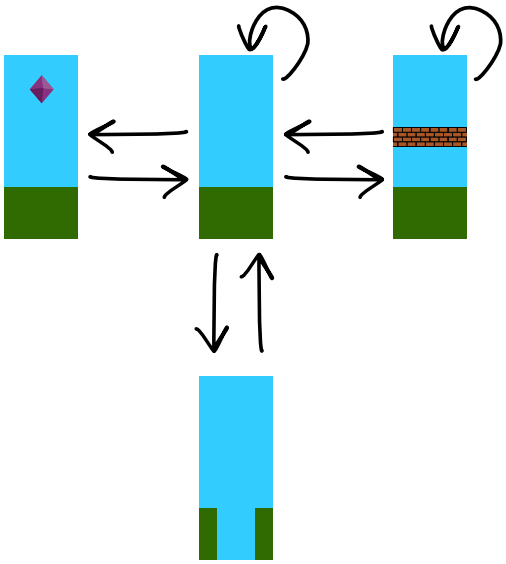
Der Umgang mit Logik ist fundamental in der Informatik, unter anderem beim Programmieren. Da verwendet man logische Ausdrücke, um festzulegen, unter welchen Bedingungen ein Programm was tun soll. In manchen Programmen wird das auch bei der Benutzung offensichtlich, etwa beim Aufsetzen eines Spam-Filters in einem E-Mail-Programm. Verwendet die Bedingung eines Spam-Filters ein **und** (zum Beispiel: die E-Mail stammt von einem unbekannten Absender **und** die E-Mail hat das Wort „Bitcoin" im Betreff), dann landet eine Mail nur dann im Spam-Ordner, wenn beide Teilaussagen wahr sind.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

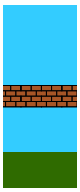
Superbebras



Im Computerspiel „Superbebras“ besteht der Hintergrund aus einer Folge von Kacheln. Der Computer fügt ständig eine neue Kachel auf der rechten Seite der Folge hinzu und entfernt gleichzeitig eine Kachel auf der linken Seite. Das sieht dann wie eine Bewegung aus.



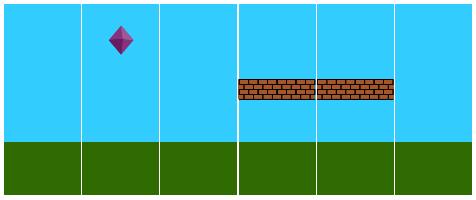
Um die nächste neue Kachel auszuwählen, verwendet der Computer das Diagramm auf der rechten Seite. Im Diagramm sucht er die vorige neue Kachel (auf der rechten Seite) und die Pfeile, die davon ausgehen. Dann wählt er zufällig eine Kachel aus, auf die einer dieser Pfeile zeigt.

Zum Beispiel kann der Computer nach der Kachel  entweder wieder die Kachel  oder die Kachel  auswählen.

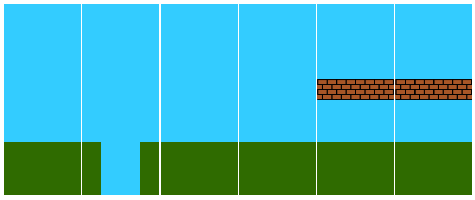
In einem Superbebras-Hintergrund muss also die Reihenfolge der Kacheln zu obigem Diagramm passen.

Eines dieser Bilder ist KEIN Superbebras-Hintergrund. Welches?

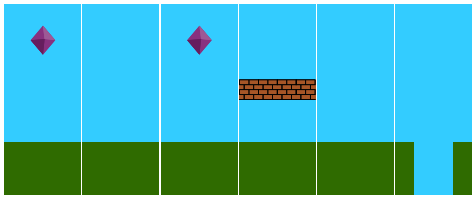
A)



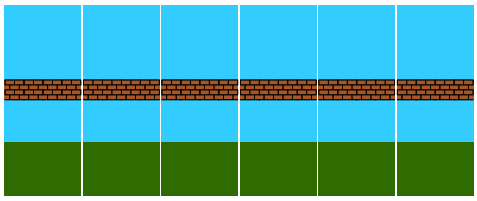
B)



C)

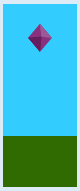


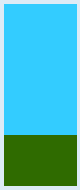
D)

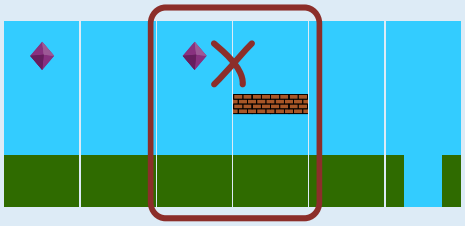


Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Antwort C ist richtig:

Das Bild aus Antwort C passt nicht zum Diagramm. Auf eine Kachel mit einem Diamanten 

kann gemäß Diagramm nur eine  Kachel folgen.

Das ist im Bild aus Antwort C nicht der Fall: 

Alle anderen Bilder passen zum Diagramm.

Es gibt mehrere Möglichkeiten festzustellen, ob ein Bild zum Diagramm passt oder nicht. Ein einfaches Vorgehen besteht darin, die Kacheln des Bildes von links nach rechts durchzugehen und immer zu schauen, ob es für zwei nebeneinander liegende Kacheln einen Pfeil im Diagramm gibt.

Man kann aber auch gezielter vorgehen: Man geht die Kacheln im Diagramm nacheinander durch, beginnend mit den Kacheln, von denen nur ein Pfeil ausgeht. Ein Pfeil ist eine Bedingung: A ⟶ B bedeutet „nach Kachel A muss Kachel B kommen“. Dann prüft man, ob das Bild diese Bedingung erfüllt.

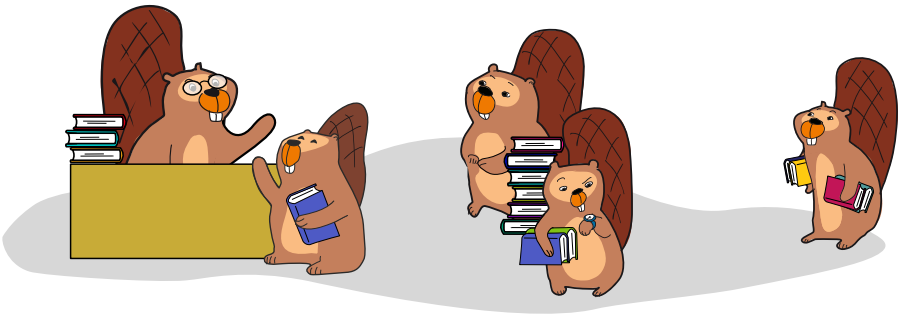
Das ist Informatik!
Einige Computerspiele – wie endlose Läuferspiele – haben einen Hintergrund, der sich horizontal bewegt und die Illusion erzeugt, dass sich der Spieler durch eine Fantasiewelt bewegt. Manchmal ist der Hintergrund kein konstantes Bild, sondern wird automatisch vom Computer erzeugt. Dies nennt man *prozedurale Generierung.* Üblicherweise werden kleinere Objekte kombiniert, um eine große Vielfalt an Hintergründen zu erzeugen.

Die Kombination der Objekte kann jedoch nicht völlig zufällig erfolgen. Sie muss bestimmten Regeln folgen, um Situationen zu vermeiden, mit denen der Spieler nicht zurechtkommt. In der Aufgabe werden diese Regeln durch ein Diagramm aus Kacheln und Pfeilen dargestellt. Solche Diagramme, in denen Objekte durch Pfeile verbunden sind, kennt die Informatik als *gerichtete Graphen* (genauer gesagt: als deren grafische Darstellung). Die Objekte werden dann als Knoten und die Pfeile als gerichtete Kanten bezeichnet. Gerichtete Graphen werden in der Informatik zur Modellierung von Zusammenhängen zwischen Objekten verwendet.

Endliche Automaten gehen noch einen Schritt weiter: Sie haben Zustände, einen Startzustand, Endzustände und Regeln, wie man von einem Zustand zum nächsten wechselt. Das ist ähnlich wie bei einem gerichteten Graphen, nur dass es für spezielle Zustände besondere Knoten gibt. Auch ein solcher Automat kann grafisch beschrieben werden, nämlich durch sein Zustandsübergangsdiagramm. Endliche Automaten werden in der Informatik zur Modellierung einfacher Abläufe oder zur Beschreibung einfach konstruierbarer Folgen eingesetzt – wie den Kachelfolgen in dieser Biberaufgabe.

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Der Nächste bitte!

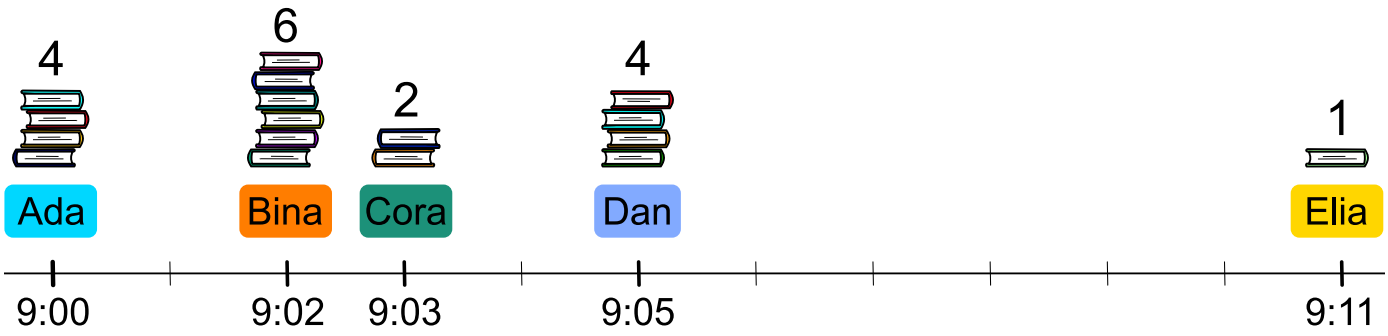


Die Biber in Holzdorf sind fleißige Leserinnen und Leser. In der Bibliothek müssen die Biber deshalb oft warten, wenn sie ihre Bücher zurückgeben wollen.

Wenn ein Biber an der Reihe ist, gibt er alle mitgebrachten Bücher zurück. Die Rückgabe eines Buchs dauert immer genau eine Minute. Ist ein Biber fertig, kommt der nächste Biber aus dem Wartebereich an die Reihe. Das ist immer der Biber mit den wenigsten Büchern.

An einem Morgen kommen nach und nach 5 Biber und wollen ihre Bücher zurückgeben.

Das Bild zeigt für jeden Biber,
- wann er im Wartebereich ankommt und
- wie viele Bücher er mitgebracht hat.



Ada kommt als erste und kann sofort ihre 4 Bücher zurückgeben.

In welcher Reihenfolge geben die Biber ihre Bücher zurück?

Ada Bina Cora Dan Elia

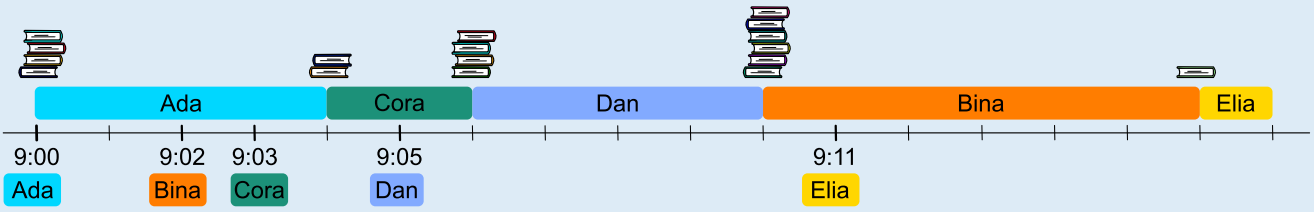
1. ____ 2. ____ 3. ____ 4. ____ 5. ____

Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

So ist es richtig:

1. Ada 2. Cora 3. Dan 4. Bina 5. Elia

Die Abbildung zeigt, wie die Bücher zurückgegeben werden.



- Zuerst ist Ada an der Reihe, um 09:00 Uhr. Es dauert 4 Minuten, bis sie ihre 4 Bücher zurückgegeben hat. Es ist nun 09:04 Uhr. Erst dann kommt der nächste Biber an die Reihe.
- Inzwischen sind Bina und Cora angekommen und warten. Cora hat 2 Bücher und Bina hat 6. Deshalb ist Cora als nächste an der Reihe. Nach 2 Minuten, um 09:06 Uhr, hat sie ihre Bücher zurückgegeben.
- Inzwischen ist Dan gekommen und wartet zusammen mit Bina. Dan hat 4 Bücher und Bina hat 6. Dan hat am wenigsten Bücher und ist als nächster dran. Nach weiteren 4 Minuten, um 9:10 Uhr, ist auch Dan fertig.
- Nun ist endlich Bina an der Reihe. Bina ist um 9:10 Uhr der einzige Biber im Wartebereich, deshalb hat sie nun die wenigsten Bücher.
- Inzwischen ist Elia gekommen. Wenn Bina um 9:16 fertig ist, ist er der einzige Biber im Wartebereich. Er kommt dann als letzter an die Reihe.

Das ist Informatik!
Während ein Computer läuft, müssen viele Aufgaben bearbeitet werden. Für jede Aufgabe wird ein Prozess gestartet, der die Aufgabe bearbeitet. Hier kommt der *Scheduler* ins Spiel. Das ist ein Programm, das zum Betriebssystem eines Computers gehört. Der Scheduler steuert den Ablauf der Prozesse. Er bestimmt, wann und für wie lange ein Prozess von der Zentraleinheit des Computers (CPU) ausgeführt wird. Wenn die CPU gut ausgelastet ist, kann es passieren, dass Prozesse warten müssen. Die Informatik kennt für diesen Fall unterschiedliche Strategien, nach denen Scheduler den nächsten Prozess zur Bearbeitung auswählen; unter anderem:

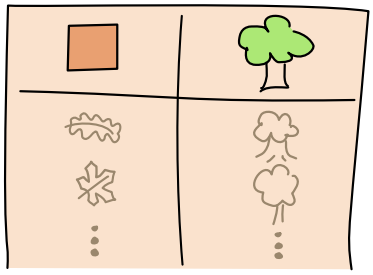
- *Priority Scheduling:* Der Prozess mit der höchsten Priorität wird als nächster (vollständig) ausgeführt.
- *First Come First Serve (FCFS):* Der Prozess, der als erster in die Warteschlange gekommen ist, wird als erster (vollständig) ausgeführt. Nach diesem Prinzip funktionieren die meisten Warteschlangen im Alltag.
- *Round Robin:* Die wartenden Prozesse werden reihum nur für eine bestimmte Zeit ausgeführt. Wenn ein Prozess dann noch nicht fertig ist, wird er unterbrochen und muss wieder an das Ende der Warteschlange. Dann ist der nächste Prozess an der Reihe.
- *Shortest Job First (SJF)*: Der Prozess mit der kürzesten erwarteten Bearbeitungszeit wird als erster (vollständig) ausgeführt.

In dieser Biberaufgabe verwendet die Bibliothek in Holzdorf bei der Buchrückgabe also die Strategie Shortest Job First: Der Biber mit den wenigsten Büchern (und deshalb der kürzesten Bearbeitungszeit) wird als nächster bedient.

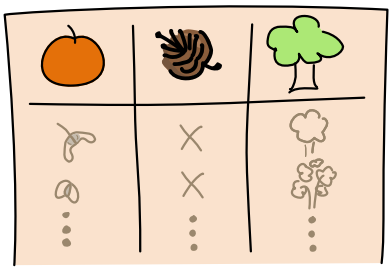
Copyright 2024 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Biberholz

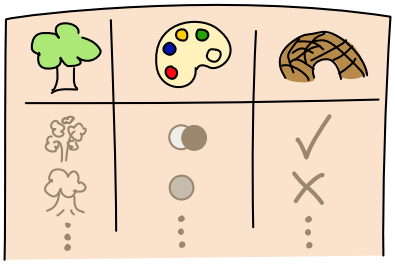
Eddie und seine Freunde gehen gern wandern. Während ihrer Wanderungen sammeln sie Informationen über die Bäume, die sie sehen, und notieren diese in lange Tabellen.



Severin sammelt Information über Blattformen und die zugehörigen Baumarten.



Quirina sammelt Informationen über Baumfrüchte, ob diese von Nadelbäumen stammen und über die zugehörigen Baumarten.



Ladina sammelt Informationen über Baumarten, über deren Holzfarben, und darüber, ob sie Biberholz für Biberburgen liefern.

Eddie hat im Wald ein Blatt gefunden und kennt dessen Form. Nun möchte er erfahren, ob die zugehörige Baumart Biberholz für Biberburgen liefert.

Welche seiner Freunde muss Eddie fragen, und in welcher Reihenfolge, um das zu erfahren?

A) Nur Ladina.
B) Erst Severin, dann Quirina.
C) Erst Severin, dann Ladina.
D) Erst Quirina, dann Severin, dann Ladina.

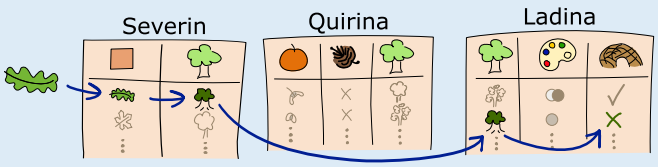
Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0

Antwort C ist richtig:

Die Information darüber, ob eine Baumart Biberholz liefert, findet sich nur in Ladinas Tabelle. Wenn Eddie jedoch nur Information über das Blatt hat, kann er keine Zeile aus Ladinas Tabelle auswählen. Er benötigt Information über die Baumart oder die Farbe des Holzes. Es genügt also nicht, nur Ladina zu fragen; Antwort A ist also falsch.

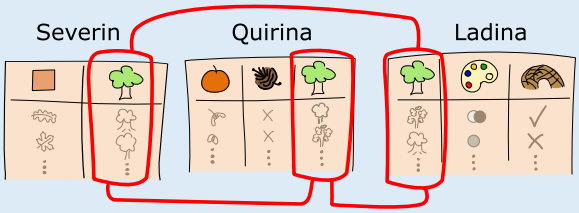
Quirinas Tabelle enthält weder Information über Blätter, noch über Biberholz. Ihre Tabelle nützt Eddie nichts, also sind die Antworten B und D falsch.

Aber Severins Tabelle enthält Information über Blätter. Da Eddie die Form des Blattes kennt, kann er zuerst die passende Zeile in Severins Tabelle auswählen und die fehlende Information über die Baumart erhalten. Anschließend kann er damit die passende Zeile in Ladinas Tabelle auswählen, um die gesuchte Information über das Holz zu erhalten. Wenn Eddie zum Beispiel anhand der Blattform und Severins Tabelle herausfindet, dass es sich um ein Eichenblatt handelt, kann er die passende Zeile in Ladinas Tabelle auswählen und herausfinden, ob Eichen Biberholz liefern.



Das ist Informatik!
Diese Biberaufgabe veranschaulicht grundlegende Konzepte *relationaler Datenbanken.* Diese werden in Informatiksystemen extrem häufig eingesetzt, nämlich zur Verwaltung großer (und auch kleiner) Datenmengen. Relationale Datenbanken bestehen aus Tabellen mit Daten – so wie die von Eddies Freunden erstellten Tabellen. Tabellen heißen auch *Relationen,* und in einer Tabelle ist jede Spalte ein *Attribut* und jede Zeile ein Datensatz bzw. ein Element der durch die Tabelle gegebenen Relation. Die Verbindung zwischen Tabellen über ein gemeinsames Attribut – hier die „Baumart" – entspricht dem Konzept der *Fremdschlüssel* in relationalen Datenbanken, die Beziehungen zwischen verschiedenen Tabellen herstellen.

Eddies Frage nach Information über gutes Bauholz für eine Biberburg anhand der Blattform würde in einem System mit Datenbanken als *Abfrage* bezeichnet. Diese Abfrage erfordert die Verknüpfung mehrerer Tabellen, um die gewünschte Information zu erhalten. Die Verknüpfungsoperation kombiniert Zeilen aus verschiedenen Tabellen anhand des gemeinsamen Schlüssels kurzzeitig zu einer gemeinsamen, größeren Tabelle. So können Daten, die über mehrere Tabellen verteilt sind (in diesem Fall Baumarten species, Blattform shape und Biberholz beavertimber), für die Beantwortung von Abfragen zusammengeführt werden. Die Daten in den Tabellen der Freunde können insbesondere über die Baumart species miteinander verbunden werden:



Relationale Datenbanken sind so wichtig, dass es in der Informatik für Abfragen und andere Operationen auf Datenbanken eine eigene Sprache gibt, nämlich SQL (Structured Query Language).

Copyright 2026 BWINF – GI e.V. Lizenz: CC BY-SA 4.0